Quantitative Automated Reasoning with Satisfiability Modulo Theories

A Thesis

submitted to the

Chennai Mathematical Institute, India

for the degree of

Doctor of Philosophy in

Computer Science

by

Arijit Shaw

August, 2026

Synopsis

Automated reasoning has significantly enhanced the reliability and trustworthiness of modern computing systems. However, existing reasoning tools remain limited when dealing with systems that incorporate randomness or operate under uncertainty. Such settings naturally demand *quantitative* reasoning frameworks capable of providing principled guarantees that account for the underlying uncertainty. Satisfiability Modulo Theories (SMT) has emerged as one of the most powerful and widely used reasoning paradigms. In this thesis, we advance SMT beyond its traditional qualitative boundaries and develop new methodologies that enable *quantitative reasoning* within the SMT framework.

The SMT framework encompasses a rich landscape of theories, each supporting distinct variable types and constraint languages, to enable reasoning across a wide variety of systems. As we extend SMT toward quantitative analysis, the nature of the quantitative queries themselves depends closely on the underlying theory and the application domain. In this thesis, we study the key quantitative questions that surface across SMT theories, especially those that are directly motivated by application needs.

The interplay between SMT theories and real-world applications gives rise to several distinct forms of quantitative analysis. For example, quantitative software verification, automating cryptographic attacks, and bug localization require *counting solutions over bit-vector* domains. Applications such as network reliability under power-flow models and probabilistic model checking mix discrete and continuous variables and ask for counts projected onto discrete variables. Problems such as fairness and robustness quantification for AI systems require volume computation over the continuous domain of linear real arithmetic. In the presence of function specifications, we are often interested in how many different functions can be implemented that satisfy the specifications. *In this thesis, we address these four classes of quantitative questions.*

We begin with the case when the theory is *discrete*, for example, the theory of quantifier-free bit-vectors. In 2014, Hadarean et al. [Had+14] observed that, in many cases, eager bit-blasting is the best approach for SMT bit-vector problems. Prior approaches to counting bit-vector formulas exploited structural properties of the formula. In [SM26], we proposed a tool that leverages eager equi-cardinal bit-blasting, followed by simplification of the formula and leveraging Boolean model counters. With a careful combination of these three components, we achieve 4× performance improvement over the current state-of-the-art for bitvector model counting.

Moving beyond purely discrete settings, the second class of problems arises in *mixed discrete-continuous domains*, requiring counts projected onto discrete variables. In [SM25a], we adapt hashing-based counting, initiated by Chakraborty, Meel, and Vardi [CMV13] and extended to SMT by Chistikov, Dimitrova, and Majumdar [CDM17], to this hybrid setting, yielding a theory-aware reduction that preserves scalability while focusing on the discrete projection. The key idea for scalability for this set of problems lies in using

a correct hash function and employing an efficient solver for hash functions. It shows 5× performance improvement over the current state-of-the-art.

Our third setting deals with quantitative reasoning over *purely continuous domains*, specifically SMT formulas over linear real arithmetic. Here, the quantitative question becomes the *volume* of the feasible region. Prior work by Ge [Ge24b] decomposes the region into *disjoint* polytopes, approximates each polytope's volume, and sums the results, but enforcing disjointness can cause a prohibitive explosion in the number of polytopes. In [SSM25], we address this bottleneck by decomposing into *non-disjoint* polytopes and aggregating via a principled union-of-sets formulation inspired by Chakraborty, Vinodchandran, and Meel [CVM22], reducing region blow-up while retaining accuracy guarantees. Our tool solves 8× more problems than the current state-of-the-art.

Finally, we address quantitative reasoning over the *space of functions*, a setting that pushes beyond variable assignments into functional domains. Functions play a significant part in SMT in terms of uninterpreted functions. Also, beyond satisfiability, SMT is also used for synthesis: given a specification, synthesize a function that satisfies it. The corresponding quantitative question asks how many satisfying functions exist. In [SJM24], we take a first step towards solving this question by counting Boolean specifications. The problem is particularly challenging, as synthesizing even one function is a hard problem. We design an algorithm, SkolemFC, that approximates the number of functions without synthesizing even one function. SkolemFC counts Skolem functions using only $O(n \log n)$ SAT calls, and shows a performance at the scale of synthesizing one function.

All our algorithms come with theoretical guarantees of (ε, δ)-style accuracy. Our algorithms are realized in four user-friendly tools: pact for hybrid discrete-projected counting, ttc for LRA volume, csb for bit-vector counting (including projected, weighted, and sampling variants), and skolemfc for Skolem-function counting. Across standard suites, these tools achieve 5×-10× improvements over prior state of the art. Across these projects, we deliberately employ different algorithmic strategies for closely related problems; a central takeaway, echoing the broader SMT solving experience, is that no single technique dominates all quantitative tasks.

Having developed these counting engines, we demonstrate their practical reach by applying them to *probabilistic model checking*. When a system exhibits probabilistic behavior, the model checking question becomes quantitative: what is the reachability probability in a Markov chain? Modern probabilistic model checkers perform extremely well on many benchmarks, yet as Holtzen et al. [Hol+21] identify, there are classes of surprisingly simple models on which they struggle due to state-space explosion. In [SJM26], we show that such cases admit a natural reduction to bit-vector model counting: the relevant execution paths are encoded as a bitvector formula, and the overall verification task is decomposed into a set of structurally simple model-counting subproblems. Using an extended version of csb [SM26] as the counting backend, our

probabilistic model checking tool MC3 handles up to 32× more parallel processes than existing probabilistic model checkers on a collection of challenging benchmarks.

The SMT tools developed in this thesis are a modest attempt to expand the frontier of quantitative reasoning. Tools were built from some real-world motivation, and once built, they are helping reached further than the problem that inspired it - as the application to probabilistic model checking illustrates. This is, perhaps, a small glimpse of the *virtuous cycle* Cook described: automated reasoning in critical application areas drives investment in foundational tools, while improvements in those tools drive further applications [Coo22]. The four tools presented here are early steps toward a much larger edifice; as the systems we build grow more probabilistic, and more uncertain, we hope this cycle will only spin faster.

List of Publications

Peer-reviewed works included in the thesis:

[SJM24] Arijit Shaw, Brendan Juba, and Kuldeep S. Meel. "An Approximate Skolem Function Counter". In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2024.

[SJM26] Arijit Shaw, Sebastian Junges, and Kuldeep S. Meel. "Covering-based Approximate Model Counting for Statistical Model Checking". In: *Under Submission*. 2026.

[SM25a] Arijit Shaw and Kuldeep S Meel. "Approximate SMT Counting Beyond Discrete Domains". In: *Proceedings of Design Automation Conference (DAC)*. 2025.

[SM26] Arijit Shaw and Kuldeep S Meel. "CSB: A Counting and Sampling Tool for Bitvectors". In: *Acta Informatica*. 2026.

[SSM25] Arijit Shaw, Uddalok Sarkar, and Kuldeep S Meel. "Efficient Volume Computation for SMT Formulas". In: *Proceedings of Knowledge Representation and Reasoning (KR)*. 2025.

Peer-reviewed works published during Ph.D., but not included in the thesis:

[SM24] Arijit Shaw and Kuldeep S. Meel. "Model Counting in the Wild". In: *Proceedings of International Conference on Knowledge Representation and Reasoning (KR)*. 2024.

[Yan+23] Jiong Yang, Arijit Shaw, Teodora Baluta, Mate Soos, and Kuldeep S Meel. "Explaining SAT Solving Using Causal Reasoning". In: *Proceedings of the Theory and Applications of Satisfiability Testing (SAT)*. 2023.

Contents

Part I

Orientation and Unifying Framework

Chapter 1

Introduction

Consider these key questions in computer science:

1. Does my program *always* satisfy a critical safety property?
2. Is my neural network robust to bounded input perturbations?
3. Does there exist an implementation that satisfies my specification?

If we want computers to answer such questions, they must *reason* about programs, learned models, and specifications. This need gives rise to *automated reasoning*, where we develop algorithms and tools that decide whether a formal statement holds. A central workhorse of automated reasoning is *Satisfiability Modulo Theories* (SMT) [BSST21]. SMT provides a uniform language for constraints over rich domains, including bit-vectors, arrays, and arithmetic, and SMT solvers decide whether a given formula is satisfiable. The workflow is simple and powerful: encode the system and the desired property as a formula φ in an appropriate theory and invoke a solver to determine satisfiability, or validity via a related query. This reduction-to-solving paradigm underpins applications in hardware [Mat+18] and software verification [Wie11], as well as neural-network verification [Kat+17; HLZ21], among many others. Its practical impact is driven by decades of solver engineering and by highly optimized systems such as [Bar+22; CGSS13; NP23; BB09], which routinely solve instances far beyond what worst-case complexity might suggest.

A striking commonality in the questions above is that they are *qualitative*: the answer is a single bit, YES or NO. While this is often the right endpoint, it can be an incomplete description of the phenomenon we care about. In modern settings, systems are deployed under uncertainty, inputs are drawn from distributions, environments are stochastic, and specifications compete along multiple dimensions. In these contexts, we often want *quantitative* information:

1. For what fraction of inputs does the program violate the property?

2. In a stochastic system, what is the probability of violating a safety property?
3. For how many inputs within a perturbation region does the network fail to be robust?
4. How large is the space of implementations satisfying the specification?

These questions replace Boolean feasibility with *size*: counting solutions in discrete domains and measuring satisfying sets in continuous ones. This shift enables risk quantification, coverage estimation, probabilistic reasoning, and principled comparison between designs or specifications. It also exposes a core algorithmic gap. Classical SMT technology is primarily optimized for deciding satisfiability, whereas the questions above require reasoning about the cardinality or measure of the satisfying region.

This thesis addresses that gap. We study *quantitative SMT*: given an SMT formula φ, compute the number of satisfying assignments for discrete variables, the volume of satisfying sets for real variables, and related quantitative summaries. Our goal is to develop algorithms that are exact when possible and approximate when necessary, while providing formal guarantees that make the results reliable for downstream tasks such as robustness analysis, probabilistic assessment, and specification evaluation.

Our perspective is informed by a closely related development in the Boolean world. For propositional formulas, the problem of counting satisfying assignments, known as model counting or #SAT, emerged as a natural quantitative analogue of SAT and grew into a mature line of research with strong algorithmic and systems contributions. Quantitative SMT can be viewed as the corresponding lift from SAT to SMT: it generalizes model counting beyond purely Boolean structure to the richer theories that arise in verification, synthesis, and analysis, and it brings counting and volume computation into the SMT pipeline.

1.1 Satisfiability Modulo Theories

Satisfiability Modulo Theories (SMT) extends Boolean satisfiability by incorporating background *theories*. In Boolean satisfiability, all variables are Boolean. In practice, however, variables may range over richer domains: real numbers, integers, or bit-vectors, to name a few. Boolean satisfiability expresses constraints as clauses in conjunctive normal form; richer domains call for richer constraint languages. For real-valued variables, for instance, it is natural to express constraints as conjunctions and disjunctions of linear inequalities.

SMT accommodates this diversity through a uniform framework. One first declares *theory atoms*, which are atomic constraints drawn from the background theory, such as linear inequalities. These atoms are then connected by an arbitrary Boolean formula, forming what is called the *Boolean skeleton* of the SMT formula.

The formal foundation is many-sorted first-order logic, developed in full in Section 2.1. Below, let us have an example to motivate the utility of the framework.

```
; Buses {1,2,3}, directed lines {(1,2),(2,3),(1,3)}
(set-logic QF_LRA)

; Availability (Bool) and flow (Real) variables
(declare-const x12 Bool)  (declare-const f12 Real)
(declare-const x23 Bool)  (declare-const f23 Real)
(declare-const x13 Bool)  (declare-const f13 Real)

; Outage: (not x_ij) => f_ij = 0
(assert (=> (not x12) (= f12 0.0)))
(assert (=> (not x23) (= f23 0.0)))
(assert (=> (not x13) (= f13 0.0)))

; Capacity: x_ij => -c_ij <= f_ij <= c_ij
(assert (=> x12 (and (>= f12 (- 1.0)) (<= f12 1.0))))
(assert (=> x23 (and (>= f23 (- 1.0)) (<= f23 1.0))))
(assert (=> x13 (and (>= f13 (- 0.5)) (<= f13 0.5))))

; Balance: outgoing - incoming = P_i
(assert (= (+ f12 f13)       1.0))   ; bus 1: P1 =  1.0
(assert (= (- f23 f12)  (- 0.3)))   ; bus 2: P2 = -0.3
(assert (= (+ f23 f13)       0.7))   ; bus 3: P3 = -0.7

(check-sat)
(get-model)
```

Listing 1.1: SMT-LIB 2 encoding of F for a three-bus power network.

Example. To see why quantitative SMT arises naturally, consider a simple model of an electrical power network. The network is a graph $G = (V, E)$, where nodes V represent buses and edges E represent transmission lines. Each line $(i, j) \in E$ has a Boolean *availability* variable x_{ij} indicating whether the line is operational, a real-valued *flow* variable f_{ij} measuring the power transmitted, and a capacity bound $c_{ij} > 0$.

The formula F encoding a *valid* network state combines three families of constraints: a line outage ($\neg x_{ij}$) forces its flow to zero; each active line must respect its capacity bounds; and at every bus the net flow must balance the local power injection or demand P_i. Formally:

$$
\begin{aligned}
F \;:=\; & \bigwedge_{(i,j)\in E} \Big(\neg x_{ij} \;\Rightarrow\; f_{ij} = 0\Big) && \text{(outage)}\\
\wedge\; & \bigwedge_{(i,j)\in E} \Big(x_{ij} \;\Rightarrow\; -c_{ij} \le f_{ij} \le c_{ij}\Big) && \text{(capacity)}\\
\wedge\; & \bigwedge_{i\in V} \Big(\textstyle\sum_{j:\,(i,j)\in E} f_{ij} \;-\; \sum_{j:\,(j,i)\in E} f_{ji} \;=\; P_i\Big). && \text{(balance)}
\end{aligned}
$$

Listing 1.1 shows the SMT-LIB 2 encoding of F for a concrete three-bus triangle network, with $c_{12} = c_{23} = 1$, $c_{13} = 0.5$, and injections $P_1 = 1.0$, $P_2 = -0.3$, $P_3 = -0.7$.

(a) x and y are bitvectors.

(b) x and y are real variable.

Figure 1.1: Solution space for same constraints in different theory.

F mixes Boolean variables $\{x_{ij}\}$ with real-valued variables $\{f_{ij}\}$, placing it squarely in the hybrid fragment `QF_LRA` of SMT-LIB. We will revisit this example in Section 2.1.

1.2 Measuring Solution Space

The goal of this thesis is to uplift the satisfiability queries to quantitative queries. Therefore, we define here what we mean by measuring solution spaces. Let's take a set of constraints.

$$x \geq 2$$
$$y \geq 2$$
$$x + y \leq 6$$

Now, how large is the solution space? That totally depends on what type of variable x and y are. If they are integers, then there are 6 solutions (Figure 1.1a). If they are real variables, then the solution space is basically the triangle enclosed by these constraints (Figure 1.1b). The area enclosed is the area of this triangle, which is 2. For higher dimensions, it goes as the Lebesgue measure of n dimensional space. One might also be interested in a hybrid case – say x is an integer variable and y is a real variable, and one wants to know how many assignments to x exists for which the constraints get satisfied. In this case, the number of solutions becomes 3.

Example. Continuing with the network A network operator may ask: *how many distinct line-availability configurations* $\langle x_e \rangle_{e \in E}$ *lead to a state in which some demand is unmet?* This is not a satisfiability question—we do not merely want to know whether such a configuration *exists*, but how many there are, and with what likelihood one arises under random line failures. Formally, letting $X = \{x_e \mid e \in E\}$ be the boolean projection variables, the answer is

$$\left| \{ \sigma|_X \mid \sigma \models F \} \right|,$$

the number of distinct X-assignments that extend to a full model of F—precisely a *projected model count* over a hybrid formula that mixes boolean and real-valued variables. Neither a pure SAT counter (which cannot handle the real flow variables) nor a numerical solver (which returns a single feasible point) answers this question. It calls for quantitative SMT.

1.3 Motivations

Proabapilitic Model Checking When system becomes stochastic, we generally want to state stochastic guarantees about those system. The domain of *probabilistic model checking* exactly this thing. Here, given a Markov chain and a horizon h, one asks for the probability that a target state is reached within h steps. The prevalent approach stores the transition matrix and iteratively propagates step-bounded reachability probabilities through it, but the matrix grows with the state space, and its decision-diagram encoding is bounded below by the number of *distinct* transition probabilities, which is frequently exponential. Holtzen et al. [Hol+21] take the opposite view: instead of marginalising paths out and tracking one probability per state, they aggregate all horizon-length paths that reach the target. Intuitively, the probability of reaching the target is just the total weight of those paths, many of which share prefixes and suffixes that a compact representation can exploit where a probability-per-entry matrix cannot. To compute it, one must sum the weights of all target-reaching paths, which directly reduces to *weighted model counting*. Here, a propositional formula over per-step transition "coins" is built so that each satisfying assignment is exactly one path to the target, each coin is weighted by its transition probability, and the weighted model count of the formula equals the finite-horizon reachability probability.

Network Reliability. Duenas-Osorio et al. [DMPV17] cast network reliability as a propositional model counting problem. Their formalization, however, captures only connectivity constraints and ignores the powerflow constraints that govern real systems. Incorporating powerflow turns reliability estimation into a hybrid SMT counting problem, which fits naturally within the pact framework of Chapter 4. That framework relies on a weighted-to-unweighted reduction [CFMV15]; handling edge-failure probabilities more faithfully therefore motivates a weighted counting framework for hybrid constraints.

Machine Learning Models Verification. SMT solvers are among the most competitive methods for neural network verification [DND25]. Quantitative verification queries, such as robustness and fairness [Bal+19], can be naturally encoded as SMT problems. Notably, Albarghouthi et al. [ADDN17] formulated fairness quantification of programs as a weighted model integration query. This line of work, however, lost momentum as ML models grew rapidly in size, causing the community to retreat from rigorous verification toward empirical fairness testing. With the scalable SMT verification techniques we develop in this thesis, we aim to revive formal fairness verification for models of practical scale.

Cyber-physical systems verification. Autonomous vehicles require verification before deployment, and because they operate under real-world uncertainty, the question is inherently quantitative. A central problem here is sim-to-real validation: checking whether failures found in simulation also occur on real sensor data. Kim et al. [Kim+22] cast one form of this check as an SMT satisfiability query over scene labels, asking whether a labelled scene can be generated by a Scenic scenario program. Their formulation returns a Boolean verdict and cannot attach a confidence. The SMT model counters developed in this thesis lift the query from satisfiability to counting, and so supply the missing quantitative answer.

Robust Reachability. Let's consider the *bug prioritization* in software verification: static analyzers and symbolic execution tools often report many potential bugs, but developers lack a principled way to distinguish critical, easily-triggered bugs from rare, hard-to-reach ones. Girol, Farinier, and Bardin [GFB21] introduce the notion of *robust reachability*: a bug is considered *robustly reachable* if it can be triggered by a large fraction of the input space, not just one specific input. Intuitively, a bug that fires for many inputs is more dangerous and exploitable than one requiring a precise, narrow input value. To quantify this, one must measure *how many* inputs or program paths lead to the buggy location, which directly reduces to SMT model counting. Here, the path condition to reach the buggy location is encoded as an SMT formula over program inputs, and the number of satisfying assignments gives a measure of the bug's robustness. This is reduced to a counting problem over theory of bitvectors.

Automatizing Cryptographic Attacks. In *automated cryptanalysis*, the target is to find an attack in a malleable encryption. Format oracle attack wants to find leaks, for any chosen ciphertext, a single bit indicating whether the decrypted plaintext is well-formed. Exploiting such a leak has traditionally demanded a expert-crafted attack for each new format check. Beck, Zinkus, and Green [BZG20] show how to derive these attacks *automatically*. The attacker keeps the set of plaintexts still consistent with every oracle answer so far, and repeatedly issues the ciphertext query that splits this candidate set most evenly. Intuitively, a query that eliminates half the surviving candidates whichever bit the oracle returns extracts the most information, whereas one that nearly always elicits the same answer teaches little. To choose such a query, one must measure *how many* candidates lie on each side of it, which directly reduces to SMT model counting. Here, the constraint describing the consistent plaintexts is encoded as a bit-vector formula over the unknown message, and the number of satisfying assignments measures how many candidates a query would rule out.

Specification engineering. The first and primary motivation stems from the observation that specification synthesis [ADG16; PFMD21] (i.e., the process of constructing $F(X, Y)$) and function synthesis form part of the iterative process wherein one iteratively modifies specifications based on the functions that are constructed by the underlying engine. In this context, one helpful measure is to determine the number of

possible semantically different functions that satisfy the specification, as often a large number of possible Skolem functions indicates the vagueness of specifications and highlights the need for strengthening the specification. Note that the use of the count is qualitative here, and hence an approximate order of magnitude (log count) suffices.

Diversity at the specification level. In system security and reliability, a classic technique is to generate and use a diverse variety of functionally equivalent implementations of components [BM15]. Although classically, this is achieved by transformations of the code that preserve the function computed, we may also be interested in producing a variety of functions that satisfy a common specification. Unlike transformations on the code, it is not immediately clear whether a specification even admits a diverse collection of functions – indeed, the function may be uniquely defined. Thus, counting the number of such functions is necessary to assess the potential value of taking this approach, and again a rough order of magnitude estimate suffices. Approximate counting of the functions may also be a useful primitive for realizing such an approach.

1.4 Thesis Contributions

This thesis advances *quantitative* SMT. Because SMT spans many theories with very different semantics, we first clarify what it means to ask a "quantitative" question in each setting and give precise problem definitions that cover the main variants of interest. Quantitative SMT sits at the intersection of decision procedures and quantitative reasoning, so progress requires ideas from both communities: SMT provides the solver architectures and theory reasoning, while quantitative methods contribute estimators, hashing/sampling schemes, and geometric techniques that make these problems tractable in practice.

While there is a rich toolbox of techniques for *qualitative* SMT solving, they are not directly applicable to quantitative queries such as counting, weighting, or measuring sets of models. To address these queries, we import and adapt methods from approximate counting, sampling, and polytope volume computation. The resulting techniques are engineered to interoperate with SMT oracles while respecting theory-specific structure. This development mirrors the evolution of SMT itself, where different theories rely on fundamentally different solving paradigms. For example, bit-blasting and lemma-on-demand for bit-vectors, and DPLL(T)-style architectures for other theories with dedicated theory solvers.

We develop algorithms for each major domain considered in the thesis, beginning with discrete domains. Since SMT is a practical discipline, quantitative SMT must also be supported by practical tools; accordingly, we implement the proposed techniques and demonstrate scalability, showing concrete improvements in performance across the domains studied.

As quantitative tools mature, they should enable compelling downstream applications. We demonstrate this by solving problems arising in areas such as *probabilistic model checking*, where, using techniques

developed for bit-vector counting, we analyze large parallel processes that are challenging for baseline approaches.

Summary of Contributions

The thesis contributes a unified algorithmic and systems-oriented account of quantitative SMT. At a high level, the contributions are as follows.

1. *Unifying framework:* We present a single viewpoint in which discrete counting, projection, weights, and continuous volume are quantitative queries over SMT encodings under one umbrella.
2. *Algorithms:* For each query class, we develop algorithms with (ε, δ)-style approximation guarantees.
3. *Tools:* We implement these techniques as easy-to-use tools supporting the corresponding quantitative queries.
4. *Applications:* We apply bit-vector weighted counting to probabilistic model checking by reducing bounded-horizon reachability in DTMCs to weighted model counting.
5. *Evaluation:* We evaluate all tools on standard benchmark families and show substantial performance improvements over prior state-of-the-art approaches while preserving the stated guarantees.

Tools and Artifacts

This thesis is accompanied by *four* open source tools.

Problem	Tool	Codebase	Dataset
Bitvector Count Count	CSB	meelgroup/csb	10.5281/zenodo.20637680
Skolem Function Count	SkolemFC	meelgroup/skolemfc	10.5281/zenodo.10404174
Hybrid Count	pact	meelgroup/pact	10.5281/zenodo.16413909
Volume Computation	TTC	meelgroup/ttc	10.5281/zenodo.16782810

Table 1.1: Tools and Artefacts.

These tools were later integrated into one single tool ttc (meelgroup/ttc). We expect to finish it and complete its documentation before the thesis submission deadline.

1.5 Reader's Guide

The remainder of the thesis is structured to move from foundations to algorithms to applications in Part I, II, III respectively. One way to read the thesis is to read sequentially. Other way to go is to go through Part I first, and then go through any of the chapters in Part II or III.

Chapter Dependencies:

1. Chapters in Part II are independent among themselves, but they depend on the notations used in Part I.
2. Chapter 7 uses some notations introduced in Chapter 2.

If you have time to read only one or two chapters beyond the preliminaries, we recommend starting with Chapter 5 (continuous volume) and/or the application on probabilistic model-checking Chapter 7.

Chapter 2

Preliminaries

The theme of this thesis is to lift SMT from a yes/no decision procedure to a quantitative framework. To this end, this chapter first introduces the syntax and semantics of SMT formulas and theories, and then formalizes solution-space measurement through a measure-theoretic notation for #SMT. We then review SMT-solving techniques and constrained counting methods used throughout the thesis, before discussing SMT counting problems. For SMT notation, we follow the conventions of Chapter 33 of the Handbook of Satisfiability [BSST21]. Readers familiar with the formal preliminaries of SMT may safely skim Section 2.1 and proceed to the quantitative queries in Section 2.2.

2.1 Syntax and Semantics

SMT formulas admit a rich vocabulary of variables, functions, and predicates, drawn from arithmetic, bit-vectors, arrays, and other theories, and combined within a single expression. To reason quantitatively about an SMT formula, we first need a precise account of what the formula *means*: which symbols denote what, which assignments of values count as solutions, and which formulas are equivalent. Mathematical logic is the framework that supplies these answers, and SMT literature adopts it as the foundation [BSST21]. We work in *many-sorted first-order logic with equality*.

The standard route to defining a formula in first-order logic proceeds in four steps:

1. Fix a *signature*, the vocabulary of function and predicate symbols available to us.
2. From this vocabulary, build *terms*, the syntactic objects that name elements of the world.
3. Out of terms, build *formulas*.
4. Give formulas meaning through *models*, which fix a universe of objects and an interpretation of every symbol.

Why many-sorted first-order logic with equality?

Standard first-order logic assigns every variable, function, and predicate a single universe. This is awkward for SMT, where a formula routinely speaks about several disjoint kinds of objects at once: a single hybrid formula may mention a 32-bit register, an integer loop counter, and a real-valued sensor reading. Forcing all three into one universe would require either auxiliary predicates that mark which "kind" each element belongs to, or a separate encoding for every combination of theories, both of which obscure the structure that SMT solvers actually exploit.

Many-sorted first-order logic resolves this cleanly by partitioning the universe into disjoint *sorts*, one per kind of object, and typing every function and predicate symbol by the sorts of its arguments and result. A bit-vector add of width 32 has signature $\mathsf{BV}_{32} \times \mathsf{BV}_{32} \to \mathsf{BV}_{32}$; integer addition has signature $\mathsf{Int} \times \mathsf{Int} \to \mathsf{Int}$; real addition has signature $\mathsf{Real} \times \mathsf{Real} \to \mathsf{Real}$; and the well-formedness rules of the logic prevent us from ever writing a term that mixes a 32-bit register with a real without an explicit conversion.

Equality is built into the logic rather than axiomatized as an ordinary predicate: within each sort, $=$ denotes genuine identity and satisfies reflexivity, symmetry, transitivity, and congruence under every function and predicate symbol. This fixed interpretation is what lets theory solvers propagate consequences of asserted equalities through a formula and exchange information across theories.

We'll focus only on quantifier-free formulas. As a running example, we'll create an SMT formula, which has the solution space as shown in the Figure 2.1.

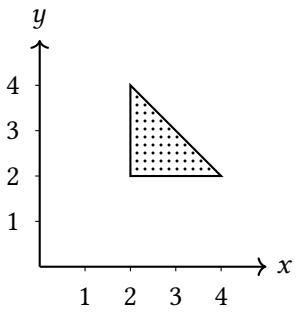


Figure 2.1: Solution space of the running example φ.

Signatures. A *signature* Σ is a set of *predicate* and *function* symbols, each with an associated arity. We write Σ^F for the function symbols of Σ and Σ^P for the predicate symbols. The 0-arity symbols of Σ^F are *constant symbols*; the 0-arity symbols of Σ^P are *propositional symbols*. Constant symbols include both fixed values, and variables.

Example. For a running example, take $\Sigma = \{\mathbb{R}, +, -, \leq, x, y\}$, the signature of linear real arithmetic together with two distinguished free variables. The function symbols are $\Sigma^F = \mathbb{R} \cup \{+, -, x, y\}$: every $r \in \mathbb{R}$ is an arity-0 constant denoting the real r itself, x and y are two further arity-0 constants standing for free real-valued variables, and $+, -$ have arity 2. The only predicate symbol is $\leq$, of arity 2.

We'll use the notation $\mathrm{Vars}(F)$ to denote the *variables* of the formula F.

We use $c, d, \ldots$ for arity-0 constants, $A, B, \ldots$ for propositional symbols, $f, g, \ldots$ for non-constant function symbols, and $p, q, \ldots$ for non-propositional predicate symbols.

Terms and formulas. Once the basic symbols of Σ have been declared, we need rules for combining them into the composite expressions that formulas are built from. There are two such kinds of expression, *terms* and *formulas*, generated by recursive grammars. Terms denote elements of the universe, while formulas make assertions about those elements. The set of terms t and formulas φ is generated by:

$$\begin{aligned} t &::= c \mid f(t_1, \ldots, t_n) \mid \mathit{ite}(\varphi, t_1, t_2), \\ \varphi &::= A \mid p(t_1, \ldots, t_n) \mid t_1 = t_2 \mid \bot \mid \top \mid \neg\varphi_1 \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \rightarrow \varphi_2. \end{aligned}$$

A formula of the form A, $p(t_1, \ldots, t_n)$, or $t_1 = t_2$ is an *atomic formula* or *atom*.

Example. Continuing the example, the application of $+$ to x and y yields the term $x + y$. Combining this with the constants 2 and 6 and applying the predicate $\leq$ produces three atoms, $2 \leq x$, $2 \leq y$, and $x + y \leq 6$. The formula $\varphi \;:=\; (2 \leq x) \wedge (2 \leq y) \wedge (x + y \leq 6)$ is the conjunction of these three atoms.

Models. To say what it means for a formula to be satisfied, we need to fix the set of values that its terms can denote. This set is the *universe* $\mathcal{U}$, a non-empty domain that each theory commits to: linear real arithmetic uses $\mathcal{U} = \mathbb{R}$, a theory of bit-vectors of width w uses $\mathcal{U} = \{0,1\}^w$, and so on. A *model* α for Σ consists of such a universe $\mathcal{U}$ together with an interpretation $(\cdot)^\alpha$ that maps every symbol of Σ to an object of the appropriate kind in A. Concretely, $(\cdot)^\alpha$ assigns:

- to each constant $a \in \Sigma^F$, an element $a^\alpha \in \mathcal{U}$;
- to each n-ary function symbol $f \in \Sigma^F$, a total function $f^\alpha \colon \mathcal{U}^n \to \mathcal{U}$;
- to each propositional symbol $B \in \Sigma^P$, a value $B^\alpha \in \{\bot, \top\}$;
- to each n-ary predicate symbol $p \in \Sigma^P$, a total function $p^\alpha \colon \mathcal{U}^n \to \{\bot, \top\}$.

This mapping extends compositionally to every term and formula.

Example. For the running example, fix the universe $\mathcal{U} = \mathbb{R}$ and interpret each numeral $r \in \mathbb{R}$ as itself, $+$ and $-$ as real addition and subtraction, and $\leq$ as the usual ordering on the reals. The constants x and y are not pinned down by these conventions, so a model of Σ is determined by choosing two real values for them. The model with $x^\alpha = 3$ and $y^\alpha = 2$ satisfies φ, since $2 \leq 3$, $2 \leq 2$, and $3 + 2 = 5 \leq 6$. The model with $x^\alpha = 0$ and $y^\alpha = 0$ does not, since the first conjunct $2 \leq 0$ already fails.

The satisfiability problem. A formula F is *satisfiable* when some model α satisfies F written $\alpha \models F$, when $F^\alpha = \top$ We write

$$\mathrm{Sol}(F) \;=\; \{\alpha : \alpha \models F\}$$

for the set of all models of F, and $\mathsf{Vars}(F)$ for the set of variable-role constants occurring in F. We use $\tau \in \mathsf{Sol}(F)$ to denote a single witness, identifying τ with the restriction of its interpretation $(\cdot)^\tau$ to $\mathsf{Vars}(F)$ when the rest of the model is clear from context. The *satisfiability problem* for a quantifier-free formula F asks whether $\mathsf{Sol}(F) = \emptyset$, and, when nonempty, optionally returns a witness τ.

Models and Satisfiability. The SMT and model-counting literature use the term *model* in different ways. In SMT, a model is an interpretation, or an assignment in the quantifier-free setting. Model's relationship to a formula is determined separately: a model may satisfy or falsify the formula [BSST21]. In contrast, in the model-counting literature, a model means a *satisfying* assignment of the formula [GSS21]. Throughout the thesis, we follow the SMT convention and explicitly refer to *satisfying models* when satisfaction of the formula is intended.

2.1.1 Theories Used in This Thesis

We now collect the specific background theories that arise in our quantitative queries.

Fixed-width bit-vectors (T_{BV}). Each variable carries a width w and ranges over $\{0,1\}^w$. The signature $\Sigma_{BV} = \langle 0, 1, +, -, \cdot, \&, |, \sim, \mathsf{extract}, \mathsf{concat}, \leq \rangle$ is width-indexed throughout, its arithmetic read modulo 2^w. Counting over T_{BV} is developed in Chapter 3, and the probabilistic model checking of Chapter 7 builds on it.

Linear real arithmetic (T_{LRA}). Here the variables range over $\mathbb{R}$, and $\Sigma_{\mathrm{LRA}} = \langle 0, 1, +, -, \{q\cdot\}_{q\in\mathbb{Q}}, \leq \rangle$ confines atoms to linear inequalities $a_1x_1 + \cdots + a_nx_n \circ b$ with rational coefficients and $\circ \in \{<, \leq, =, \geq, >\}$. We call this fragment *LRA*, and take up volume computation over it in Chapter 5.

Uninterpreted functions (T_{UF}). Uninterpreted functions in T_E serve as an abstraction barrier: details of an operation that are irrelevant to the property under analysis can be hidden behind an uninterpreted symbol. As a small illustration, the set $\{a \cdot (f(b) + f(c)) = d,\ b \cdot (f(a) + f(c)) = d,\ a = b\}$ may appear to require arithmetic reasoning, but replacing $+$ and $\cdot$ by uninterpreted binary functions g and h yields a T_E-unsatisfiable set decidable purely by congruence closure.

Other theories. Other than these, there are many other theories, including theory of linear integer arithmetic, strings, arrays, non-linear arithmetic etc. Interested reader can refer to the textbook by Kroening and Strichman [KS16] for a detailed description.

Theory Combinations An interesting capability of the SMT is to *combine* theories – use multiple theories in a single formula. theory at once. Given two theories T_1, T_2 with signatures Σ_1, Σ_2, their *combination* $T_1 \oplus T_2$ is the set of all $(\Sigma_1 \cup \Sigma_2)$-models whose Σ_i-reduct (the model obtained by retaining only the interpretations

of symbols in Σ_i) is isomorphic to a model of T_i. When the signatures are disjoint and each T_i is finitely axiomatized, this coincides with the theory axiomatized by the union of the axiom sets [BSST21].

Discrete and continuous theories. Among the theories above, some of them are *discrete* – like the case of bitvectors and integers – the solution set is countable in this case. In the other hand, theories like real arithmetic is continuous – the solution set is uncountable in this case.

Hybrid SMT formula. We'll use an specific term *hybrid formula* to denote the case where the formula contains both discrete and continuous theory combined.

Logics. Different applications call for different combinations of theories, and reasoning about them requires a common vocabulary. The *many-sorted* framework accommodates this naturally: each theory contributes its own sorts and symbols to a shared signature. The SMT-LIB initiative [BFT16] standardizes this by fixing a family of named *logics*, each specifying which theories, and hence which sorts and symbols, are in scope. A logic is therefore a named combination of theories over this shared signature. We work with the following quantifier-free fragments: QF_BV (bit-vectors), QF_LRA (linear real arithmetic), QF_LIA (linear integer arithmetic), QF_ABV (arrays of bit-vectors), QF_UFBV (uninterpreted functions over bit-vectors), QF_BVFP (bit-vectors and floating point), and the *hybrid* fragment QF_BVLRA that mixes bit-vectors with reals. We call a fragment *hybrid* when it combines at least one discrete theory and at least one continuous theory; the projected counting setting of Chapter **??** is built around exactly this case.

2.2 Measuring Solution Space for SMT

The major task of this thesis is to lift SMT from a yes/no decision problem to a quantitative one. Towards that goal, we first describe the families of quantitative queries that arise across the theories considered in this thesis, then introduce a unified measure-theoretic template that subsumes them.

2.2.1 Quantitative Queries

We can now state the four families of quantitative questions studied in this thesis. In each case, F is a quantifier-free SMT formula over an appropriate background theory and the question is to compute a numerical summary of $\mathsf{Sol}(F)$.

Model counting. When the theory is discrete, then the set $\mathsf{Sol}(F)$ is countable. In this case, the quantitative query is to count the size $|\mathsf{Sol}(F)|$.

Projected model counting. We are often interested in counting over only a subset of the variables. Given a formula F and a subset S of its variables, we write $\alpha_{\downarrow S}$ for the *restriction* of a model α to S: the assignment that keeps the values α gives to the variables in S and discards the rest. The projected solution set is then

$$\mathsf{Sol}(F)_{\downarrow S} \;=\; \{\alpha_{\downarrow S} \,:\, \alpha \in \mathsf{Sol}(F)\},$$

and its size $|\mathsf{Sol}(F)_{\downarrow S}|$ is the projected model count of F over S.

Special case for Projected counting in hybrid theories. In case of hybrid formulas, even though the $\mathsf{Sol}(F)$ may be not countable, $\mathsf{Sol}(F)_{\downarrow S}$ is countable, if S is over only discrete variables.

Volume computation in LRA. Let K be the whole solution space of the given formula F, which has d dimensions. Let $\mathcal{B} \subset \mathbb{R}^n$ be an n-dimensional measurable set. The volume of $\mathcal{B}$ is defined as $\mathsf{Volume}(\mathcal{B}) = \int_{\mathcal{B}} d\mathbf{x}$, where $d\mathbf{x}$ denotes the differential volume element. In Cartesian coordinates, this element is expressed as $d\mathbf{x} = dx_1\, dx_2 \cdots dx_n$. The *volume* of a measurable set $K \subseteq \mathbb{R}^n$ is the Lebesgue integral

$$\mathsf{Volume}(K) \;=\; \int_{\mathbb{R}^n} I_K(x)\, dx,$$

Counting with uninterpreted functions. When F contains uninterpreted function symbols $g_1, \ldots, g_k$, a model is no longer determined by a variable assignment alone: it must additionally fix, for each g_i, a total function g_i^α of the appropriate sort. An element of $\mathsf{Sol}(F)$ is therefore a pair consisting of a variable assignment together with a tuple $(g_1^\alpha, \ldots, g_k^\alpha)$, and $\mathsf{Sol}(F)$ lies in the product of these two factors. Two natural counting queries arise, one for each projection:

1. Projecting onto the variable-assignment factor recovers the ordinary projected count: the number of assignments τ for which *some* tuple of symbol interpretations completes τ into a model of F. This treats the UF symbols as existentially abstracted auxiliaries.
2. Projecting onto the symbol-interpretation factor gives the number of tuples $(g_1^\alpha, \ldots, g_k^\alpha)$ that occur in some model of F. This is *uninterpreted-function counting*, and is the relevant query in settings such as SyGuS, where one wants to gauge how *tight* a specification is by counting its admissible implementations.

The two projections live on very different scales. With n Boolean inputs and m Boolean outputs, the variable-assignment factor has size 2^{n+m}, while the symbol-interpretation factor has size $\left(2^{2^n}\right)^m = 2^{m \cdot 2^n}$, one tuple per choice of Boolean function for each UF symbol. Existential abstraction therefore collapses a *doubly* exponential candidate space into a *singly* exponential one, and this gap is the source of the difficulty that distinguishes uninterpreted-function counting from ordinary projected counting. In this thesis we focus on a specific instance of the second query: counting Skolem functions.

Counting Skolem functions. Let $F(X, Y)$ be a Boolean specification over inputs $X = \{x_1, \ldots, x_n\}$ and outputs $Y = \langle y_1, \ldots, y_m \rangle$. A *Skolem function vector* for F is a tuple $\Psi(X) = \langle \psi_1(X), \ldots, \psi_m(X) \rangle$ realizing the existential quantification of the outputs:

$$\exists Y\, F(X, Y) \;\equiv\; F(X, \Psi(X)).$$

We write $\mathsf{Skolem}(F, Y)$ for the set of all such vectors, identifying any two that agree on every input assignment for which F is satisfiable. The *Skolem function counting* query asks for $|\mathsf{Skolem}(F, Y)|$, or, when the count is too large to manipulate directly, for $\log |\mathsf{Skolem}(F, Y)|$. This is a quantitative query over higher-order objects, functions rather than assignments, and serves as our canonical example of counting in functional domains.

Weighted Model Counting. An extension of model counting is weighted model counting, where for each model α, we assign an weight $w(\alpha)$. The weighted model count, therefore is defined as

$$w(F) = \sum_{\alpha \in \mathsf{Sol}(F)} w(\alpha)$$

2.2.2 Approximation Guarantees

All four queries above are intractable in the worst case, so most of this thesis is concerned with approximation. The guarantees we provide are uniformly of (ε, δ) form, parameterized by a tolerance $\varepsilon \in (0, 1]$ and a confidence $\delta \in (0, 1]$.

For (projected) model counting, an (ε, δ)-*approximate counter* is a randomized algorithm that, on input (F, S), returns a value c satisfying

$$\Pr\left[(1-\varepsilon)\,|\mathsf{Sol}(F)_{\downarrow S}| \;\le\; c \;\le\; (1+\varepsilon)\,|\mathsf{Sol}(F)_{\downarrow S}|\right] \;\ge\; 1-\delta.$$

For volume computation, the analogous guarantee is

$$\Pr\left[(1-\varepsilon)\,\mathsf{Volume}(\mathsf{Sol}(F)) \;\le\; \widehat{V} \;\le\; (1+\varepsilon)\,\mathsf{Volume}(\mathsf{Sol}(F))\right] \;\ge\; 1-\delta,$$

where $\widehat{V}$ is the algorithm's estimate. These guarantees are standard in approximate counting [CMV21].

NP Oracles and SAT oracles. Given a Boolean formula F, an NP oracle determines the satisfiability of the formula. A SAT solver is a practical tool solving the problem of satisfiability. Following definition of

Delannoy and Meel [DM22], a SAT oracle takes in a formula F and returns a satisfying assignment σ if F is satisfiable and $\perp$ otherwise. The SAT oracle model captures the behavior of the modern SAT solvers. In some of the chapters, we'll analyze the runtime of the algorithms with respect to oracles.

FPRAS. A Fully Polynomial Randomized Approximation Scheme (FPRAS) is a randomized algorithm that, for any fixed $\varepsilon > 0$ and any fixed probability $\delta > 0$, produces an answer that is within a factor of $(1 + \varepsilon)$ of the correct answer, and does so with probability at least $(1 - \delta)$, in polynomial time with respect to the size of the input, $1/\varepsilon$, and $\log(1/\delta)$.

2.3 A Unified Template for Quantitative SMT

The diversity of theories and their combinations in SMT makes a single quantitative definition unwieldy. We follow Chistikov, Dimitrova, and Majumdar [CDM17] in adopting the language of *measured theories*: a formula's quantitative content is captured by equipping its model space with a measure and asking for the measure of the satisfying set. The four query families of Section 2.2 are recovered as instances of this template, distinguished only by which symbols of the formula are treated as the object of measurement.

Setup. Recall from Section 2.1 that a Σ-model α specifies, in particular,

- an element $a^\alpha \in A$ for each 0-arity constant $a \in \Sigma^F$, and
- a total function $f^\alpha \colon A^n \to A$ for each n-ary function symbol $f \in \Sigma^F$.

A theory T pins down the interpretation of some symbols (e.g. $+$ and $\leq$ in $T_{\mathbb{R}}$) and leaves the rest free; we call the free symbols of F its *parameters*. As noted in Section 2.1, we routinely identify a model $\tau \in \mathrm{Sol}(F)$ with its restriction to the parameters of F. Each parameter contributes a factor to the space in which this restriction lives:

- a 0-arity constant of sort D contributes a factor D;
- an n-ary function symbol of sort $D_{\text{in}} \to D_{\text{out}}$ contributes a factor $D_{\text{out}}^{D_{\text{in}}}$.

Each query family of this thesis arises by choosing a tuple S of parameters of F, equipping each corresponding factor with a measure, and measuring the projection of $\mathrm{Sol}(F)$ onto its S-factors.

Definition 2.3.1 (Measured theory)**.** Let T be a Σ-theory, F a quantifier-free Σ-formula, and S a tuple of parameters of F. Equip the factor associated with each $s \in S$ with a measure (counting, Lebesgue, or otherwise), and let $(\Omega_S, \mathcal{F}_S, \mu_S)$ be the resulting product measure space. Write $\mathrm{Sol}_S(F) \subseteq \Omega_S$ for the projection of $\mathrm{Sol}(F)$ onto its S-factors. We say T is *measured at* S if $\mathrm{Sol}_S(F) \in \mathcal{F}_S$ for every F.

A brief introduction to Measure Theory

To formalize the concept of "size" (such as length, area, or volume), *measure theory* defines *which sets can be measured* and *how to measure them.*

The first concept is of σ-algebra, which formalizes the collection of *measurable sets.* Measurable sets are the specific subsets within a domain to which we can formally assign a "size". The complete collection of all measurable sets for a given space forms an abstract mathematical structure known as a σ-algebra.

Definition 2.3.2 (σ-Algebra)**.** A σ-algebra is a collection of subsets $\mathcal{F}$ of a space Ω that contains the empty set and is closed under complementation and countable unions.

Definition 2.3.3 (Measure)**.** A measure is a function from the collection of measurable sets (σ-algebra) to the non-negative real numbers (including $+\infty$) that satisfies the following properties:

1. The measure of the empty set is zero.
2. The measure of a countable union of pairwise disjoint sets is the sum of the measures of the individual parts.

Definition 2.3.4 (Measure Space)**.** A measure space consists of three objects (X, A, μ), where X is any set, A is a σ-algebra on X, and μ is a measure.

Definition 2.3.5 (Borel σ-algebra)**.** Let X be a topological space. The *Borel σ-algebra* on X, denoted $\mathcal{B}(X)$, is the smallest σ-algebra on X that contains every open set of X. Equivalently, $\mathcal{B}(X)$ is the σ-algebra generated by the collection of open sets of X, i.e., $\mathcal{B}(X) = \sigma(\{U \subseteq X : U \text{ is open}\})$.

Definition 2.3.6 (Lebesgue measure on $\mathbb{R}^d$)**.** The d-dimensional Lebesgue measure λ^d on $\mathbb{R}^d$ is the standard notion of d-dimensional *volume*: it assigns to each axis-aligned rectangle (box) $R = \prod_{i=1}^{d}(a_i, b_i)$ the value $\lambda^d(R) = \prod_{i=1}^{d}(b_i - a_i)$, and it extends this assignment in a countably additive way to a large collection of sets.

One can treat λ^d as a measure on the Borel σ-algebra $\mathcal{B}(\mathbb{R}^d)$: it is a function $\lambda^d : \mathcal{B}(\mathbb{R}^d) \to [0, \infty]$ such that $\lambda^d(\varnothing) = 0$ and for any pairwise disjoint sequence $(A_i)_{i\geq 1} \subseteq \mathcal{B}(\mathbb{R}^d)$ we have $\lambda^d(\bigcup_{i\geq 1} A_i) = \sum_{i\geq 1} \lambda^d(A_i)$, and it agrees with the usual volume on boxes, i.e., $\lambda^d(\prod_{i=1}^{d}(a_i, b_i)) = \prod_{i=1}^{d}(b_i - a_i)$. In particular, λ^d assigns 0 volume to points (and more generally to lower-dimensional sets), e.g., $\lambda^d(\{x\}) = 0$, and $(\mathbb{R}^d, \mathcal{B}(\mathbb{R}^d), \lambda^d)$ forms a measure space in which Borel sets are measurable and λ^d is the volume function.

Definition 2.3.7 (#SMT)**.** We define the measure of an SMT formula F, #SMT of F at S is

$$\#\mathsf{SMT}_S(F) \;:=\; \mu_S\big(\mathsf{Sol}_S(F)\big).$$

The associated quantitative SMT problem $\#\mathsf{SMT}[T, S]$ is: given F over T, compute $\mathsf{Count}_S(F)$.

The four query families of this thesis instantiate Definition 2.3.1 by their choice of S and per-factor measures.

Remark 1 (Discrete counting)**.** Take $S = \mathsf{Vars}(F)$ with each x_i of finite sort, and equip each factor with the counting measure. Then μ_S is counting measure on $\prod_{x\in\mathsf{Vars}(F)} D_x$, and $\mathsf{Count}_S(F) = |\mathsf{Sol}(F)|$ recovers ordinary discrete model counting.

Remark 2 (Volume)**.** Take $S = \mathsf{Vars}(F)$ with each x_i of real sort, and equip each factor with the Lebesgue measure. Then μ_S is the $|S|$-dimensional Lebesgue measure on $\mathbb{R}^{|S|}$, and $\mathsf{Count}_S(F)$ is the volume of the satisfying region.

Remark 3 (Projected counting)**.** Take $S \subsetneq \mathsf{Vars}(F)$; the remaining variable-role constants are absorbed by the projection $\mathsf{Sol}(F) \to \Omega_S$. Per-factor measures are chosen as in Remark 1 or Remark 2 according to sort. In the discrete case $\mathsf{Count}_S(F)$ is the projected model count; in the continuous case it is the volume of the projection.

Remark 4 (Skolem counting)**.** Let $F = F(X, Y)$ with $X = \{x_1, \ldots, x_n\}$ and $Y = \{y_1, \ldots, y_m\}$. Replace each output y_j by a fresh n-ary function symbol f_j of sort $D_X \to D_{y_j}$, and take $S = \{f_1, \ldots, f_m\}$. Each factor $D_{y_j}^{D_X}$ carries the counting measure (in the finite case). The satisfying condition for a tuple $(f_1^\alpha, \ldots, f_m^\alpha) \in \Omega_S$ is

$$\forall a \in D_X\colon \quad \big(\exists b \in D_Y.\ F[X \mapsto a,\ Y \mapsto b]\big) \implies F\big[X \mapsto a,\ Y \mapsto f^\alpha(a)\big],$$

so $\mathsf{Sol}_S(F) = \mathsf{Sk}(F)$ as defined in Section 2.2, and $\mathsf{Count}_S(F) = |\mathsf{Sk}(F)|$.

First-order vs. higher-order parameters. Remarks 1–3 measure 0-arity factors of a Σ-model—its variable-role-constant layer. Remark 4 measures n-ary factors—its function-symbol layer. Both layers are already part of any Σ-model under Section 2.1; the unified template simply chooses which to vary. The doubly-exponential cardinality gap noted in Section 2.2 is the size difference between the two: with n Boolean inputs and m Boolean outputs, the 0-arity factor for Y has size 2^m, while the n-ary factor for $f_1, \ldots, f_m$ has size $2^{m\cdot 2^n}$.

2.4 Background on Solving Techniques

2.4.1 Paradigms for SMT solving

Before we dig into different techniques for measuring solution space of SMT, let's have a look into how SMT decision procedure works. We'll heavily use these techniques and extend them in our solution space measurement system.

Boolean abstraction. A crucial idea in SMT solving is Boolean abstraction. SMT solvers treat the propositional skeleton of a formula and its theory atoms separately. To formalize this, we associate with Σ a propositional signature Ω that contains every propositional symbol of Σ together with one fresh propositional symbol for each non-propositional ground atom. The *propositional abstraction* is a bijection T2B from quantifier-free formulas to propositional formulas over Ω that is the identity on propositional symbols of Σ, maps each non-propositional atom to its associated fresh propositional symbol, and is homomorphic with respect to the Boolean connectives. We write B2T for its inverse, abbreviate $\mathsf{T2B}(F)$ as F^p, and lift the notation to sets of literals by writing $\mu^p = \{\ell^p : \ell \in \mu\}$. The Boolean abstraction is one-sided: for any theory T,

$$\mu \models_p F \implies \mu \models_T F, \qquad \text{but not the converse,}$$

where $\models_p$ denotes propositional entailment via the abstraction. For example, with $F = (x \le 0) \wedge (x \ge 0) \wedge (x = 1)$ over $T_{\mathbb{R}}$, the abstraction $F^p = p_1 \wedge p_2 \wedge p_3$ is propositionally satisfiable, yet the underlying theory literals are jointly $T_{\mathbb{R}}$-inconsistent. This asymmetry, which prevents propositional reasoning from settling theory questions on its own, is the engine of every modern SMT solver and reappears repeatedly in this thesis.

Eager and lazy approaches. The target for SMT was to extend Boolean satisfiability to *theories*. In this regard, two broad approaches exist [BSST21], eager and lazy. The *eager* approach translates the input formula in one shot to an equisatisfiable propositional formula and invokes a SAT solver on that. The *lazy* approach abstracts out some part of the formula and refines that as formula gets complicated. The eager approach is prevelant in solving theory of bit-vectors and floating points, while the lazy approach is prevelant in most others. Let us discuss the key paradigms here.

CDCL(T): This is the most dominant paradigm for SMT solving, used by solvers such as z3 [DB08] and cvc5 [Bar+22] proposed by Tinelli [Tin02]. The approach lifts the CDCL procedure from propositional SAT solving to the theory level by combining a SAT solver with one or more dedicated *theory solvers*. Given an SMT formula φ over some background theory $\mathcal{T}$, the solver first constructs a *Boolean abstraction* $\hat{\varphi}$ by

replacing each theory atom with a fresh propositional variable. The CDCL engine searches for a satisfying assignment to $\hat{\varphi}$; if one is found, the corresponding set of theory literals is passed to the theory solver to determine whether the assignment is consistent in $\mathcal{T}$. If it is, a satisfying model has been found and the solver returns SAT. If the theory solver finds that the concrete form of the model to $\hat{\varphi}$ is not satisfiable by the theory constraints, it produces a *theory conflict clause*, which is then added to the Boolean formula as a learned clause. The CDCL engine resumes its search under this additional constraint. This loop continues until either a $\mathcal{T}$-consistent model is found or the Boolean abstraction becomes unsatisfiable, in which case the solver returns UNSAT.

Eager and Lemmas-on-Demand: For solving bit-vector and floating-point formulas, eager approach often dominates. The SMT solver STP [GD07] is the first solver to implement this paradigm. The key idea is to simplify the formula and In later SMT solvers like Boolector [BB09] and its successor Bitwuzla [NP23], this appraoch is refined by the idea of abstraction to a different setting called *lemmas on demand*. Rather than constructing a purely propositional abstraction of the input formula φ, these solvers construct a *bit-vector abstraction* $\alpha(\varphi)$ by abstracting away theory atoms from theories such as arrays and uninterpreted functions as fresh bit-vector or Boolean constants, while retaining the bit-vector structure of the formula directly. An eager bit-vector solver then solves to $\alpha(\varphi)$. If one is found, dedicated theory solvers check whether the candidate model is consistent with the abstracted constraints. If an inconsistency is detected, the theory solver generates a *lemma*. A lemma is a clause that rules out the current inconsistent assignment. This lemma is added to $\alpha(\varphi)$, and the abstraction-refinement loop restarts to find a new candidate model under the tightened abstraction.

Model Constructing Satisfiability (MCSAT): This paradigm, introduced by Jovanovic, Barrett, and De Moura [JBD13], fundamentally departs from the Boolean abstraction approach. Rather than separating Boolean reasoning from theory reasoning, MCSAT interleaves them by constructing a *model* directly over the original theory variables. The solver maintains a partial assignment over both Boolean and theory variables simultaneously. When a conflict arises, it performs *theory-aware* conflict analysis and clause learning, producing explanations expressed directly in the theory language. This tighter integration allows MCSAT to handle non-linear arithmetic and other expressive theories more naturally than CDCL(T), since it avoids the semantic mismatch that can arise when Boolean abstractions fail to capture the structure of the underlying theory.

Other Approaches. Some other SMT solvers take radically different approach. STP simplifies the formula, passes it to a SAT formula. Q3B creates a BDD compilation and solves that for quantified boolean formulas.

2.4.2 Tools for Counting

In different constrained counting scenario, over the years, a couple of different techniques has been developed. We can classify them into roughly five different categories.

1. **Decision diagrams.** The core idea is to *compile* the solution space into a tractable symbolic representation, generally decision diagram. Now, counting reduces to a dynamic program over the compiled object. Classic examples include binary decision diagrams (BDDs) [Bry86] and related variants that aim to represent the satisfying assignments compactly while preserving exactness [DM02]. Once compiled, model counting is typically linear in the size of the diagram. The main limitation is that compilation can blow up in the worst case; nevertheless, for instances with exploitable structure, decision-diagram based techniques can be extremely effective and have been used extensively in CNF counting [Dar01; LLM16; KJ21; SM25b].
2. **Generating functions.** In this paradigm, one constructs an algebraic object like generating function, whose coefficients encode the number of feasible solutions of different types. Counting is then performed by extracting coefficients or evaluating the function at particular points. A prominent application is counting *lattice points in polytopes*, where rational generating functions provide a compact encoding of the integer points and enable efficient counting in fixed dimension; see, e.g., Barvinok [Bar94].

The above two methods give exact counts, while in many of the cases we are happy with approximate counts. Below we describe some of the approximate counting techniques.

3. **Monte Carlo estimators.** Constrained counting is often done by simulations – sampling uniformly from an covering set and checking how many of that elements satisfy the constraints. The central problem arises when the covering body is exponentially larger than the constrained set. There are multiple techniques to handle the exponential difference case.
4. **Telescoping products.** One such technique to handle exponential larger bodies is to build *telescoping* product. The idea is to build set of bodies which are related to each other and not very different. The foundational trick by Jerrum, Valiant, and Vazirani [JVV86] is to write the count as a product of ratios that are each bounded away from zero. Fix variables one at a time, forming nested solution sets $S = S_0 \supseteq S_1 \supseteq \cdots \supseteq S_n = \{\text{trivial}\}$, and write $\cdot \prod_i \frac{|S_{i-1}|}{|S_i|}$. Each ratio is typically in a manageable range, so it can be estimated to good relative accuracy by sampling from the smaller set and checking membership in the larger one. This is extensively used in *polytope volume computation*.
5. **MCMC Methods.** Markov chains are often used for such purposes. Where instead of sampling points randomly and checking whether they belong inside a body, the idea is to start with an initial point, making random moves starting from that chain. Many counting problems can be phrased as estimating the measure of a union of sets (e.g., counting satisfying assignments of a DNF formula as

the size of the union of its terms). Coverage-based estimators build unbiased (or low-bias) Monte Carlo estimates by sampling from component sets and correcting by the *coverage* (how many sets contain the sampled element). This line of work underlies classical randomized approximation schemes for DNF counting and related problems; a canonical reference is Karp, Luby, and Madras [KLM89]. The practical attractiveness of this paradigm is that it can succeed with relatively weak oracles (sampling from individual components plus membership tests), but variance control is often the central technical challenge.

6. **Hashing-based counters.** Hashing-based methods use (approximately) k-wise independent hash functions to randomly partition the solution space into many *cells* (typically by conjoining random XOR/parity constraints), and then estimate the global count from the number of solutions in a randomly chosen cell. Informally, if the partition is "balanced," then $|\mathrm{Sol}(\varphi)| \approx 2^m \cdot |\mathrm{Sol}(\varphi \wedge h(b) = \mathbf{b})|$ for a random hash h with m constraints. Algorithmically, one searches for a partition granularity at which cells are small enough to be counted (or bounded) by an NP/SAT/SMT oracle, while concentration guarantees yield (ε, δ)-style approximations. This paradigm has been instantiated for a range of discrete counting tasks (including CNF and DNF variants, as well as streaming/sketch-like settings) and forms the backbone of many modern approximate model counters.

Two interesting applications of by MC methods.

Polytope Volume Computation has been a central focus in computational geometry since [DF88] proved it #P-hard, followed by Dyer, Frieze, and Kannan's FPRAS development (1991). Rigorous algorithmic advances have improved complexity bounds from $\widetilde{O}(n^{23})$ to $\widetilde{O}(n^3)$ [AK91; KLS97; LV06; LD12; CV18][a]. Though these algorithms contain large hidden constants, practical implementations have been achieved by carefully relaxing theoretical guarantees [CV16; CF21].

Estimating the size of a set union has been a well-studied problem since the work of Karp and Luby (1983), who proposed a $O(m \log^2 |\Omega|)$-time algorithm, where m denotes the number of sets and $|\Omega|$ represents the universe size. Subsequent research has extensively explored streaming settings, where sets arrive sequentially over time [FM85; GT01; KNW10]. More recent methods employ sampling-based strategies in the context of streaming algorithms [MVC21], which have also been adapted for DNF counting [Soo+24]. These approaches achieve a $\widetilde{O}(nm)$-time complexity for estimating the solution count of a DNF formula, where m is the number of clauses and n is the number of variables in the DNF formula. Our work builds upon and extends techniques developed in the DNF counting framework.

[a] $\widetilde{O}(\cdot)$ hides the polylogarithmic factors in $O(\cdot)$.

2.5 Related Work

Counting in Bit-vector The success of propositional model counters, especially approximate ones, led to the adaptation of these techniques for word-level constraints. Chistikov, Dimitrova, and Majumdar [CDM15] extended hashing-based model counting from the approximate CNF model counter ApproxMC [CMV13] to word-level benchmarks using bit-blasting. Chakraborty et al. [CMMV16] developed SMTApproxMC, which lifts hash functions to handle word-level constraints. Kim et al. introduced a system for statistical estimation to continuously refine the probabilistic estimate of model counts, although it lacks (ε, δ)-guarantees [KM18].

Projected Couning in Hybrid Domains The success of propositional model counters, particularly approximate model counters, prompted efforts to extend the techniques to word-level constraints. Chistikov et al.[CDM15] used bit-blasting to extend the propositional model counting technique to word-level benchmarks. Chakraborty et al.[CMMV16] designed SMTApproxMC by lifting the hash functions for word-level constraints. Kim and McCamant[KM18] designed a system to estimate model count of bitvector formulas. Ge et al. [Ge+18] developed a probabilistic polynomial-time model counting algorithm for bit-vector problems, and also developed a series of algorithms in the context of related SMT theories to compute or estimate [GB21; GMZ18; Ge+19] the number of solutions for linear integer arithmetic constraints. A closely related problem in the hybrid domain of Boolean and rational variables is *Weighted Model Integration* (WMI) [BPV15], which involves computing the volume given the weight density over the entire domain. Extensive research addresses WMI through techniques such as predicate abstraction and All-SMT [MPS17; MPS19], as well as methods leveraging knowledge compilation [Kol+18].

Volume Computation in LRA *SMT Volume Computation* was first addressed by Ma, Liu, and Zhang, who developed the vinci tool [MLZ09] for exact volume computation. Their approach performs intelligent disjoint decomposition of the solution space into convex polytopes using a *bunching* strategy, followed by exact computation of individual polytope volumes. Later, [GMZZ18] designed polyvest, which extended vinci by incorporating MCMC-based techniques for polytope volume computation. Recently, Ge introduced the SharpSMT tool [Ge24b], which integrates and optimizes various techniques from previous work of polyvest and vinci [GMZZ18; MLZ09] to create a more comprehensive solution for SMT volume computation. Bringmann and Friedrich [BF10] proposed a method for computing the volume of the union of polytopes, building on the union algorithm of [KLM89]. Abboud, Ceylan, and Dimitrov [ACD20; ACD22] extended this approach to the WMI problem under a DNF representation.

Weighted Model Integration (WMI) [BPV15] represents a closely related problem in the hybrid domain of Boolean and rational variables, involving the computation of volume given weight density over the entire domain. This area has seen significant research advances through diverse approaches, including

predicate abstraction and All-SMT techniques [MPS17; MPS19], knowledge compilation methods [Kol+18], and structurally-aware algorithms [Spa+22; Spa+24]. However, while WMI focuses primarily on computing weighted integration across domains, our work specifically addresses the fundamental problem of determining the exact volume of the solution space.

Counting Functions. The specifications for the functions are expressed in terms of quantified Boolean formulas (QBFs). The problem of counting the number of solutions in true QBF formulas has been studied in different definitions. [BELM12] defined the problem of AllQBF - finding all assignments of the free variables of a given QBF such that the formula evaluates to true. CountingQBF [SMKS22] poses a similar query - counting the number of assignments of the free variables of a given QBF such that the formula evaluates to true. However, their relevance to counting functions isn't clear. In more recent work, [PMS23] investigated the problem of *level-2 solution counting* for QBF formulas and developed an enumeration-based approach for counting the number of level-2 solutions for ∀∃-QBFs. In this work, the solutions are expressed as tree models, and for true formulas, the number of *different* tree models is the same as the number of Skolem functions.

Part II

Algorithms for different SMT counting problems

Chapter 3

Discrete Domains: Counting Solutions for Bit-vectors

The theory of bitvectors is among the most widely used theories in SMT, arising naturally from the way modern computers represent and manipulate information. For satisfiability over bitvector formulas, the dominant approach is *bit-blasting* — reducing the formula to Conjunctive Normal Form (CNF) and delegating to an off-the-shelf SAT solver. However, this pipeline has seen limited adoption in the model *counting* setting, where existing techniques instead exploit structural properties of the formula or apply hash-based methods directly. In this chapter, we investigate whether modern CNF model counters can serve as a foundation for extending SMT solvers to support counting over bitvectors. The main contribution of this chapter is csb, an efficient tool for exact and approximate (projected) model counting over the theory of bit-vectors. The tool, csb, operates by bit-blasting the bit-vector formula into CNF, then invoking an off-the-shelf CNF model counter to perform counting. Central to this pipeline is a model-preserving conversion that guarantees the model counts of the original SMT formula and the resulting CNF formula coincide.

The theory of bit-vectors is among the most widely used theories in SMT, arising naturally from the way modern computers represent and manipulate information, making it prevalent in hardware and software verification tasks [Bar+05; NPZ12; HJ19; Lei10]. The question of model counting over bit-vectors arises in various contexts, including quantitative verification of software [GFB21; TW21] and automating cryptographic attacks [BZG20]. We start the technical contribution of the thesis by developing a model counting system for this domain.

Model counting over bit-vectors is particularly challenging for two reasons. First, bit-vectors succinctly represent large domains, making even the decision problem NEXP-complete [KFB16]. For example, a variable x of bit-width 1024 has 2^{1024} possible values. Second, bit-vector arithmetic admits complex non-linear constraints — multiplication, division, modular arithmetic, shifts — that create highly irregular solution spaces, resistant to the decomposition and symmetry exploitation that make counting tractable in simpler theories.

For satisfiability over bit-vector formulas, the dominant approach is *bit-blasting* — reducing the formula to Conjunctive Normal Form (CNF) and delegating to an off-the-shelf SAT solver [BB09; NP23; GD07]. Modern solvers employ various simplification and abstraction operations before passing the formula to the SAT solver. The problem of counting over bit-vectors can be addressed through two methods: (i) reasoning directly over bit-vectors, or (ii) reducing the problem to CNF. While the former has been studied using lifting Boolean model counting techniques for word-level constraints [CDM15; CMMV16; KM18], the latter has not been thoroughly assessed – despite bit-blasting being the standard approach for satisfiability.

A naive pipeline of bit-blasting followed by CNF model counting is insufficient for two reasons. First, the bit-blasting procedures in SMT solvers are equisatisfiable but not model-preserving: the solution counts of the original formula and the resulting CNF need not coincide. Second, bit-blasting introduces a large number of auxiliary variables – encoding internal circuit structure – that have no correspondence to the original bit-vector variables, inflating the search space of the model counter and degrading performance. Both issues must be addressed to achieve a correct and efficient system.

The main contribution of this chapter is handling all these issues an present a tool csb, for exact and approximate model counting over the theory of bit-vectors. csb operates by bit-blasting the bit-vector formula into CNF via a model-preserving transformation, then invokes an off-the-shelf CNF model counter with . Built on top of the SMT solver STP, csb integrates the approximate model counter ApproxMC and the exact model counter Ganak. We evaluate csb on 661 benchmarks drawn from cryptography and software verification; csb successfully counts 661, against 111 for the previous state-of-the-art SMTApproxMC. Specifically, this chapter contributes:

1. csb, a bit-vector model counter integrating CNF-based exact and approximate counting into the SMT solver STP, outperforming SMTApproxMC across 661 benchmarks.
2. An extension to projected model counting, supporting both exact and approximate variants.

The rest of the chapter is organized as follows: Section 3.2 presents the csb framework; Section 3.4 describes experimental results; Section 3.5 concludes.

3.1 Preliminaries and Problem Statement

This chapter studies quantitative reasoning for the theory T_{BV} of quantifier-free fixed-width bit-vectors introduced in Chapter 1. We adopt the notation of Chapter **??** throughout: for a bit-vector formula F over $X = \mathrm{Vars}(F)$ of word-level variable-role constants, $\mathrm{Sol}(F)$ denotes the set of satisfying models, and $\mathrm{Sol}(F)_{\downarrow S}$ its restriction to a designated subset $S \subseteq X$.

The quantitative queries of interest are instances of Definition 2.3.1 with counting measure on each factor: the non-projected count $|\mathrm{Sol}(F)|$ and the projected count $|\mathrm{Sol}(F)_{\downarrow S}|$. An (ε, δ)-*approximate (projected) counter* takes $(F, S, \varepsilon, \delta)$ and returns c satisfying

$$\Pr\left[\, \frac{|\mathrm{Sol}(F)_{\downarrow S}|}{1+\varepsilon} \;\le\; c \;\le\; (1+\varepsilon)\,|\mathrm{Sol}(F)_{\downarrow S}| \,\right] \;\ge\; 1-\delta;$$

when $S = X$ this is non-projected approximate counting. We use the phrase *non-projected model counting* for the case $S = X$ whenever contrast with $S \subsetneq X$ matters.

Independent support. Several preprocessing techniques for bit-vector model counting are organized around computing a small subset of variables whose values determine the rest of any solution's projection. We capture this with the following definition.

Definition 3.1.1 (Independent support)**.** Let F be a quantifier-free bit-vector formula over X, and let $S \subseteq X$. A subset $\mathcal{I} \subseteq S$ is an *independent support* of S if for all $\tau_1, \tau_2 \in \mathrm{Sol}(F)$,

$$\tau_{1\downarrow \mathcal{I}} \;=\; \tau_{2\downarrow \mathcal{I}} \;\Longrightarrow\; \tau_{1\downarrow S} \;=\; \tau_{2\downarrow S}.$$

Intuitively, an independent support of S is a subset of variables whose values pin down the S-restriction of any solution. Small independent supports enable preprocessing that reduces counting on F to counting on a formula over a smaller variable set [LLM16; SM19].

Why bit-vector counting is hard. A defining feature of T_{BV} is that short formulas can have enormous solution counts. Consider three variables x, y, z each of bit-width 1024 and the formula

$$\varphi \;:=\; (x < y) \;\wedge\; (z = x \cdot y).$$

Although φ is small as a syntactic object, $|\mathrm{Sol}(\varphi)| \approx 2^{2046}$, exponential in the bit-width. Enumeration-based exact counting is hopeless in this regime, which motivates the (ε, δ) relaxation above and the algorithmic developments in the remainder of this chapter.

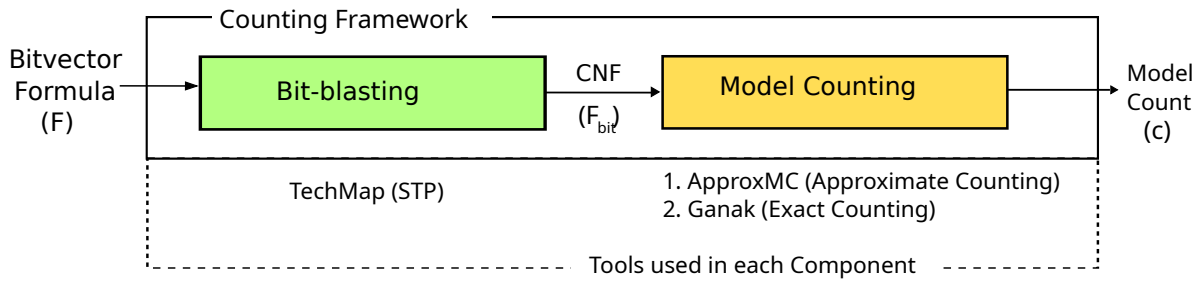


Figure 3.1: The architecture for counting in csb.

3.2 Equi-Cardinal Bit-blasting Pipeline

We develop csb, a model counting tool for the quantifier-free fragment of the theory of bit-vectors. To achieve this, csb first bit-blasts the formula to a Boolean CNF formula, then applies an off-the-shelf CNF model counter or CNF sampler to solve the problems of model counting , respectively. By reducing the problem to CNF, our tool csb leverages ongoing improvements in propositional model counting.

As shown in Figure 3.1, the model counting part of csb comprises two primary components: (i) a tool for bit-blasting SMT2 formulas into CNFs and (ii) an off-the-shelf model counter.

Phase 1: Bit-blasting The bit-blasting component takes an input bit-vector formula F and outputs a CNF formula F_{bit} such that $|\text{Sol}(F_{\text{bit}})| = |\text{Sol}(F)|$. The most commonly used method for converting a bit-vector formula into a propositional Boolean formula involves representing the formula as an And-Inverter Graph (AIG) circuit, which is subsequently converted into conjunctive normal form (CNF). There are multiple techniques available for converting an AIG to CNF, with the two most commonly used being: (i) TseitinEnc: the standard Tseitin encoding [Tse83] and (ii) TechMap: technology mapping-based logic synthesis [EMS07]. The TseitinEnc method uses auxiliary variables for sub-circuits of AIG, whereas the TechMap method performs various optimizations on the circuit to produce a minimized CNF. From the perspective of satisfiability, studies have revealed that the performance of the encoding scheme is reliant on the benchmark set being investigated, with TseitinEnc exhibiting superior results in some benchmark sets and TechMap being more effective in others [JLS09]. In the context of csb, TseitinEnc and TechMap showed similar performance, and we use TechMap for bit-blasting.

A subset of bits in F_{bit} directly corresponds to the actual bits of the formula F, while the rest are auxiliary variables required for encoding constraints. These auxiliary variables are not part of the independent support of F_{bit}. Approximate model counters and samplers achieve better performance when provided with smaller independent support. During bit-blasting, csb identifies and tracks the variables belonging to the independent support versus auxiliary variables, relaying this information to the model counter.

Phase 2: Propositional Model Counting The second phase of csb involves a propositional model counter that takes the CNF formula F_{bit} and returns c, the value of $|\text{Sol}(F_{\text{bit}})|$, or an approximation of that. We investigated various approaches to model counting, including both compilation-based counters and hashing-based approximate counters. In recent years, there has been a proliferation of model counters, and the annual model counting competitions have demonstrated that different model counters have distinct strengths and weaknesses for various problem types. In the context of this research, we examine the applicability of five model counters within the framework of csb: four compilation-based exact counters SharpSAT-TD [KJ21], Ganak [SRSM19], D4 [LM17], and GPMC; and a hashing-based approximate counter ApproxMC [SM19]. In csb, we use ApproxMC as the model counter because it shows the best performance on the test instances. Upon comparing the performances of different CNF counters, we chose the best CNF counters for exact and approximate counting. In the exact model counting mode of csb, we use the latest version of Ganak, while in the approximate model counting mode, we use ApproxMC.

3.2.1 Extensions for Projection

The second class of problems csb solves is projected model counting. The architecture is the same as non-projected counting, but there are a few key differences.

Phase 1: Bit-blasting The bit-blasting component in the projected model counting mode of csb is similar to the non-projected model counting mode. We use TechMap for bit-blasting. In addition to that, it keeps track of which variables to keep projections on. On the blasted CNF, the variables are marked as projection variables. The CNF counting problem, therefore, represents a projected model counting problem. Given a bitvector formula F, and a projection set $P \subset \text{Vars}(F)$, csb detects $P_{\text{bit}} \subset \text{Vars}(F_{\text{bit}})$ from the bitblasted formula F_{bit}, such that $\text{Sol}(F_{\text{bit}})_{\downarrow P_{\text{bit}}} = \text{Sol}(F)_{\downarrow P}$. The resultant problem becomes a projected CNF counting problem of formula F_{bit}, projected on P_{bit}.

Phase 2: Propositional Projected Model Counting The CNF F_{bit} along with the projection variables P_{bit} generated in the previous phase are passed to the projected model counter. In this context, we experimented with four projected model counters Ganak [SRSM19], D4 [LM17], GPMC, and ApproxMC [SM19]. Upon comparing the performances of different CNF counters, we chose the best CNF counters for exact and approximate projected CNF counting. Similar to non-projected counting, in the exact model counting mode of csb, we use the latest version of Ganak, while in the approximate model counting mode, we use ApproxMC.

3.2.2 Implementation

A simple approach to build the tool is to use a standalone shell script that interfaces with an SMT solver to generate a CNF file, followed by processing with a model counter or sampler. However, using shell scripts could lead to performance bottlenecks due to the overhead of file read-writes, complicate error handling, and create challenges in deployment and portability by requiring specific environmental setups and external binaries. Therefore, we chose a tightly integrated approach, embedding the counter and samplers directly within the SMT solver as a library. This integration yields a single binary that efficiently produces both model counts and samples. We implement csb using STP as the underlying SMT solver. We integrate ApproxMC to implement the approximate model counter, Ganak to integrate the exact model counter. Moreover, we integrate Arjun as a preprocessor. The default parameters for the approximate model counting mode are $\varepsilon = 0.8$ and $\delta = 0.2$, adhering to the standard values used in the model counting community.

Input format Since projection is irrelevant in the context of satisfiability, SMT-Lib [BST+10] does not incorporate the concept of projection. To address this, we propose an extended format for problem input. Variables can be designated as projection variables using the `proj-var` keyword. Each `proj-var` command can include one or more variables and multiple `proj-var` commands are supported. Projection variables can be declared at any point in the file, provided they are specified after the variable declaration and before the `check-sat` command. This format is analogous to the one used in propositional model counting [FHS24b]. An example bitvector formula with projection variables is presented in Figure 3.2.

3.3 Correctness and Guarantees

Given the two-phase structure of csb, correctness of the end-to-end pipeline requires that each phase satisfy its specification. The expected property from the second phase (CNF counting) is: given a Boolean formula, count (or sample uniformly) from its satisfying assignments. The model counters and samplers we use have been extensively validated in the literature [SM25b; Cha+15; CM19] and in the Model Counting Competition [HFS25]; we therefore treat CNF counting correctness as a trusted component.

Thus, our primary concern is the correctness of the BV→CNF bit-blasting pipeline. We establish that this pipeline is model preserving, i.e., the satisfying assignments of the input BV formula F are in one-to-one correspondence with the satisfying assignments of the resulting CNF F_{bit}. We now review the toolchain to justify this property. Recall that the BV→CNF pipeline comprises two stages: (i) BV→AIG and (ii) AIG→CNF. For the two steps, we rely on the following model-preservation properties.

(i) *BV→AIG:* Given a BV formula F, this step produces an AIG F_{aig}, which has primary inputs corresponding to the bits of the input variables of F. The solution spaces of F and F_{aig} coincide over

```
(set-logic QF_BV)

(declare-const a (_ BitVec 6))
(declare-const b (_ BitVec 6))
(declare-const c (_ BitVec 6))
(declare-const d (_ BitVec 6))

(proj-var a b)
(proj-var d)

(assert (bvult a #b001010))
(assert (bvult b #b011110))
(assert (= (bvadd c d) #b001000))

(check-sat)
```

Figure 3.2: Example SMT formula with projection variables

these primary inputs. The AIG construction is compositional in the structure of the BV formula: Boolean connectives are translated into equivalent and/not gate structure, and bit-vector operations are implemented using standard textbook circuits. We manually inspected each per-operator circuit to confirm semantic preservation. By compositionality, F_{aig} and F therefore admit the same models.

(ii) *AIG→CNF:* Our toolchain uses technology-mapping-based CNF generation [EMS07]: it applies function-preserving rewrites to the AIG F_{aig}, and then emits a CNF F_{bit} whose auxiliary variables are constrained by bi-implications. Consequently, every satisfying input assignment of F_{aig} has a unique extension to a satisfying assignment of F_{bit}; equivalently, $\text{Sol}(()F_{\text{aig}})$ is in bijection with the projection of $\text{Sol}(F_{\text{bit}})$ onto the input variables.

3.4 Experimental Evaluation

The evaluation procedure was conducted using the following setup. We conducted all our experiments on a high-performance computer cluster, with each node consisting of Intel Xeon Gold 6248 CPUs. We set the memory limit to 16 GB for all configurations and ran each solver instance on a single core. To adhere to the standard timeout used in model counting competitions, we set the timeout for all experiments to 3600 seconds. [1]

Baseline For the problem of model counting, we compare our performance with the prior state-of-the-art bit-vector model counter, SMTApproxMC. As noted in the related works, there are no available projected model counters or uniform samplers for bit-vectors, so we evaluate the relative performance of both of them based on the total number of benchmarks.

[1] Benchmarks and dataset available at: https://bit.ly/csb-dataset

Benchmarks We collected a substantial set of benchmarks that pertain to the problem of model counting and naturally encode them into bit-vector formulas. The benchmarks were produced using several software tools for various purposes. These include CounterSharp [TW21], which is a quantitative software reliability estimation tool; an algorithm designed for testing robust reachability [GFB21]; Delphinium, which is a cryptographic tool for automating chosen ciphertext attacks [BZG20]; and SMC, previous work on bit-vector counting [KM18]. The total number of benchmarks used in our evaluation amounts to 661. For projected model counting, we used the same set of benchmarks with the addition of projection variables. In the case of CounterSharp benchmarks, the projection variables were predefined within the benchmarks. For the remaining benchmarks, we randomly selected 30% of all variables as projection variables.

With this setup, in this work, we sought to answer the following questions:

RQ1. How does csb's performance compare with the existing state-of-the-art in different problems?
RQ2. How do the various components of csb, such as model counting and bit-blasting, impact its overall performance?

Summary of Results In model counting mode, csb can solve over four times more instances than SMTApproxMC. Out of a total of 661 instances, csb-approx can count 661, csb-exact can count 418; whereas SMTApproxMC is able to count 111. In projected counting mode, csb-exact solves 643 instances, and csb-approx solves 659 instances.

RQ1. Comparison with Prior State-of-the-Art

As csb solves three completely different problems, we compare the performance in the following three subsections.

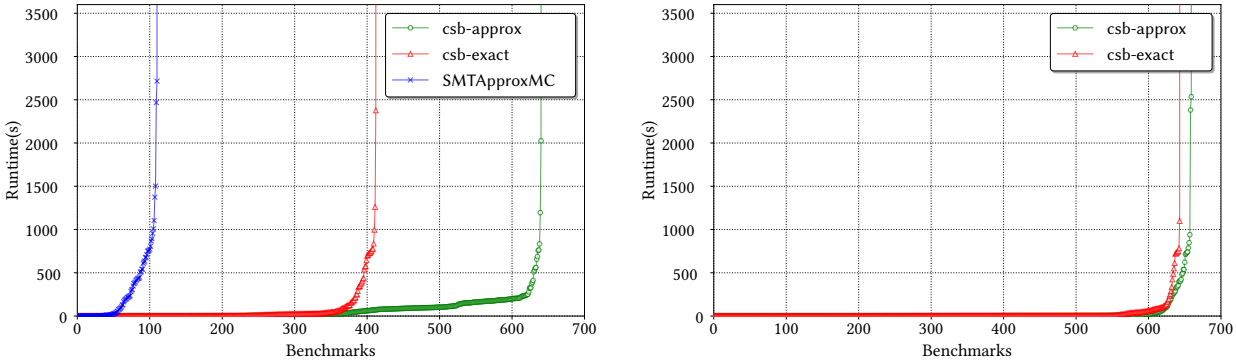
Problem 1: Model Counting

We experimented with configurations for csb and compared its performance with SMTApproxMC, the current state-of-the-art approximate bit-vector model counter. Table 3.1 shows the improvement in performance demonstrated by csb. In approximate mode, csb counted 661 instances out of 661, which is six times the number of instances counted by SMTApproxMC, which could count 111 instances. Even in exact mode, csb performs better than SMTApproxMC, which is an approximate counter. The average Runtime[2] of csb is also great. In the cactus plot in Figure 3.3a, we compare the performance. The x-axis represents the number of instances, while the y-axis shows the time taken. A point (i, j) in the plot indicates that a counter counted j benchmarks out of the total benchmarks in a set in less than or equal to i seconds. The

[2]We use the PAR-2 score as a proxy for average runtime. PAR-2 score (penalized average runtime) assigns a runtime of two times the time limit for each benchmark not solved by a counter.

Counter	Instances Counted	Average Runtime (s)
SMTApproxMC	111	5973.6
csb-exact	418	2795.9
csb-approx	**661**	**368.3**

Table 3.1: Performance comparison on 661 non-projected instances.



(a) Non-projected counting: csb vs. SMTApproxMC

(b) Projected counting: two modes of csb

Figure 3.3: Performance of csb.

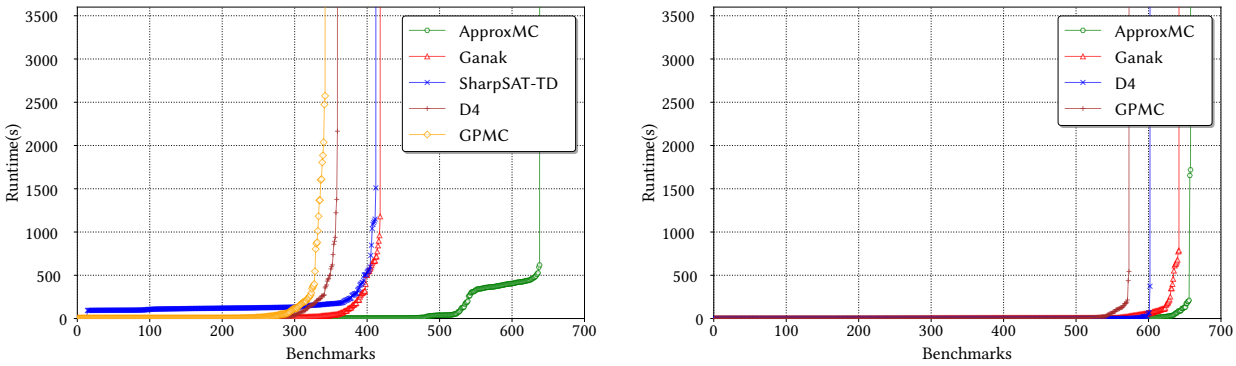
cactus plot reveals that while SMTApproxMC struggles to solve more than 111 instance, csb solves more than 600 instances, each with less than 500s.

Problem 2: Projected Model Counting

As there is no existing uniform bit-vector projected model counting tool, we do not have a baseline for comparing csb against. We count the number of instances solved by the approximate and exact mode of csb out of 661 instances. As Table 3.2 indicates, in approximate mode csb solves 659 instances: almost all the benchmark set consisting of 661 instances. The performance in exact mode is also quite close, where csb solves 643 instances. Moreover, the cactus plot in Figure 3.4b indicates that in both modes, csb solves 600 instances in a few seconds.

Counter	Instances Counted	Average Runtime (s)
csb-exact	643	301.4
csb-approx	**659**	**140.2**

Table 3.2: Performance comparison on 661 projected instances.



(a) Non-projected instances.

(b) Projected instances.

Figure 3.4: Performance comparison of model counters.

RQ2. Impact of Different Components

As shown in Section 3.2, there are multiple components for csb, and there are multiple tools/algorithms available for each component. To determine which tools to use for each component, we performed experiments. Below, we summarize the impact of each component.

Impact of CNF Counters in non-projected counting To find the best counters to use in csb, we selected solvers from the 2024 Model Counting Competition [FHS24a] for comparison. By combining each model counter with its best possible preprocessing settings, we observed that ApproxMC delivered the best performance, solving 661 out of 661 benchmarks. The second-best performance was achieved by Ganak, which solved 418 benchmarks. The other exact counters — SharpSAT-TD, D4, and GPMC — performed similarly to Ganak, solving 414, 359, and 342 instances, respectively. As shown in the cactus plot in Figure 3.3a, ApproxMC outperforms all other counters significantly, while the exact counters exhibit comparable performance.

Impact of CNF Counters in projected counting Similar to the non-projected case, we evaluate projected CNF counters on the projected CNF counting instances generated by csb. The counter SharpSAT-TD was excluded from this evaluation as it does not support projected model counting. ApproxMC achieved the best performance, solving 659 instances. Among the exact counters, Ganak performed the best, solving 643 instances. The other counters showed similar performance: D4 solved 602 instances, and GPMC solved 573 instances. The cactus plot in Figure 3.4b shows that, unlike the non-projected case, the performance gap between ApproxMC and the exact counters is minimal, with all counters performing comparably.

Impact of bit-blasting method The performance of csb remains relatively unchanged when the bit-blasting technique is switched between TseitinEnc and TechMap in csb. However, if Arjun is not employed

as a preprocessing technique, using TseitinEnc as a bit-blasting technique instead of TechMap can improve the performance of csb. This improvement is observed across all CNF-counters.

Impact of Preprocessing The preprocessors had a minimal positive impact on the performance of exact model counting tools, and in a few cases, they even showed a slight negative effect. For the approximate model counting tool ApproxMC, Arjun demonstrated a positive impact, enabling the solving of 643 instances, which is 473 more than without preprocessing.

3.5 Summary

This work introduced csb, an extension of the SMT solver STP that utilizes CNF counters to efficiently address the counting problem over bit-vector formulas efficiently, demonstrating strong performance on a large application benchmark set. The tool csb supports exact and approximate projected and non-projected model counting, marking the first solution for exact counting, and projected counting on bit-vector formulas. Since its inception, the tool has been used both in industry [Ter26] and academia [AFT26] alike.

Chapter 4

Hybrid Domains: Counting beyond Discrete Domains

A particularly challenging class of SMT counting problems arises when formulas span multiple theories, combining discrete and continuous variables, yet the count is required only over a discrete subset of variables, such as bitvectors and Booleans. Such instances arise naturally in applications including network reliability with powerflow constraints, probabilistic model checking, and cyber-physical systems verification. Despite their practical importance, no existing tool addresses this problem directly, and bit-blasting-based approaches are not expected to scale to these instances. In this chapter, we bridge this gap by introducing pact, a projected SMT model counter for hybrid formulas. The pact algorithm employs hashing-based approximate model counting to estimate solution counts with formal theoretical guarantees, making only a logarithmic number of SMT solver calls relative to the number of projection variables via optimized hash functions.

SMT solvers are widely employed for applications that require reasoning over both continuous and discrete variables. For example, to encode hybrid systems in cyber-physical systems or planning, it is essential to have both variables. The existing work in SMT counting is unable to handle SMT formulas with continuous variables.

In this work, we seek to remedy the aforementioned situation. In particular, we focus on a large class of SMT formulas, which we refer to as *hybrid SMT formulas*. The hybrid SMT formulas are defined over both discrete and continuous variables, and we are interested in solutions projected over discrete variables. Our investigations for the development of counting techniques for hybrid SMT formulas are motivated by

their ability to model several interesting and relevant applications, such as robustness quantification of cyber-physical systems and counting reachable paths in software (for detailed discussion, see Section **??**).

The primary contribution of this work is the development of a model counting tool, pact, for efficient projected counting of hybrid SMT formulas. The framework approximates the model count with (ε, δ) guarantees. pact supports SMT formulas with various theories, including linear and non-linear real numbers, floating-point arithmetic, arrays, bit-vectors, or any combination thereof. The projection variables are over bit-vectors. pact employs a hashing-based approximate model counting technique, utilizing various hash functions such as multiply-mod-prime, multiply-shift, and XOR. The algorithm makes $O(\log(|S|))$ calls to the SMT oracle, where S is the set of projection variables. We have implemented a user-friendly tool based on CVC5, which will be released post-publication. pact supports a diverse array of theories including QF_ABV, QF_BVFP, QF_UFBV, QF_ABVFPLRA, QF_ABVFP, QF_BVFPLRA.

To demonstrate runtime efficiency, we conduct an extensive empirical evaluation over 14,202 benchmark instances. Out of these 14,202 instances, pact successfully finished on 603 instances, while Baseline could finish only on 13 instances. Importantly, Baseline fails on counts above 3,570, while pact handles instances with over 1.7×10^{19} solutions.

4.1 Preliminaries and Problem Statement

Hash functions. A *hash function* $h : U \to [m]$ maps elements from a universe U to a range $[m] = \{0, 1, \ldots, m-1\}$. A *pairwise independent hash function* is a hash function chosen from a *family* $\mathcal{H}$ of functions $h : U \to [m]$ such that, for any two distinct elements $x_1, x_2 \in U$ and for any $i_1, i_2 \in [m]$: $\Pr[h(x_1) = i_1 \wedge h(x_2) = i_2] = 1/m^2$. A *vector hash function* $h : U^d \to [m]$ extends this concept by mapping d-dimensional vectors of w-bit integers to the range $[m]$. In this paper, the term *hash function* refers to *hash-based constraint*, represented as $h(\mathbf{x}) = \alpha$. The solutions of the formula $F \wedge (h(\mathbf{x}) = \alpha)$ form a subset of the solutions to F, restricted to those where the hash function maps to α.

Problem Statement. We shall introduce the projected counting problem, defined by the specific theory of the formula and the projection variables.

Definition 4.1.1 (Count$\mathcal{T}_{\downarrow\mathcal{P}}$(F,S))**.** Given a logical formula F defined over the SMT theory $\mathcal{T} \cup \mathcal{P}$; and a projection set S on theory $\mathcal{P}$; where $\mathcal{T}$ is either discrete or continuous or combination of both, and $\mathcal{P}$ is a discrete theory, Count$\mathcal{T}_{\downarrow\mathcal{P}}$(F, S) refers to the problem of counting $|\mathsf{Sol}(F)_{\downarrow S}|$.

In this work, we consider BV as $\mathcal{P}$. Any possible theory or combination of theories can serve as $\mathcal{T}$. Consequently, the resulting counting problems take forms such as CountBVFPLRA$_{\downarrow\mathsf{BV}}$, CountBVFP$_{\downarrow\mathsf{BV}}$, and similar variations.

4.2 Hash Families and Solving

The choice of hash function is one of the most important parts in a hashing-based model counter like pact. In pact, we experiment with three different pairwise independent vector hash functions, which have been used in different literature.

- *Multiply mod prime* ($\mathcal{H}_{prime}$) [Tho15]: For a prime number p, and independent random values $\mathbf{a} = a_0, \ldots, a_{d-1}, b \in [p]$, the hash function from $[p]^d$ to $[p]$ is given by:

$$h_{\mathbf{a},b}(\mathbf{x}) \equiv \left(\sum_{i \in [d]} a_i x_i + b \right) \bmod p = \alpha$$

- *Multiply-shift* ($\mathcal{H}_{shift}$) [Die96]: For independent random values $\mathbf{a} = a_0, \ldots, a_{d-1}, b \in [2^{\overline{w}}]$, where $\overline{w} \geq w + \ell - 1$ the hash function from $[2^w]^d$ to $[2^\ell]$ is given by:

$$h_{\mathbf{a},b}(\mathbf{x}) \equiv \left(\sum_{i \in [d]} a_i x_i + b \right) [\overline{w} - \ell, \overline{w}) = \alpha$$

- *Bitwise XOR* ($\mathcal{H}_{xor}$) [CW77]: This is a particular case of the multiply mod prime scheme when $p = 2$.

$$h_{\mathbf{a}}(\mathbf{x}) \equiv \bigoplus_{\{i \mid a_i = 1\}} x_i = \alpha$$

Given the type of hash function to generate, the procedure GenerateHash generates the constraints of the form $h_{\mathbf{a},b} = \alpha$, where α is a randomly chosen element from the domain of the hash function. When this hash function-based constraint H is added to the input formula F, $F \wedge H$ satisfies only those solutions of H for which the hashed value by $H_{\mathbf{a},b}$ is α. therefore, the number of solutions of $F \wedge H$ is approximately p^{th} fraction of number of solution of F. $\mathcal{H}_{prime}$ and $\mathcal{H}_{shift}$ are word-level hash functions that allow us to choose a number of partitions we want to divide our solution space into. When pact uses these hash functions, along with the projection set S, and the family, GenerateHash takes a parameter ℓ, and generates a hash function with domain size p, such that $2^\ell \leq p < 2^{\ell+1}$. Specifically, in $\mathcal{H}_{shift}$, $p = 2^\ell$, in $\mathcal{H}_{prime}$, p is the smallest prime $> 2^\ell$. In case of $\mathcal{H}_{xor}$, $\ell = 1, p = 2$. As S is evident from the context, and $\mathbf{a}, b$ are randomly generated, we use the notation h instead of $h(S)$ to denote the hash functions.

While each of the hashing constraints partitions the solution space into p slices, we often want to partition it into more. To partition the solution space into p^c cells, we use the Cartesian product of c hash functions: $\mathcal{H} \times \mathcal{H} \times \cdots \times \mathcal{H}$. We use the notation $H_{[i]}$ to denote the Cartesian product of the first $i+1$ hash functions, i.e., $H_{[i]} = h_0 \times h_1 \times \cdots \times h_i$.

Slicing. In GenerateHash subroutine of pact, the hash functions have particular domain sizes. But, the bitvectors can have arbitrary width; we *slice* them into bitvectors of smaller width so that the value of (sliced) bitvector lies within the domain of the hash function. Instead of defining hash functions on the variables from S, we define hash functions on *slices* of the variables, defined as follows: For a bitvector $\mathbf{x}$ of width w, we define $\lceil w/\ell \rceil$ slices of width ℓ: $x(\lceil w/\ell \rceil - 1), \ldots, x(1), x(0)$, where $x(i) = x\,[(i+1)\ell - 1 : i\ell]$.

4.3 Algorithm and Analysis

We introduce pact, our tool for approximate counting of SMT formulas. It processes a formula F, a set of projection variables S, a *tolerance* ε, and a *confidence* δ to produce an approximation of $|\mathsf{Sol}(F)_{\downarrow S}|$ within the desired tolerance and confidence. The main idea behind pact involves dividing the solution space into equally sized *cells* using hash functions and then enumerating the solutions within each *cell*.

Algorithm 1 pact($F, S, \varepsilon, \delta$, family)

```
1:  L ← ∅, it ← 0
2:  thresh, numIt, ℓ ← GetConstants(ε, δ, family)
3:  C[0] ← SaturatingCounter(F, S, thresh)
4:  if C[0] ≠ ⊤ then return C[0]
5:  while it < numIt do
6:      C ← ∅, countFound ← ⊥, i ← 0
7:      H ← GenerateHash(S, ℓ, family)
8:      while countFound = ⊥ do
9:          i ← NextIndex(C, i)
10:         C[i] ← SaturatingCounter(F ∧ H_[i], P, thresh)
11:         if C[i] < thresh ∧ C[i − 1] = ⊤ then
12:             C′, H′ ← FixLastHash(F, S, C, H, i, ℓ)
13:             L.append(GetCount(C′[i], H′))
14:             countFound ← ⊤, it++
15: return FindMedian(L)
```

Algorithm 1 presents the main algorithm pact. The algorithm starts by setting constants of the algorithm, the value for thresh and numIt from the values of ε and δ, depending on the hash family being used. The constants arise from technical calculations in the correctness proof of the algorithm, and the values are shown GetConstants subroutine (Algorithm 3). The value of thresh determines the maximum size of a *cell*. A cell is considered *small* if it has a number of solutions less than this threshold. The value of numIt determines how many times the main loop of the program (lines 5 - 14) is repeated. In each iteration of the main loop, an approximate count is generated, which is stored in the list L. While each of the approximate counts might fail to provide an estimate with the desired δ, the median of the counts of this list gives the approximation of model count with (ε, δ) guarantees.

The main loop of the algorithm begins with the subroutine GenerateHash, which produces a list of hash functions H selected from one of the families $\mathcal{H}_{shift}$, $\mathcal{H}_{prime}$, or $\mathcal{H}_{xor}$ to be used during the current iteration. In each iteration, pact maintains a list C of numbers, where the element $\mathsf{C}[i]$ represents the size of a *cell* after applying the first i hash functions from H, denoted as $H_{[i]}$. The subroutine NextIndex, called in line 9, uses a galloping search to identify an index i in C where the value of $\mathsf{C}[i]$ has been computed. The parameter ℓ in GenerateHash determines the range of hash functions generated. Specifically, (i) for $\mathcal{H}_{shift}$, the range of hash functions is set to 2^ℓ, and (ii) for $\mathcal{H}_{prime}$, GenerateHash constructs hash functions of a range of the smallest prime larger than 2^ℓ.

Following that, in line 10-11, pact checks by a call to SaturatingCounter whether $|\mathsf{Sol}(F \wedge H_{[i]})_{\downarrow S}| < \mathsf{thresh}$ and $|\mathsf{Sol}(F \wedge H_{[i]})_{\downarrow S}| \geq \mathsf{thresh}$. With this condition, pact is enabled a rough estimate of the model count, but to get the estimate within desired error bounds, pact uses the FixLastHash subroutine in line 12. This subroutine eliminates the last hash, $H[i]$, and introduces a new hash function that reduces the number of solution partitions. The subroutine iterates the procedure to find two hash functions h' and h'', such that h'' divides the space into $^k/_2$ parts, while h' divides into k parts. Now if $|\mathsf{Sol}(F \wedge H_{[i-1]} \wedge h')_{\downarrow S}| < thresh$, $|\mathsf{Sol}(F \wedge H_{[i-1]} \wedge h'')_{\downarrow S}| \geq \mathsf{thresh}$, FixLastHash returns $H' = H_{[i-1]} \wedge h'$ and $C' = |\mathsf{Sol}(F \wedge H_{[i-1]} \wedge h')_{\downarrow S}|$. However, when pact uses $\mathcal{H}_{xor}$, the call to FixLastHash is unnecessary, as it already partitions the solution space into two parts using one hash function.

The subroutine GetCount approximates $\mathsf{Sol}(F)_{\downarrow S}$ by multiplying C' with the number of partitions generated by all the hashes used in H'. This number is then appended to the list L, and pact continues to the next iteration of the main loop. Once the main loop generates count for numIt times, we take the median of all the counts, which is an approximation for $\mathsf{Sol}(F)_{\downarrow S}$ with desired guarantees.

Algorithm 2 FixLastHash$(F, S, \mathsf{C}, H, i, \ell)$

1: **if** family= $\mathcal{H}_{xor}$ **then return** C, H
2: **while** $\ell > 1$ **do**
3: $\quad \ell \leftarrow \lfloor {}^{\ell}/_{2} \rfloor$
4: $\quad h^{(\ell)} \leftarrow \mathsf{GenerateHash}(S, \ell, \mathsf{family})$
5: $\quad c \leftarrow \mathsf{SaturatingCounter}(F \wedge H_{[i-1]} \wedge h^{(\ell)}, S, \mathsf{thresh})$
6: $\quad$ **if** $c \neq \top$ **then** $\mathsf{C}[i] = c, H[i] = h^{(\ell)}$
7: $\quad$ **else return** C, H
8: **return** $\perp$

Algorithm 3 GetConstants$(\varepsilon, \delta, \mathsf{family})$

1: $\mathsf{thresh} \leftarrow 1 + 9.84\left(1 + \frac{\varepsilon}{1+\varepsilon}\right)\left(1 + \frac{1}{\varepsilon}\right)^2$
2: **if** family = $\mathcal{H}_{xor}$ **then** iters $\leftarrow \lceil 17 \log \frac{3}{\delta} \rceil, \ell \leftarrow 1$
3: **else** iters $\leftarrow \lceil 23 \log \frac{3}{\delta} \rceil, \ell \leftarrow 4$
4: **return** thresh, iters, ℓ

4.3.1 Enumerating in a cell.

Another crucial step in pact is to determine when the size of a cell is less than the threshold. pact uses the SaturatingCounter subroutine to enumerate solutions of the formula $F \wedge H_{[i]}$ to determine the size. An SMT solver is employed to find a solution res for the given formula F. The projection of this solution, $\mathsf{res}_{\downarrow S}$, is then *blocked* by adding a constraint $\neg(\mathsf{res}_{\downarrow S})$. Subsequently, the solver is asked to find another solution. This process is repeated in a loop until the solver finds thresh many solutions or reports UNSAT.

4.3.2 Searching for the number of partitions.

The number of index of C goes upto $O(|S|)$. The task for NextIndex subroutine is to find the index, for which the cell size is in the range $[1, \mathsf{thresh}]$. The NextIndex finds the index by $O(\log(|S|))$ many calls by employing a galloping search method.

We defer the detailed descriptions of GenerateHash, NextIndex, SaturatingCounter, GetCount and to the technical report of the work.

4.3.3 Analysis

Theorem 4.3.1. Let F be a formula defined over a set of variables V and discrete projection variables S. Let $\mathsf{Est} = \mathsf{pact}(F, S, \varepsilon, \delta)$ be the approximation returned by pact, and $c = |\mathsf{Sol}(F)_{\downarrow S}|$. Then,

$$\Pr\left[\frac{c}{1+\varepsilon} \leq \mathsf{Est} \leq (1+\varepsilon)c\right] \geq 1 - \delta$$

Moreover, pact makes $O(\log(|S|)\frac{1}{\varepsilon^2}\log(\frac{1}{\delta}))$ many calls to an SMT solver.

Proof. We defer the proof to the accompanying technical report. □

4.3.4 Impact of Hash Function Families

Our empirical evaluation indicates SaturatingCounter is computationally the most expensive subroutine during execution of pact. Recall that SaturatingCounter is invoked over the formula instance $F \wedge H_{[i]}$; naturally, the choice of the hash function family impacts the practical difficulty of the instance $F \wedge H_{[i]}$. Below, we highlight the tradeoffs offered by different hash function families along many dimensions.

- *Bit-level vs. Bitvector Operations:* The hash function $\mathcal{H}_{xor}$ operates at the bit level, whereas $\mathcal{H}_{shift}$ and $\mathcal{H}_{prime}$ function on bitvectors, making the latter two more amenable to SMT reasoning. This fundamental difference in operation also enables $\mathcal{H}_{prime}$ and $\mathcal{H}_{shift}$ to vary the hash function's domain sizes, a flexibility not available with $\mathcal{H}_{xor}$.

Table 4.1: Number of instances counted. (Projection on BV variables.)

Logic	Baseline	pact$_{prime}$	pact$_{shift}$	pact$_{xor}$
QF_ABVFPLRA (30)	–	–	–	4
QF_ABVFP (333)	9	1	1	31
QF_ABV (2922)	–	–	–	287
QF_BVFPLRA (55)	–	–	–	37
QF_BVFP (10434)	2	73	88	227
QF_UFBV (428)	2	9	2	17
Total (14202)	13	83	91	603

- *Number of hash functions:* As each hash function cannot partition the solution space by more than two partitions, $\mathcal{H}_{xor}$ requires more number of constraints for counting an instance compared to $\mathcal{H}_{shift}$ and $\mathcal{H}_{prime}$. A number of constraints generally make the problem harder for the solver to solve efficiently.
- *Complexity of required operations:* Multiplication and modulus operations impose significant computational overhead on SMT solvers, making $\mathcal{H}_{prime}$ and $\mathcal{H}_{shift}$ more complicated than $\mathcal{H}_{xor}$. Moreover, when pact uses $\mathcal{H}_{xor}$, it leverages the native XOR reasoning capability of CryptoMiniSat SAT solver inside pact, further increasing the counter's performance.
- *Requirement of bitwidth:* To represent the value of $\sum a_i x_i$, a bitvector of width $2w$ is required in $\mathcal{H}_{shift}$, whereas a bitwidth of $2w + d$ is needed in $\mathcal{H}_{prime}$, as the hash value in $\mathcal{H}_{shift}$ is calculated modulo 2^{2w}, while in $\mathcal{H}_{prime}$ is computed modulo a prime. Since SMT solver performance degrades quickly with increasing bitwidth, $\mathcal{H}_{shift}$ is a more favorable choice in this context. In an alternative implementation of $\mathcal{H}_{prime}$, each term $a_i x_i$ could be represented modulo a prime, but this would require an additional d modulo operations, increasing complexity.

4.3.5 Implementation and Supported Logics

We implement pact on top of CVC5, a modern SAT solver. pact uses CVC5 for parsing the formula and running the SMTSolve procedure. In the problem of Count$\mathcal{T}_{\downarrow\mathcal{P}}$, pact solves all of the theories and theory combinations in SMT-Lib for $\mathcal{T}$ and theories of bit-vectors for $\mathcal{P}$. To enhance efficiency, we utilize the SMT solver in its incremental mode, allowing each subsequent query to leverage the information gained from previous calls. Similar incremental calls are different calls to SaturatingCounter at different iterations of pact.

4.4 Experimental Evaluation

We implemented the proposed algorithm on top of state-of-the-art SMT solver cvc5. We used the following experimental setup in the evaluation:

Baseline. We compared the performance of pact with the current state-of-the-art tool by Chistikov, Dimitrova, and Majumdar [CDM17]. We refer to the tool as CDM, after the initials of its authors.

Benchmarks. Our benchmark suite comprises 14,202 instances from the SMT-Lib 2023 release. To minimize bias, we adopted a benchmark selection methodology inspired by early works on propositional model counting. We initially selected all instances supported by six theories. Subsequently, we filtered out instances where the number of solutions was very small (less than 500 models) or where even satisfiability was computationally challenging, as determined by cvc5's inability to find a satisfying assignment within 5 seconds.

Environment. We conducted all our experiments on a high-performance computer cluster, with each node consisting of Intel Xeon Gold 6148 CPUs. We allocated one CPU core and an 8GB memory limit to each solver instance pair. To adhere to the standard timeout used in model counting competitions, we set the timeout for all experiments to 3600 seconds. We use values of $\varepsilon = 0.8$ and $\delta = 0.2$, in line with prior work in the model counting community.

We conduct extensive experiments to understand the following:

RQ1. How does the runtime performance of pact compare to that of Baseline, and how does the performance vary with different hash function families?

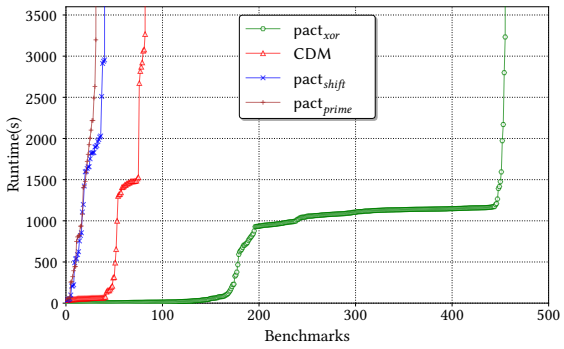
RQ2. How accurate is the count computed by pact in comparison to the exact count?

Summary of Results. pact solves a significant number of instances from the benchmarks. It solved 603 instances, while the Baseline counted only 13 instances. Among different hash families, pact performs the best while it uses $\mathcal{H}_{xor}$ hash functions. The accuracy of pact is also noteworthy; the average approximation error is 3.3% while using $\mathcal{H}_{xor}$ hashes.[1]

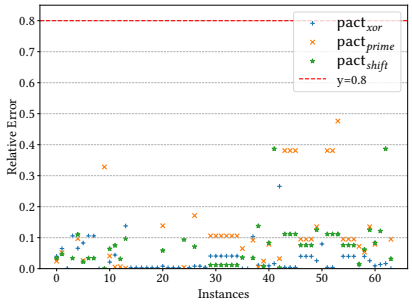
4.4.1 Performance of pact

We evaluate the performance of pact based on two metrics: the number of instances solved and the time taken to solve those instances. To differentiate pact utilizing different hash function families, we use the notations pact_{prime}, pact_{shift}, and pact_{xor}.

[1]The benchmarks and logfiles are available at https://doi.org/10.5281/zenodo.16413909



(a) Cactus plot comparing performances.



(b) Accuracy: observed error vs. the theoretical bound.

Figure 4.1: Performance and accuracy

Instances solved. For each of the logic, we look at the number of instances solved. Out of 14,202 instances, Baseline could solve only 13 instances. Conversely, pact$_{xor}$ could solve 603 instances, demonstrating a substantial improvement compared to Baseline. The performance varies across different logics, which we represent in Table 4.1. The number of instances solved is 4.2% of the total number of instances, which is expected, given the target problem for these instances is satisfiability, and pact solves a more complex problem.

Comparison of Hash Functions. In the Table 4.1, we compare the performance of pact when it utilizes different hash functions for partitioning the solution space. While all of them perform much better in comparison to Baseline, the best performance is shown by pact$_{xor}$, which solved 603 instances. The performances of pact$_{prime}$ and pact$_{shift}$ are similar, solving 83 and 91 instances.

Solving time comparison. A performance evaluation of Baseline and pact is depicted in Figure 6.1a, which is a cactus plot comparing the solving time. The x-axis represents the number of instances, while the y-axis shows the time taken. A point (i, j) in the plot represents that a solver solved j benchmarks out of the 14,202 benchmarks in the test suite in less than or equal to j seconds. The different curves plot the performance of Baseline and pact with different hash functions.

Performance against the number of solutions. The primary constraint in the enumeration-based method is its capacity for solution counting. The baseline model caps at 3,570 solutions, while pact counts up to 1.7×10^{19}. Figure **??** presents a time versus solution count comparison, where the x-axis represents sorted instances by solution count and the y-axis indicates the time required for Baseline and pact to solve an instance. Thus, a point (x, y_1) for Baseline and (x, y_2) for pact represents the time taken to solve an instance with x solutions, where y_1 seconds is the time for Baseline (exact count) and y_2 seconds is the time for pact (approximate count by pact). A marker at $y = 3600$ denotes a timeout. The graph demonstrates that pact solves many high-count instances rapidly, unlike Baseline, which often times out as solution

counts increase. The performance of pact_{xor} is better than pact_{shift} or pact_{prime} in this regard as well - the maximum count returned by them are in magnitudes of 10^7, while pact_{xor} is of 10^{19}.

4.4.2 Quality of Approximation

From our benchmark set, only 13 instances were solved by Baseline. To increase the number of instances for which we know the exact count, we also include the benchmarks with model counts between 100 and 500 in this section - resulting in 64 instances. We quantify the quality of approximation with the parameter error $e = \max\left(\frac{b}{s}, \frac{s}{b}\right) - 1$, where b is the count from Baseline and s from pact. this definition of error aligns with the ε used in the algorithm's theoretical guarantees and can be interpreted as the observed value of ε. Analysis of all 64 cases found that for pact_{xor}, the maximum e to be 0.26 and the average to be 0.03, signifying pact substantially outperforms its theoretical bounds, which is 0.8. In Figure 6.1b, we illustrate the quality of approximation for these instances. The x-axis lists the instances, while the y-axis displays the relative error exhibited by a configuration of pact. A dot (x, y) in the graph indicates x^{th} instance showed a relative error of y. The graph indicates that, for most instances, the error lies below 0.2, with a few instances falling between 0.2 and 0.8. The error for pact_{shift} and pact_{prime} is relatively higher than pact_{xor}, with average error being 0.07 and 0.12 and maximum error being 0.39 and 0.48. Our findings underline pact's accuracy and potential as a dependable tool for various applications.

4.5 Summary

In this work, we introduced pact, a projected model counter designed for hybrid SMT formulas. Motivated by the diverse applications of model counting and the role of hashing, we explored the impact of various hash functions, examining both bit-level and bitvector-level approaches. Our empirical evaluation demonstrates that pact achieves strong performance on a broad application benchmark set. A dedicated XOR reasoning engine significantly enhanced pact's performance, suggesting that further development of specialized reasoning engines for bit vector-level hash functions could be a promising research direction. Additionally, pact's theoretical framework supports the SMT theory of integers (with specified bounds) as projection variables; this feature is not yet implemented and remains an avenue for future work.

Chapter 5

Continuous Domains: Volume of LRA Regions

In this chapter, we present ttc, an algorithm for #SMT problem over LRA formulas. It decomposes the solution space into overlapping convex polytopes, computes each volume, then combines them via a set-union calculation. The design draws on streaming-mode set-union methods, polytope volume computation, and AllSAT enumeration. Benchmarks show ttc is faster than prior tools by a wide margin.

In the previous two chapters, we saw how to solve the #SMT problem when the theory is discrete or hybrid, but in both the cases, we limited the study in counting in a domain where the counting problem itself was in a discrete setting. Many SMT theories operate over continuous domains — linear and non-linear real arithmetic being the main examples. For these theories, a core quantitative problem is computing the volume of the satisfiable region of an SMT formula. This arises directly in quantitative verification, such as measuring the robustness of neural networks. For theory of linear real arithmetic, the #SMT problem transforms into volume computation. There has been some progress for the problem in recent years [GMZZ18; MLZ09], however, as we see further, they have scalability challenge. In this chapter, we address the question: *Given an SMT LRA formula, can we design an efficient volume computation algorithm?* Our primary contribution is the development of ttc, a novel algorithmic framework that provides an affirmative answer to this question. The ttc algorithm approximates the volume of SMT LRA formula solution spaces with provable theoretical guarantees.

We start with a very related and well-studied problem to SMT volume computation: the problem of volume computation of *bounded convex bodies*. Sophisticated exact and approximate methods to solve the problem have been developed in the last few decades. Although the exact volume problem is #P-hard, the seminal

work by Dyer, Frieze, and Kannan [DFK91] showed a polynomial-time randomized approximation algorithm (FPRAS) for this problem. Furthermore, advancements have not only improved the asymptotic running times of these algorithms [LS90; AK91; Lov91; LS92; LS93; KLS97; LD12; CV18] but also yielded practically efficient methods that forgo certain theoretical guarantees to eliminate prohibitive hidden constants in their runtime [CV16].

A central challenge in applying these ideas to SMT lies in the non-convex nature of the solution spaces generated by SMT formulas. Unlike convex bodies, non-convex regions are more complex to analyze due to their irregular shapes and potential discontinuities. A natural strategy to overcome this hurdle is to partition the non-convex space into a union of convex bodies, where each convex piece can be more easily managed with existing techniques. Decomposing a non-convex SMT solution space into convex components introduces its own set of challenges. Current state-of-the-art techniques can't handle the union of non-disjoint components. In many cases, the decomposition yields an excessive number of disjoint components, which may not accurately reflect the underlying structure of the solution space. In practice, solution spaces manifest as unions of overlapping, non-disjoint polytopic regions with boundaries and intersections that encode critical constraint information. This overlapping structure is not merely theoretical—our empirical analysis reveals cases where state-of-the-art decomposition techniques transform a natural representation of 7 overlapping polytopes into an unwieldy collection of 20,595 disjoint components, creating unnecessary computational complexity.

Our approach builds upon recent advancements in counting distinct elements across set unions in streaming models by Meel, Vinodchandran and Chakraborty (MVC,2021). The fundamental challenge in adapting the MVC algorithm to volume computation arises from the inherent difference between discrete and continuous domains. The MVC algorithm was specifically engineered for discrete settings, while volume computation operates in continuous space. We overcome this obstacle through a principled discretization approach, effectively reducing continuous volume computation to the problem of counting lattice points within a carefully constructed fine-grained lattice space.

We have implemented ttc and evaluated it on a comprehensive benchmark suite. The results demonstrate significant gains in scalability and accuracy. Out of a benchmark set of 1131 instances, ttc solved 1112, while the current state of the art can solve only 145.

Our approach is based on MVC, which has demonstrated practical performance advantages over KLM [Soo+24]. Departing from [BF10; ACD22], we relax the error tolerance to $O(\varepsilon)$ for sampling and volume estimation, rather than $O(\varepsilon^2/n)$; ε is the volume-approximation factor and n the dimension.

Applications. The theory of linear real arithmetic has significant applications in the formal verification of systems with real variables. These include hybrid systems such as cyber-physical systems [Kol+23], and control systems [CMT12] and timed systems [CGSS13]. Advanced verification tools like Reluplex [Kat+17]

extend SMT solving to neural networks by encoding real-valued variables for network inputs. Extending these approaches to quantitative verification would require LRA solvers with efficient volume computation capabilities. This follows the established pattern where model counting tools have enabled quantitative verification advances in software [GFB21; TW21] and binarized neural networks [Bal+19].

5.1 Preliminaries and Problem Statement

The Boolean abstraction F_B of F is obtained by replacing each theory atom with a fresh Boolean variable while leaving the original Boolean variables unchanged. We assume that F_B is expressed in Disjunctive Normal Form (DNF) as $F_B = \bigvee_{i=1}^{m} c_i$, where each cube is given by $c_i = \bigwedge_{j=1}^{n_i} \ell_{ij}$, with each ℓ_{ij} being a Boolean literal (either a variable or its negation). And-Inverter Graphs (AIGs) are used as a compact representation for Boolean functions as a circuit, where each node corresponds to a two-input AND gate or an inverter.

We define a mapping M on the Boolean literals corresponding to theory atoms such that for each such literal ℓ, $M(\ell)$ is its corresponding linear inequality (or its negation). Pure Boolean variables are not mapped, as they do not contribute geometric constraints. For each cube c_i, the conjunction $\bigwedge_{j=1}^{n_i} M(\ell_{ij})$ (with the understanding that M is applied only to theory literals) defines a (possibly empty) convex polytope, which we denote by $\mathcal{P}(c_i) = \{x \in \mathbb{R}^n \mid \forall\, \ell \in c_i$ (theory literal), $M(\ell)(x)$ holds$\}$.

A polyhedron is defined as the intersection of a finite number of half-spaces in $\mathbb{R}^n$, and a polytope is a bounded polyhedron whose volume (measured via the Lebesgue measure) can be computed. Concretely, any polytope K can be written in the form $\{x \in \mathbb{R}^n \mid Ax \leq b\}$, where A is a $m \times n$ matrix and b is a $n \times 1$ vector. Each row of the system $Ax \leq b$ defines one of the m half-spaces (the facets of K). We denote $\text{facet}(K) = m$, $\text{Sol}(K) = \{\, x \in \mathbb{R}^n \mid Ax \leq b\}$, and $\text{Volume}\,(K)$ as the number of facets, the feasible region, and the volume of K, respectively.

Since we are working over the domain $\mathbb{R}$, precision is crucial in our work. In our setting, we define precision as the number of digits after the decimal point. For example, the number 0.123456789 has a precision of 9.

```
(set-logic QF_LRA)

(declare-const x Real)
(declare-const y Real)

(assert (or
  (and (> x 20) (< x 40) (> y 20) (< y 40))
  (and (> x 10) (< x 30) (> y 10) (< y 30))))

(check-sat)
```

Figure 5.1: SMT file.

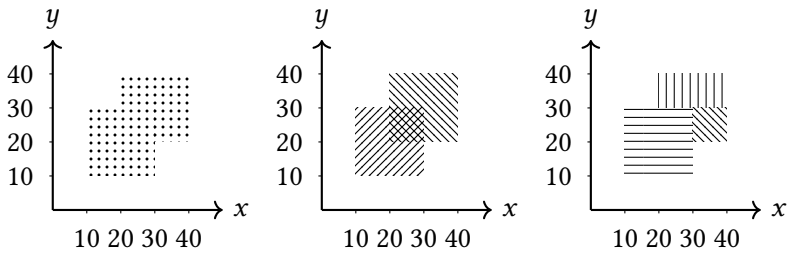


Figure 5.3: Solution space of SMT formula (left). Disjoint (middle) and non-disjoint (right) decomposition of the solution space of the formula.

Illustrative Example. Figure 5.1 shows the QF_LRA formula over two real variables x and y, which defines two overlapping square regions in the xy-plane. As depicted in Figure 5.3, each square has an area of 400, with an overlapping area of 100. Thus, the union of the two regions has an area of 700. Internal nodes represent logical conjunctions, and any inverted edges (dots) indicate negations. Figure 5.3 (left) illustrates a disjoint decomposition of the solution space, where each region is separated so that no two subsets overlap. This often simplifies volume computations but may require more partitions. By contrast, Figure 5.3 (right) shows a non-disjoint decomposition, allowing subsets to overlap. Figure 5.2 compares the number of cubes generated by disjoint decomposition and non-disjoint decomposition on our benchmark set.

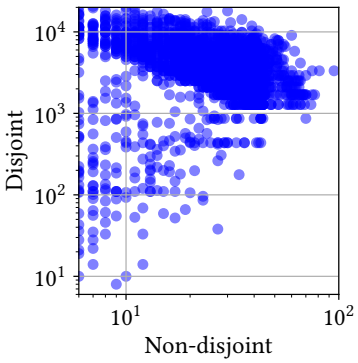


Figure 5.2: Comparison of the number of polytopes in disjoint and non-disjoint decomposition (notice the non-equal axis).

Problem Statement Let K be the whole solution space of the given formula F, which has d dimensions. Let $\mathcal{B} \subset \mathbb{R}^n$ be an n-dimensional measurable set. The volume of $\mathcal{B}$ is defined as $\text{Volume}(\mathcal{B}) = \int_{\mathcal{B}} d\mathbf{x}$, where $d\mathbf{x}$ denotes the differential volume element. In Cartesian coordinates, this element is expressed as $d\mathbf{x} = dx_1\, dx_2 \cdots dx_n$.

5.2 Algorithm:

Given an SMT LRA formula F_L, the ttc algorithm returns an estimate of $\text{Volume}(\mathsf{F}_\mathsf{L})$. Initially, the algorithm decomposes the solution space of the SMT formula into non-disjoint polytopes and computes the volume for each polytope. Subsequently, it estimates the volume of the union of the polytopes' solution space using a sampling-based approach.

Since we only have a volume computation algorithm for convex polytopes, but the solution space of an SMT formula may be non-convex, we decompose the solution space as a union of convex polytopes. First, we observe that using F_BD, the Boolean abstraction of F_L in DNF form, we can capture the solution space

Algorithm 4 $\mathsf{ttc}(F, \varepsilon, \delta)$

$\mathsf{F_{BC}}$,$\mathsf{M_{LA}} \leftarrow$ BooleanAbstraction($\mathsf{F_L}$)
$\mathsf{F_{BD}} \leftarrow$ toDNF($\mathsf{F_{BC}}$)
$b \leftarrow$ GetPrecision($\mathsf{F_{BD}}$,M,$\frac{\varepsilon}{8}$)
$\varepsilon' \leftarrow \frac{\varepsilon}{12}$, $\delta' \leftarrow \frac{\delta}{2m}$
Thresh $\leftarrow \max\left(24 \cdot \frac{\ln(24/\delta)}{(1-\varepsilon')\varepsilon'^2}, 6(\ln\frac{6}{\delta} + \ln m)\right)$
$p \leftarrow 1$; $\mathcal{X} \leftarrow \emptyset$
for $i = 1$ to m **do**
    $t \leftarrow$ ComputeVolume(Polytope(G_i), ε', δ')
    for $s \in \mathcal{X}$ **do**
        if $s \in$ Polytope(G_i) **then** remove s from $\mathcal{X}$
    while $p \geq \frac{\text{Thresh}}{t}$ **do**
        Remove every element of $\mathcal{X}$ with prob. $1/2$
        $p \leftarrow p/2$
    $N_i \leftarrow$ Poisson($t \cdot p$)
    while $N_i + |\mathcal{X}| >$ Thresh **do**
        Remove every element of $\mathcal{X}$ with prob. $1/2$
        $N_i \leftarrow$ Poisson($t \cdot p/2$) and $p \leftarrow p/2$
    $S \leftarrow$ GenerateSamples(Polytope(G_i), N_i, b)
    $\mathcal{X}$.Append(S)
Output $|\mathcal{X}|/p$

of $\mathsf{F_L}$ as a union of polytopes. Let $G_i = \bigwedge_{j=1}^{n_i} \ell_{ij}$ be a cube in the DNF. Then, the conjunction $\bigwedge_{j=1}^{n_i} M(\ell_{ij})$ of the corresponding linear inequalities defines a (possibly empty) convex polytope, which we denote by Polytope(G_i). We can show that $\bigcup_{i=1}^{m}$ Polytope(G_i) = Sol($\mathsf{F_L}$) (Lemma 5.3.4). This relation shows that the DNF representation of the Boolean abstraction of an SMT formula can be used to decompose its solution space into convex polytopes.

To efficiently compute the union of these polytopes, we leverage recent breakthroughs in streaming algorithms for set union operations. The central idea is to compute the union of volumes by maintaining a representative set of points that approximates the total volume. The main caveat of this approach is that the algorithm requires the underlying sets to be finite, while the volume of a polytope cannot be measured directly with a finite number of points. To overcome this issue, we consider an axis-parallel lattice in $\mathbb{R}^n$ with cell side length 10^{-b}, defined as $\mathbb{L}^n = 10^{-b}\mathbb{Z}^n = \{(10^{-b}k_1, 10^{-b}k_2, \ldots, 10^{-b}k_n) \mid k_i \in \mathbb{Z} \text{ for } i = 1, \ldots, n\}$. If b is sufficiently large, we can use this lattice to establish a relationship between the number of lattice points contained within a polytope, $|K \cap \mathbb{L}^n|$, and the volume of the polytope, Volume (K).

We present our algorithm, ttc, in Algorithm 4, and in the subsequent part, we describe the algorithm in detail.

In line 1 of ttc, we parse the formula F_L and construct its Boolean abstraction as a circuit, specifically as an And-Inverter Graph (AIG). Then, in line 2, ttc converts the circuit to DNF. The circuit representation enables us to avoid introducing any auxiliary variables. In essence, converting the circuit to DNF is equivalent to solving a circuit AllSAT problem, for which we leverage recent advances in the literature.

Based on the desired accuracy ε of the volume estimate, we begin by computing the precision parameter b using GetPrecision in line 3, and threshold value Thresh in line 2, which determines approximately how many points will be maintained during the algorithm's execution. In the main loop (lines 7 to 16), we process each cube Polytope(G_i) sequentially. For each cube, we first compute the (approximate) volume of its corresponding polytope using a polytope volume computation algorithm ComputeVolume (line 8). Next, in line 7, the algorithm removes from the current set $\mathcal{X}$ all solutions that are already accounted for. We then determine the number of solutions N_i that would be sampled from Polytope(G_i) if each solution were independently sampled with probability p; here, N_i is modeled by a Poisson distribution. Since we wish to keep the size of $\mathcal{X}$ bounded by Thresh, if the sum $|\mathcal{X}| + N_i$ exceeds Thresh, we decrease p and adjust N_i accordingly—this adjustment is performed by resampling N_i from a Poisson distribution with the revised parameter (p) and by removing elements from $\mathcal{X}$ with probability 1/2 (line 9). Next, we sample N_i solutions from the cube G_i uniformly at random and add to $\mathcal{X}$—this is essentially done by uniformly sampling a lattice point from Polytope(G_i) $\cap\, \mathbb{L}^n$. Finally, the algorithm outputs the final volume estimate $\frac{|\mathcal{X}|}{p}$.

Algorithm 5 GetPrecision(F_{BD},M, η)

```
1:  maxRatio← 1,  r ← 0
2:  for G_i in F_BD do
3:      P^i ← Polytope(G_i)
4:      r ← r + facet(P^i)
5:  for i = 1 to m do
6:      γ̂ ← MaxInscribedBall(P^i)
7:      γ ← (16n/η) √(log 4r/η)
8:      if γ/γ̂ > maxRatio then
9:          maxRatio ← γ/γ̂
10: return ⌈log_10(maxRatio)⌉
```

Getting precision. From [KV97], we know that if an n-dimensional ℓ_2-ball of radius $\gamma = \Omega(n\sqrt{\log f})$ can be inscribed in an n-dimensional polytope with f facets, then there exists a relationship between the number of integer lattice points inside the polytope and its volume. When working with the lattice $\mathbb{L}^n$ instead of the standard integer lattice $\mathbb{Z}^n$, the effective radius decreases, but the same relationship still applies. Furthermore, as shown in Lemma 5.3.6, the same relationship holds for a union of polytopes, provided each polytope contains an ℓ_2-ball of radius $\gamma = \Omega\left(n\sqrt{\log \sum_i \text{facet}(P_i)}\right)$. In Algorithm 5, we process each polytope sequentially (lines 5–9). For each polytope P^i, we compute $\hat{\gamma}$, the maximum radius of a ball that

can be inscribed in P^i, using an LP algorithm (line 6). The required precision for this polytope is then determined as $\lceil \log_{10}(\gamma/\hat{\gamma}) \rceil$. After processing all polytopes, we take the maximum of these precision values and return it.

5.3 Correctness and Guarantees

Theorem 5.3.1. Given a set $\mathsf{F_L}$, and parameters $\varepsilon, \delta \in (0, 1]$, ttc returns an estimate Est of Volume (F) such that with probability at least $1 - \delta$,

$$\mathsf{Est} \in [(1-\varepsilon) \cdot \mathsf{Volume}\,(\mathsf{F_L}),\ (1+\varepsilon) \cdot \mathsf{Volume}\,(\mathsf{F_L})]\,.$$

Theorem 5.3.2. The ttc algorithm takes exponential time w.r.t. v in decomposing the polytope, where v is the number of variables in $\mathsf{F_{BC}}$. The number of cubes m can be exponential of v as well.

Proof. The runtime complexity follows from the dual-rail algorithm of circuit AllSAT algorithm. There exists CNF formulas, for which DNF representation is exponential sized, resulting in the exponential blowup of m. □

Theorem 5.3.3. Let m denote the number of decomposed polytopes. Then the ttc algorithm runs in time $O(mn^4)$, where n is the number of dimensions and the $O(\cdot)$ notation suppresses polylogarithmic factors in ε and δ.

Proof. The algorithm's main loop (lines 7–16) executes exactly m iterations. In each iteration, the volume computation (line 8) requires $O(n^4)$ time, while the sampling step (line 15) consumes $O(n^3)$ time per sample. Since these two operations dominate the computational cost in each iteration, the total runtime is given by $m \times O(n^4) = O(mn^4)$, where the polylogarithmic dependencies on ε and δ are hidden in the $O(\cdot)$ notation. □

Now we prove the theorem 5.3.1 using the following lemmas.

Lemma 5.3.4. Let $\mathsf{F_{BD}} = \bigvee_{i=1}^{m} G_i$ be the DNF abstraction of the SMT formula $\mathsf{F_L}$. Then,

$$\bigcup_{i=1}^{m} \mathsf{Polytope}(G_i) = \mathsf{Sol}(\mathsf{F_L})$$

Proof. First, we show the soundness of each cube: we claim that for every cube G_i in F_{BD}, $\mathsf{Polytope}(G_i) \subseteq \mathsf{Sol}(\mathsf{F}_{\mathsf{L}})$. Let $G_i = \bigwedge_{j=1}^{n_i} \ell_{ij}$ be an arbitrary cube of F_{BD}. By construction, we have $\ell_{i,j} \models \mathsf{F}_{\mathsf{BD}}$. Therefore, by the property of Boolean abstraction, if $\bigwedge M(\ell_{i,j})$ can be satisfied, then F_{L} is satisfied. Any point $x \in \mathsf{Polytope}(G_i)$ satisfies $\bigwedge M(\ell_{i,j})$, therefore $x \in \mathsf{Sol}(\mathsf{F}_{\mathsf{L}})$, proving that $\mathsf{Polytope}(G_i) \subseteq \mathsf{Sol}(\mathsf{F}_{\mathsf{L}})$.

Next, we show the completeness of the union, that $\mathsf{Sol}(\mathsf{F}_{\mathsf{L}}) \subseteq \bigcup_{i=1}^{m} \mathsf{Polytope}(G_i)$. Consider any arbitrary point $x \in \mathsf{Sol}(\mathsf{F}_{\mathsf{L}})$. Now x induces a Boolean assignment satisfying F_{BD}. Since $\mathsf{F}_{\mathsf{BD}} = \bigvee_{i=1}^{m} G_i$, there exists at least one cube G_i such that x satisfies every literal in G_i. By the definition of M, we have that for every $l \in G_i$, $M(l)(x)$ holds, which implies that $x \in \mathsf{Polytope}(G_i)$. Thus, $\mathsf{Sol}(\mathsf{F}_{\mathsf{L}}) \subseteq \bigcup_{i=1}^{m} \mathsf{Polytope}(G_i)$. □

Lemma 5.3.5 (Theorem 3 of [KV97])**.** Let P be a polytope in $\mathbb{R}^n$ that contains an ℓ_2-ball of radius at least $8n\sqrt{\log(2\mathrm{facet}(P)/\eta)}$, where $0 < \eta \leq 1$ is a constant. Then $\mathsf{Volume}\,(P)$ satisfies the following bounds,

$$\mathsf{Volume}\,(P) \in \left[\frac{1-\eta}{2}|P \cap \mathbb{Z}^n|,\ \ (1+\eta)|P \cap \mathbb{Z}^n|\right]$$

We extend this result to the case of the union of polytopes in the following lemma.

Lemma 5.3.6. Let $Q = \bigcup_{i=1}^{m} P_i$ with $P_i = \mathsf{Polytope}(G_i)$. If every P_i contains an ℓ_2-ball of radius at least $\frac{16n}{\eta}\sqrt{\log\frac{4r}{\eta}}$ where $0 < \eta \leq 1$ a constant and $r = \sum_{i=1}^{m} \mathrm{facet}(P_i)$, then

$$\mathsf{Volume}\,(Q) \in [(1-\eta)\,|Q \cap \mathbb{Z}^n|, (1+\eta)\,|Q \cap \mathbb{Z}^n|]$$

Proof. Recall the description of P_i as $P_i = \{x \in \mathbb{R}^n \mid A_{ij}x \leq b_{ij}\ (j = 1, \ldots, \mathrm{facet}(P_i))\}$ and set

$$\kappa := \sqrt{2\log\frac{4}{\eta}} + \sqrt{2\log r}, \quad k := \max_{i,j} \frac{b_{ij} + \kappa||A_{ij}||}{b_{ij} - \kappa||A_{ij}||}$$

where $r = \sum_{i=1}^{m} \mathrm{facet}(P_i)$ and $||\cdot||$ denotes the ℓ_2-norm. Expand/shrink every facet by $\pm\kappa||A_{ij}||$ and write $P_i' := \{x \mid A_{ij}x \leq b_{ij} + \kappa||A_{ij}||\}$, $P_i'' := \{x \mid A_{ij}x \leq b_{ij} - \kappa||A_{ij}||\}$, and let $Q' := \bigcup_i P_i'$, $Q'' := \bigcup_i P_i''$. Let X be the randomized rounding process as defined in [KV97], which takes a point $p \in \mathbb{R}^n$ and returns a point in the lattice $x \in \mathbb{Z}^n$ such that $|x - p| \leq 1$. To complete the proof, we will need the following lemma.

Lemma 5.3.7. The following properties hold for the sets Q' and Q'': **(a)** Draw p uniformly from the continuous body Q'. Then for every $x \in Q \cap \mathbb{Z}^n$, $\Pr[\mathsf{X}_p = x] \ \geq\ \frac{1-\eta/2}{\mathsf{Volume}(Q')}$. **(b)** Draw p uniformly from the *continuous* body Q''. The event that p still lies in Q satisfies $\Pr[\mathsf{X}_p \in Q] \ \geq\ 1 - \frac{\eta}{2}$, and **(c)** for every $x \in Q \cap \mathbb{Z}^n$, $\Pr[\mathsf{X}_p = x] \ \leq\ \frac{1}{\mathsf{Volume}(Q'')}$.

We now use this lemma to complete the proof. Consider the case where p is drawn uniformly from Q''. Then using lemma 5.3.7(b) and 5.3.7(c) we have:

$$\begin{aligned} 1 - \frac{\eta}{2} &\leq \sum_{x \in Q \cap \mathbb{Z}^n} \Pr[\mathsf{X}_p = x] \\ &\leq \sum_{x \in Q \cap \mathbb{Z}^n} \frac{1}{\text{Volume}\,(Q'')} \leq \frac{|Q \cap \mathbb{Z}^n|}{\text{Volume}\,(Q)} \end{aligned}$$

The last inequality follows from $Q \subseteq Q''$. Since for all $0 < \eta \leq 1$ we have, $\left(1 - \frac{\eta}{2}\right)^{-1} \leq (1 + \eta)$, therefore, $\text{Volume}\,(Q) \leq (1 + \eta)|Q \cap \mathbb{Z}^n|$. Again, consider the case where p is drawn uniformly from Q'. Then, using lemma 5.3.7(a)

$$\begin{aligned} 1 &\geq \sum_{x \in Q \cap \mathbb{Z}^n} \Pr[\mathsf{X}_p = x] \\ &\geq \sum_{x \in Q \cap \mathbb{Z}^n} \frac{1 - \eta/2}{\text{Volume}\,(Q')} = \left(1 - \frac{\eta}{2}\right) \cdot \frac{|Q \cap \mathbb{Z}^n|}{\text{Volume}\,(Q')} \end{aligned}$$

Now consider $p \in Q'$. Then $A_{ij} p \leq b'_{ij}$ for all i, j and therefore $A_{ij} \frac{p}{k} \leq b''_{ij}$ for all i, j. This implies that $\frac{p}{k} \in Q''$. This implies that $Q' \subseteq kQ''$. Therefore, $\text{Volume}\,(Q') \leq k^n \text{Volume}\,(Q'')$. Since each P_i contains an ℓ_2-ball of radius at least $\frac{16n}{\eta}\sqrt{\log \frac{4r}{\eta}}$, it follows that for the pair i, j at which k is attained, we have $b_{ij} \geq \frac{16n}{\eta}\sqrt{\log \frac{4r}{\eta}}||A_{ij}||$. Consequently,

$$k^n = \left(\frac{\frac{16n}{\eta}\sqrt{\log \frac{4r}{\eta}} + \kappa}{\frac{16n}{\eta}\sqrt{\log \frac{4r}{\eta}} - \kappa} \right)^n$$

Claim 1. For $n \in \mathbb{N}$ we have,

$$k^n \leq \frac{1 + \eta/4}{1 - \eta/4}$$

Using Claim 1, we can upper bound $\text{Volume}\,(Q')$ as follows: $\text{Volume}\,(Q') \leq \frac{1+\eta/4}{1-\eta/4} \cdot \text{Volume}\,(Q'') \leq \frac{1+\eta/4}{1-\eta/4} \cdot \text{Volume}\,(Q)$. Using the inequality $\frac{1-\eta/4}{1+\eta/4} \cdot \left(1 - \frac{\eta}{2}\right) \geq (1 - \eta)$ for all $0 < \eta \leq 1$, we have $\text{Volume}\,(Q) \geq (1 - \eta)|Q \cap \mathbb{Z}^n|$, completing the proof of the lemma. □

Lemma 5.3.8. Let $Q = \bigcup_{i=1}^m P_i$ with $P_i = \text{Polytope}(G_i)$ such that every P_i follows the condition of lemma 5.3.6, then we have the following bound on the number of lattice points in Q,

$$\text{Volume}\,(Q) \in \left[\frac{1 - \eta}{10^{b \cdot n}} |Q \cap \mathbb{L}^n|,\ \frac{1 + \eta}{10^{b \cdot n}} |Q \cap \mathbb{L}^n| \right]$$

where b is the precision parameter selected by GetPrecision.

Proof. We define a canonical isomorphism $\phi : \mathbb{L}^n \to \mathbb{Z}^n$. For convenience, we use the same notation to denote the image of a polytope P_i (resp. Q) under ϕ, writing $\phi(P_i)$ (resp. $\phi(Q)$). This mapping scales the distance between any two consecutive points $[a, b]$ along a dimension by 10^b. Consequently, a ball of radius r in $\mathbb{L}^n$ corresponds to a ball of radius $10^b r$ in $\mathbb{Z}^n$. Furthermore, the volume of a polytope P_i (resp. Q) scales by a factor of $10^{b \cdot n}$ when transformed into $\mathbb{Z}^n$, leading to $\text{Volume}(P_i) \cdot 10^{b \cdot n}$ (resp. $\text{Volume}(Q) \cdot 10^{b \cdot n}$). Since the mapping ϕ preserves the lattice structure, we have $|\phi(P_i) \cap \mathbb{Z}^n| = |P_i \cap \mathbb{L}^n|$ and $|\phi(Q) \cap \mathbb{Z}^n| = |Q \cap \mathbb{L}^n|$.

Finally, the choice of b as selected by GetPrecision guarantees that each P_i contains a ball of radius at least $10^{-b} \frac{16n}{\eta} \sqrt{\log(4 \sum_{i=1}^{m} \text{facet}(P_i)/\eta)}$. Therefore, we can apply lemma 5.3.6 to complete the proof. □

Lemma 5.3.9. Let $Q = \bigcup_{i=1}^{m} \text{Polytope}(G_i)$. Then ttc outputs an estimate Est such that with probability at least $1 - \delta$,

$$\text{Est} \in \left[\frac{(1 - \varepsilon/3)}{10^{b \cdot n}} |Q \cap \mathbb{L}^n|, \frac{(1 + \varepsilon/3)}{10^{b \cdot n}} |Q \cap \mathbb{L}^n| \right]$$

Now we finish the proof of theorem 5.3.1 by combining the results from lemma 5.3.9 and lemma 5.3.8.

Proof of theorem 5.3.1. Using lemma 5.3.9, we have $\text{Est} \geq \frac{1-\varepsilon/3}{10^{b \cdot n}} \cdot |Q \cap \mathbb{L}^n|$, and from lemma 5.3.8, $\frac{|Q \cap \mathbb{L}^n|}{10^{b \cdot n}} \geq \frac{\text{Volume}(Q)}{(1+\varepsilon/8)}$. Combining these two inequalities, we get $\text{Est} \geq \frac{1-\varepsilon/3}{1+\varepsilon/8} \cdot \text{Volume}(Q) \geq (1 - \varepsilon) \cdot \text{Volume}(Q)$. Similarly, the corresponding upper bounds from lemma 5.3.9 and lemma 5.3.8 we yield $\text{Est} \leq \frac{1+\varepsilon/3}{1-\varepsilon/8} \cdot \text{Volume}(Q) \leq (1 + \varepsilon) \cdot \text{Volume}(Q)$. □

5.3.1 Implementation

We implemented a Python prototype[1] for testing the algorithm and its efficiency. We use the following tools for different parts of the Algorithm 4.

Decomposing into Polytopes. We use a combination of the existing SMT solver and Circuit AllSAT solver to decompose the SMT solution space into convex polytopes. Specifically, we do the following:

Boolean Abstraction. To create the Boolean abstraction of the SMT formula in line 1 of Algorithm 4, we instrument CVC5 [Bar+22] to create the abstraction as an AIG. By default, CVC5 creates the abstraction as a CNF, which we wanted to avoid, since converting to CNF introduces numerous auxiliary variables, making it difficult to enumerate the solutions in a DNF form. For this purpose, we carefully examined each different type of Boolean operation used in SMT formulas, which is typically translated to CNF, including And, Or, Iff, Implies, Ite, and Xor. For each of these operations, we constructed corresponding gates for the AIG. This avoids unnecessary blow-up and retains the semantic structure of the original formula.

[1] Source code: https://github.com/meelgroup/ttc

Circuit to DNF. To convert the AIG to DNF in line 2, we use the HALL tool [FNS23; FNSS24], which employs a dual-rail based implementation to generate all solutions of the AIG. The solutions are represented as a non-disjoint DNF.

Constructing the Polytopes. While instrumenting CVC5 to generate the Boolean abstraction, we simultaneously capture which variables correspond to which linear inequalities. For each *cube* of the DNF, therefore, we maintain a mapping indicating which set of linear inequalities this cube corresponds to. Given this map and the cube, we construct the polytope.

Polytope Volume Computation. In ComputeVolume (line 8), we employ the Gaussian cooling-based algorithm developed by [CV16], which offers a more practical implementation of their theoretically rigorous approach [CV18]. While the original algorithm [CV18] provides volume approximation with (ε, δ) guarantees, its prohibitive running time of $10^{16}O(n^3)$ limits practical applications. The key insight in [CV16] was that these substantial constant factors can be significantly reduced in practical scenarios without severely compromising accuracy, though this comes at the cost of theoretical guarantees. Our implementation utilizes VolEsti, the software package by Chalkis and Fisikopoulos (2021) that implements this practical algorithm.

Preprocessing The polytopes generated by the Polytope(c) procedure may contain redundancies and hidden equalities. Hidden equalities render the polytope *degenerate* - resulting in zero volume in d dimensions. Additionally, redundancies significantly complicate the volume computation for the underlying algorithm. We address these challenges by incorporating CDD as a preprocessing step, which effectively identifies hidden equalities and eliminates redundant inequalities. This preprocessing phase yields substantial performance improvements, particularly valuable when processing inequalities derived from SMT files, which often lack optimal formulation.

Polytope Sampling. For polytope sampling algorithm GenerateSamples in line 15, we employ the Monte Carlo random walk hit-and-run algorithm [Smi84]. Each random walk begins from a point within the convex body and executes a specified number of steps, termed the *walk length*. Greater walk lengths produce final points less correlated with the starting position. The number of steps required to generate an uncorrelated point, one approximately sampled from a distribution, is known as the *mixing time*. Lovász and Vempala(2004) showed that $10^{10}O(n^2)$ is a sufficient mixing time for hit and run. However, such requirements are computationally intractable in practice. Following the empirical observations of Lovász and Deák (2012), we therefore constrain our implementation to n steps of hit-and-run, which offers a reasonable balance between sampling quality and computational efficiency.

Compromises. While ttc is theoretically sound, the implementation involves a few compromises that prioritize efficiency over strict adherence to theoretical guarantees:

1. For volume approximation, algorithms with theoretical guarantees require a running length of $10^{16}O(n^3)$ steps, which is impractical. Therefore, in our implementation (line 8), we use a practical volume algorithm that employs a convergence-based criterion to determine running length.
2. The sampling algorithm in line 15 relies on the hit-and-run method. Known results indicate that achieving independent samples requires $10^{10}n^2$ steps, which is also impractical. However, Lovász and Deák (2012) demonstrated that taking n steps of hit-and-run provides practically independent samples. As a result, we use n steps in our implementation. Notably, the theoretical guarantees of this sampling algorithm are in ℓ_1 norm, that is, the samples are guaranteed to be within ℓ_1 distance of the uniform distribution over the polytope. However, ttc requires samples to be within ℓ_∞ distance of the uniform distribution - a stronger requirement than the ℓ_1 guarantees provided by the hit-and-run sampling algorithm.

It is worth noting that the state-of-the-art tool, SharpSMT (in the non-exact mode i.e., when relying on polyvest), also makes similar compromises, and therefore, ttc and SharpSMT (in non-exact mode) have similar behavior. The exact mode of SharpSMT, on the other hand, fails to scale to larger instances, as demonstrated in the following section.

5.4 Experimental Evaluation

We evaluated our implementation concerning both efficiency and accuracy.

Baseline. For performance evaluation, we used the current state of the art volume computation framework SharpSMT[Ge24b], which offers two distinct modes: the polyvest algorithm, which estimates the volume, and vinci, which performs exact volume computation. To establish meaningful comparisons, we utilized polyvest for performance analysis and vinci for accuracy check. Another relevant baseline is [ACD20]. Their approach assumes a DNF representation, i.e., a set of polytopes as input; for this purpose we would pass the polytopes generated at line 2 of Algorithm 4. The publicly available implementation appears to be an early prototype, and in our environment we observed volume discrepancies on some benchmark-derived instances. Resolving these likely requires additional engineering, so we defer a thorough integration and evaluation of this baseline to future work.

Benchmarks. As a first step, we sought to rely on benchmarks from SMT-Lib, but these benchmarks could not be handled by polyvest or vinci owing to them containing extremely thin geometric regions where dimensions may be constrained to narrow ranges (e.g., $(0,10^{-7})$). In these cases, polyvest and vinci fail to handle the required precision and incorrectly classify these polytopes as degenerate with zero volume. To avoid results confounded by numerical precision rather than algorithmic differences, we do not include SMT-LIB in our evaluation. Accordingly, we focus on the construction of synthetic instances that are

Solver	Solved	PAR-2
vinci	142	6097.63
polyvest	145	6199.73
ttc	**1112**	**255.48**

Table 5.1: Performance comparison on 1131 instances.

disjunctions of intersecting polytopes, with each polytope defined in H-representation ($Ax \leq b$). In total, our benchmark suite consists of 1131 benchmarks.

1. We vary two parameters: (1) dimension parameter n: ranges from 6 to 34, (2) number of polytopes m: ranges from 6 to 42.
2. For each instance, we generate polytopes using three different geometric shapes studied by [CV16]: (a) *Cubes:* n-dimensional cubes with random bounds, then translated and rotated. (b) *Zonotypes:* Minkowski sum of n different d-dimensional vectors generated randomly. (c) *Simplex:* Shapes where all coordinates are nonnegative and sum to at most 1, defined as $\{x \in \mathbb{R}^n : \sum_{i=1}^{n} x_i \leq 1, x_i \geq 0\}$.

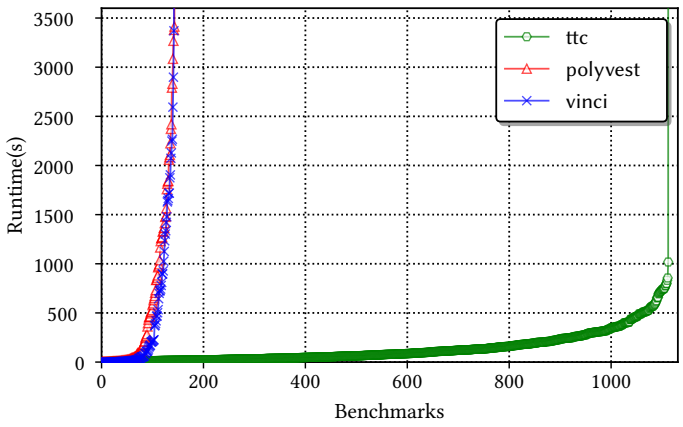
Environment. We conducted all our experiments on a high-performance computer cluster, with each node consisting of Intel Xeon Gold 6148 CPUs. We allocated one CPU core and a 5GB memory limit to each solver instance pair. To adhere to the standard timeout used in model counting competitions, we set the timeout for all experiments to 3600 seconds. We use values of $\varepsilon = 0.8$ and $\delta = 0.2$, in line with prior work in the model counting community.

With the above setup, we conduct extensive experiments to understand the following:

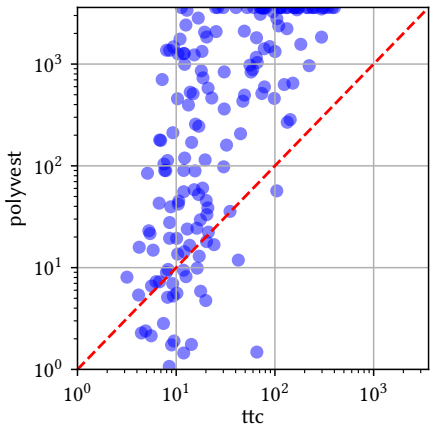
RQ1. How does the runtime performance of ttc compare to that of Baseline?
RQ2. How does the performance of ttc scale with different benchmark parameters?
RQ3. How accurate is the count computed by ttc in comparison to the exact count?

Summary of Results. ttc achieves a significant performance improvement over Baseline by finishing on 1112 instances in a benchmark set consisting of 1131, while Baseline could only finish on 145 instances. Baseline barely finishes on instances with more than 25 polytopes or 20 dimensions, while ttc seamlessly handles 40 polytopes of 35 dimensions. The accuracy of the approximate count is also noteworthy, with an average error of a count by ttc of only 0.059[2].

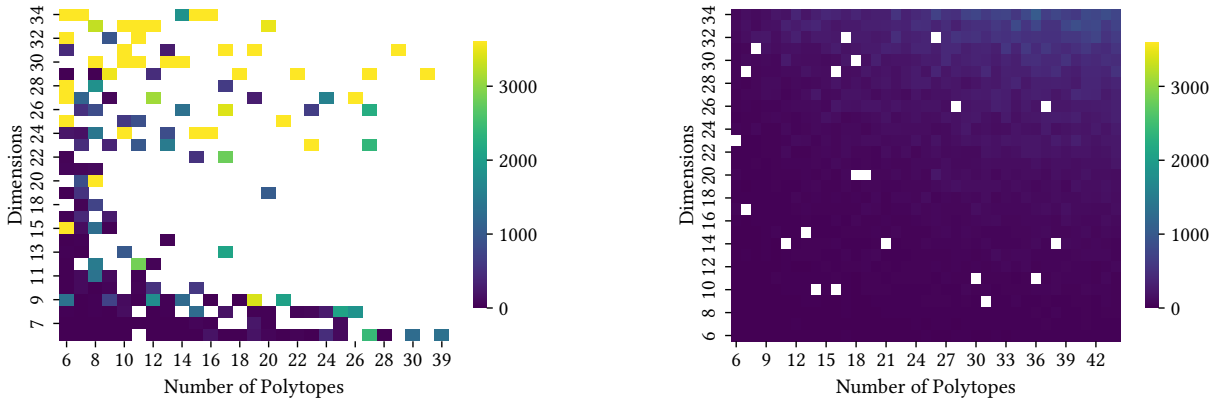
[2]Benchmarks and logfiles: https://doi.org/10.5281/zenodo.16782810



(a) Cactus plot comparing runtime of different tools.



(b) Runtime comparison of ttc w.r.t. polyvest.



(a) polyvest

(b) ttc

Figure 5.5: Time taken w.r.t. dimensions and number of polytopes.

5.4.1 Performance of ttc

Instances Solved. In Table 5.1, we compare the number of benchmarks that can be solved by Baseline and ttc. First, it is evident that the Baseline only solved 145 out of the 1131 benchmarks in the test suite, indicating its lack of scalability. Conversely, ttc solved 1112 instances, demonstrating a substantial improvement compared to Baseline.

Solving Time Comparison. A performance evaluation of Baseline and ttc is depicted in Figure 6.1a, which is a cactus plot comparing the solving time. The x-axis represents the number of instances, while the y-axis shows the time taken. A point (i, j) in the plot represents that a solver solved j benchmarks out of the 1131 benchmarks in the test suite in less than or equal to j seconds. The curves for Baseline and ttc indicate that for a few instances, Baseline was able to give a quick answer, while in the long run, ttc could solve many more instances given any fixed timeout.

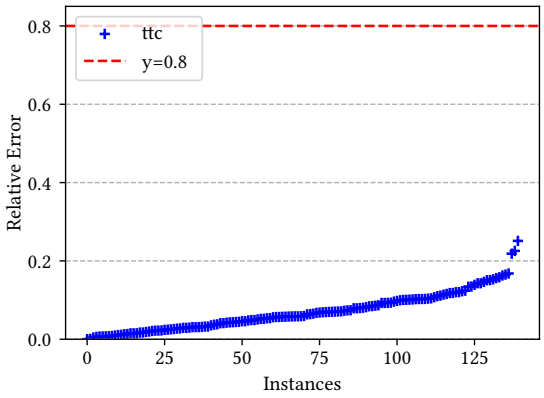


Figure 5.6: Quality of approximation: observed error.

Theoretical		0.1	0.4	0.8
Observed	Median	0.03	0.04	0.04
	Max	0.22	0.20	0.39

Table 5.2: Theoretical vs. observed error at different ε.

In Table 5.1 we also show the PAR-2 score of the solvers, which is the mean runtime over all instances, assigning a cost of $2T$ to each instance timed out at T. ttc shows significantly small PAR-2 score. In Figure 5.4b, we present a comparative analysis of solving times between ttc and Baseline. Each data point (x, y) represents an instance that was solved in x seconds by ttc and y seconds by Baseline. Points appearing below the dotted red diagonal line indicate instances where Baseline outperformed ttc in solving time. ttc demonstrates superior performance on the vast majority of instances.

Time utilization in different components. Theoretically, AIG to DNF conversion can take exponential amount of time. However, in our experiments, the DNF conversion time was negligible. The maximum time spent on this conversion was 1.2 seconds, with most instances completing the conversion in less than one second. The majority of the computation time was spent on calculating the individual volumes of the polytopes. For all instances that took more than 100 seconds to solve, over 60% of the time was dedicated to polytope volume computation.

5.4.2 Scaling

In Figure 5.5a and 5.5b, we evaluate the scalability of ttc and Baseline with respect to both the number of polytopes and dimensions. The plots are organized with the number of polytopes increasing from left to right along the x-axis, while the number of dimensions increases from top to bottom along the y-axis. Each pixel corresponds to a specific instance in our benchmark dataset, with the color intensity representing the solver's runtime performance. As demonstrated in Figure 5.5a, Baseline exhibits significant performance

degradation when handling instances exceeding 12 polytopes or 12 dimensions. By contrast, Figure 5.5b reveals that ttc efficiently processes configurations with up to 40 polytopes and 35 dimensions without notable performance deterioration.

5.4.3 Quality of Approximation

In our experimental evaluation, we found the exact volume of 142 benchmarks from vinci, enabling us to calculate the error made by ttc on these instances. We quantify the error made by ttc by the parameter $e = \frac{|b-s|}{b}$, where b represents the count from vinci and s from ttc. This measure is the *observed error*, analogous to the theoretical error guarantees provided by ttc. Analysis of all 142 cases found the median e to be 0.059, geometric mean 0.038, and maximum 0.39, contrasting sharply with a theoretical guarantee of 0.8. This signifies ttc substantially outperforms its theoretical bounds. In Figure 5.6 we plot the observed error, where x-axis, we have the benchmarks, and on the y-axis we have the observed errors. The observed error is below 0.2 for most of the instances.

In Table 5.2 we showed different observed errors when we run ttc with different ε values. The median and maximum observed errors decrease with the theoretical ε. The maximum *observed error* with $\varepsilon = 0.1$, is greater than theoretical, which is not unnatural, given the (ε, δ) guarantee nature.

5.5 Summary

This work introduces ttc, a scalable approximate SMT volume computation tool that demonstrates exceptional performance on practical benchmarks. Our approach harnesses probabilistic techniques to deliver theoretical guarantees on computation results, and empirical results significantly surpassing theoretical guarantees. Our work suggests several promising research directions. First, many formulas contain equality constraints that result in zero volume when computing in d dimensions. A natural extension would be to develop methods for correctly computing $(d-k)$-dimensional volume in such cases. Second, while we prioritized performance over strict theoretical guarantees in our implementation, experimental results consistently demonstrate error rates well below theoretical bounds. This raises the intriguing question, whether rigorous guarantees can be established for our current implementation without sacrificing its performance advantages.

Chapter 6

Functional Domains: Counting Skolem Functions

SMT solvers support uninterpreted functions (UFs): functions with a declared signature but no fixed interpretation. Given a formula over UFs, the standard counting question asks how many assignments to the variables satisfy the formula. In this chapter, we study a more semantic counting question: how many distinct interpretations of the uninterpreted function satisfy the formula? We focus on a highly restricted fragment of QF_UFBV, where counting naturally reduces to counting Skolem functions. Since synthesizing even one Skolem function is computationally challenging, counting all such functions is even more difficult. The main contribution of this chapter is SkolemFC, an approximate counting algorithm that estimates the number of satisfying Skolem functions without constructing any of them.

Uninterpreted functions (UFs) are an indispensable component of SMT. They are functions with declared signatures but no fixed interpretations. UFs are often used in verification tasks to abstract out complicated functions while preserving functional consistency. Since this thesis focuses on different quantitative questions in SMT, a natural question to ask in the presence of UFs is: *how many interpretations of an uninterpreted function satisfy a given formula?* This count gives a quantitative measure of the solution space of an SMT *specification*. In particular, it can be used to detect under-specified constraints, evaluate abstraction precision, and assess test-suite strength.

In its full generality, SMT allows uninterpreted functions to be used in highly expressive ways, including multiple applications of the same function and nested function applications. As a first step toward counting UF interpretations in this full generality, we focus on a highly restricted fragment of QF_UFBV. In this

restriction, each formula contains a single uninterpreted function symbol and exactly one application of that function. Moreover, each variable appears either as an input to the function or as part of its output, but not both. This restriction removes complications arising from congruence between multiple UF applications and from nested dependencies, while still retaining the central semantic difficulty of UF counting: counting complete function interpretations rather than satisfying assignments.

Despite this restriction, the resulting counting problem is nontrivial. We show that this fragment naturally reduces to counting Skolem functions. Since QF_UFBV has finite bit-vector domains, the inputs and outputs of the uninterpreted function can be bit-blasted into Boolean variables, yielding a Boolean specification $F(X, Y)$. Intuitively, a formula containing one UF application $y = f(x)$ induces a specification in which the interpretation of f plays the role of a Skolem function mapping inputs to outputs. Thus, each satisfying interpretation of the uninterpreted function corresponds to a Skolem function satisfying the induced specification.

In this chapter, we compute the number of possible Skolem functions for a given specification $F(X, Y)$. Counting Skolem functions is the natural analogue of #SAT for functional synthesis: whereas #SAT counts satisfying assignments of a Boolean formula, Skolem function counting counts satisfying functional witnesses for a specification. This problem has recently been explored in the context of counting level-2 solutions for valid quantified Boolean formulas (QBFs) [PMS23].

More precisely, recall that given two sets $X = x_1, \ldots, x_n$ and $Y = y_1, \ldots, y_m$ of variables, and a Boolean formula $F(X, Y)$ over $X \cup Y$, the problem of Boolean functional synthesis is to compute a vector $\Psi = \langle \psi_1, \ldots, \psi_m \rangle$ of Boolean functions ψ_i, often called Skolem functions, such that $\exists Y, F(X, Y) \equiv F(X, \Psi(X))$. Informally, given a specification between inputs and outputs, the task is to synthesize a function vector Ψ that maps each assignment of the inputs to an assignment of the outputs so that the combined assignment satisfies the specification whenever such an assignment exists. Skolem synthesis is a fundamental problem in formal methods and has been investigated by theoreticians and practitioners alike over the past few decades. The past few years have witnessed the development of techniques that showcase the promise of scalability in their ability to handle challenging specifications [JLH09; TV17; RTRS18; Aks+19; GSRM21].

The main contribution of this chapter is SkolemFC, an approximate counting algorithm that estimates the number of satisfying Skolem functions without constructing any of them. In summary, this chapter makes the following contributions. First, we formulate a restricted UF interpretation-counting problem for QF_UFBV. Second, we show that this problem naturally reduces to Skolem function counting. Third, we present SkolemFC, an approximate counter for estimating the number of satisfying Skolem functions. Finally, we empirically evaluate the approach on benchmark specifications.

The scalability of today's Skolem synthesis engines is reminiscent of the scalability of SAT solvers in the early 2000s. Motivated by the scalability of SAT solvers [Fro+21], researchers sought algorithmic

frameworks for problems beyond satisfiability, such as MaxSAT [ABL13; LM21], model counting [GSS21; CMV21], sampling [CMV14], and the like. The development of scalable techniques for these problems also helped usher in new applications, even though the initial investigation had not envisioned many of them. In a similar vein, motivated in part by this development of scalable techniques for functional synthesis, we investigate the Skolem counting problem. We observe in Section 6.1.1 that algorithms for such tasks also have potential applications in security and the engineering of specifications. Being a natural problem, we will see that our study also naturally leads to deep technical connections between counting functions and counting propositional models and the development of new techniques, which is of independent interest.

Counting Skolem functions indeed raises new technical challenges. The existing techniques developed in the context of propositional model counting either construct (implicitly or explicitly) a representation of the space of all models [Thu06; SRSM19; DPV20] or at least enumerate a small number of models [CMV13; CMV16; SM19; YM23], which in practice amounts to a few tens to hundreds of models of formulas constrained by random XORs. Such approaches are unlikely to work efficiency in the context of Skolem function counting, where even finding one Skolem function is hard.

The primary contribution of this work is the development of a novel algorithmic framework, called SkolemFC, that approximates the Skolem function count with a theoretical guarantee, using only linearly many calls to an approximate model counter and almost-uniform sampler. First, we observe that Skolem function counting can be reduced to an exponential number of model counting calls, serving as a baseline Skolem function counter. The core technical idea of SkolemFC is to reduce the problem of approximate Skolem function counting to only linearly many (in $m = |Y|$) calls to propositional model counters. Of particular note is the observation that SkolemFC can provide an approximation to the number of Skolem functions without enumerating even one Skolem function. As approximate model counting and almost-uniform sampling can be done by logarithmically many calls to SAT oracle, we show that Skolem function counting can also be reduced to polynomially many calls to a SAT oracle.

To measure the impact of the algorithm, we implement SkolemFC and demonstrate its potential over a set of benchmarks arising from prior studies in the context of Skolem function synthesis. Out of 609 instances, SkolemFC could solve 375 instances, while a baseline solver could solve only eight instances. For context, the state-of-the-art Skolem function synthesis tool Manthan2 [GSRM21] effectively tackled 509 instances from these benchmarks, while its precursor, Manthan [GRM20], managed only 356 instances with a timeout of 7200 seconds.

6.1 Applications and Related Work

This work has an interesting connection to the counting of Skolem functions. This connection opens up several potential applications and relates to a body of work that was not covered in Chapter 2 of the thesis,

where we discussed related work on SMT counting and its applications. In this section, we therefore discuss related work on Skolem function synthesis and applications of Skolem function counting.

6.1.1 Applications

This problem of Skolem function counting arises in several potential application areas.

Specification engineering. The first and primary motivation stems from the observation that specification synthesis [ADG16; PFMD21] (i.e., the process of constructing $F(X, Y)$) and function synthesis form part of the iterative process wherein one iteratively modifies specifications based on the functions that are constructed by the underlying engine. In this context, one helpful measure is to determine the number of possible semantically different functions that satisfy the specification, as often a large number of possible Skolem functions indicates the vagueness of specifications and highlights the need for strengthening the specification. Note that the use of the count is qualitative here, and hence an approximate order of magnitude (log count) suffices.

Diversity at the specification level. In system security and reliability, a classic technique is to generate and use a diverse variety of functionally equivalent implementations of components [BM15]. Although classically, this is achieved by transformations of the code that preserve the function computed, we may also be interested in producing a variety of functions that satisfy a common specification. Unlike transformations on the code, it is not immediately clear whether a specification even admits a diverse collection of functions – indeed, the function may be uniquely defined. Thus, counting the number of such functions is necessary to assess the potential value of taking this approach, and again a rough order of magnitude estimate suffices. Approximate counting of the functions may also be a useful primitive for realizing such an approach.

Evaluation of a random Skolem function. Although synthesis of Skolem functions remains challenging in general, we note that approximate counting enables a kind of incremental evaluation by using the standard techniques for reducing sampling to counting. More concretely, given a query input, we can estimate the number of functions that produce each output: this is trivial if the range is small (e.g., Boolean), and otherwise, we can introduce random XOR constraints to incrementally specify the output. Once an output is specified for the query point, we may retain these constraints when estimating the number of consistent functions for subsequent queries, thereby obtaining an approximately uniform function conditioned on the answers to the previous queries.

6.1.2 Related Work

Despite the lack of prior studies focused on the specific problem of counting Skolem functions, significant progress has been made in synthesizing these functions. Numerous lines of research have emerged in the

field of Skolem function synthesis. The first, incremental determinization, iteratively pinpoints variables with distinctive Skolem functions, making decisions on any remaining variables by adding provisional clauses that render them deterministic [RS16; RTRS18; Rab19]. The second line of research involves obtaining Skolem functions by eliminating quantifiers using functional composition and reducing the size of composite functions through the application of Craig interpolation [JLH09; Jia09]. The third, CEGAR-style approaches, commence with an initial set of approximate Skolem functions, and proceed to a phase of counter-example guided refinement to improve upon these candidate functions [Joh+15; ACJS17; Aks+18]. Work on the representation of specification $F(X, Y)$ has led to efficient synthesis using ROBDD representation and functional composition [BJ11], with extensions to factored specifications [TV17; CFTV18]. Notable advancements include the new negation normal form, SynNNF, amenable to functional synthesis [Aks+19]. Finally, a data-driven method has arisen [GRM20; GSRM21], relying on constrained sampling to generate satisfying assignments for a formula F.

A related problem in the descriptive complexity of functions definable by counting the Skolem functions for *fixed* formulas have been shown to characterize $\#AC_0$ [HV19]. By contrast, we are interested in the problem where the *formula* is the input. Our algorithm also bears similarity to the FPRAS proposed for the descriptive complexity class $\#\Sigma_1$ [DHKV21], which is obtained by an FPRAS for counting the number of *functions* satisfying a DNF over atomic formulas specifying that the functions must/must not take specific values at specific points. Nevertheless, our problem is fundamentally different in that it is easy to find functions satisfying such DNFs, whereas synthesis of Skolem functions is unlikely to be possible in polynomial time.

6.2 Preliminaries and Problem Statement

We use the notation of Skolem functions instead of uninterpreted functions, as this significantly simplifies the notation.

We use lowercase letters (with subscripts) to denote propositional variables and uppercase letters to denote a subset of variables. The formula $\exists Y F(X, Y)$ is existentially quantified in Y , where $X = \{x_1, \ldots, x_n\}$ and $Y = \{y_1, \ldots, y_m\}$. By n and m we denote the number of X and Y variables in the formula. Therefore, $n = |X|, m = |Y|$. For simplicity, we write a formula $F(X, Y)$ as F if the X, Y is clear from the context. A *model* is an assignment (true or false) to all the variables in F, such that F evaluates to true. Let $\mathrm{Sol}(F)_{\downarrow S}$ denote the set of models of formula F projected on $S \subseteq X \cup Y$. If $S = X \cup Y$, we write the set as $\mathrm{Sol}(()F)$. Let σ be a partial assignment for the variables X of F. Then $\mathrm{Sol}(F \wedge (X = \sigma))$ denotes the models of F where $X = \sigma$.

Propositional Sampling. Given a Boolean formula F and a projection set S, a *sampler* is a probabilistic algorithm that generates a random element in $\mathrm{Sol}(F)_{\downarrow S}$. An *almost uniform sampler* G takes a tolerance

parameter ε along with F and S, and guarantees $\forall y \in \mathrm{Sol}(F)_{\downarrow S}, \frac{1}{(1+\varepsilon)|\mathrm{Sol}(F)_{\downarrow S}|} \leq \Pr[G(F,S,\varepsilon) = y] \leq \frac{(1+\varepsilon)}{|\mathrm{Sol}(F)_{\downarrow S}|}$. Delannoy and Meel [DM22] showed, $\log(n)$ many calls to a SAT oracle suffices to generate almost uniform samples from a formula with n variables.

Skolem Functions. Given a Boolean specification $F(X,Y)$ between set of inputs $X = \{x_1, \ldots, x_n\}$ and vector of outputs $Y = \langle y_1, \ldots, y_m \rangle$, a function vector $\Psi(X) = \langle \psi_1(X), \psi_2(X), \ldots, \psi_m(X) \rangle$ is a Skolem function vector if $y_i \leftrightarrow \psi_i(X)$ and $\exists Y F(X,Y) \equiv F(X,\Psi)$. We refer to Ψ as the Skolem function vector and Ψ_i as the Skolem function for y_i. We'll use the notation $\mathrm{Skolem}(F,Y)$ to denote the set of possible $\Psi(X)$ satisfying the condition $\exists Y F(X,Y) \equiv F(X,\Psi(X))$. Two Skolem function vectors Ψ_1 and Ψ_2 are different, if there exists an assignment $\sigma \in \mathrm{Sol}(F)_{\downarrow X}$ for which $\Psi_1(\sigma) \neq \Psi_2(\sigma)$.

For a specification $\exists Y F(X,Y)$, the number of Skolem functions itself can be as large as $2^{n \cdot 2^m}$, and the values of n and m are quite large in many practical cases. Beyond being a theoretical possibility, the count of Skolem functions is often quite big, and such values are sometimes difficult to manipulate and store as 64-bit values. Therefore, we are interested in the logarithm of the counts, and define the problem of approximate Skolem function counting as following:

Restricted Uninterpreted Functions considered here. Within a signature Σ, an *uninterpreted function* is a non-constant function symbol $f \in \Sigma^F$ on which the theory imposes no axioms: a model may interpret f as *any* total function $f^\alpha : \mathcal{U}^n \to \mathcal{U}$ of the declared type. We study a restricted fragment of QF_UFBV subject to three conditions: (i) Σ contains a single uninterpreted symbol f; (ii) the formula φ contains exactly one application of f; and (iii) the variables $\mathrm{Vars}(\varphi)$ are partitioned into an input block, the arguments of f, and an output block, the variables related to its result, with no variable belonging to both. Up to this partition, φ has the form $\bar{y} = f(\bar{x}) \wedge \varphi_0(\bar{x}, \bar{y})$, where $\bar{x}$ and $\bar{y}$ are disjoint vectors of bit-vector variables and φ_0 is a bit-vector constraint. Restricting to one application removes congruence between distinct applications of f and nested terms such as $f(f(\cdot))$, while retaining the central difficulty of the problem: the objects being counted are interpretations of f, that is, complete functions, rather than satisfying assignments.

From Uninterpreted Functions to Skolem Functions. Since every bit-vector domain is finite, this fragment bit-blasts into a purely Boolean problem, as we did in Chapter 3. Writing $f : \{0,1\}^n \to \{0,1\}^m$ for the type of f, its argument bits become the inputs $X = \{x_1, \ldots, x_n\}$ and its result bits the outputs $Y = \langle y_1, \ldots, y_m \rangle$; bit-blasting $\bar{y} = f(\bar{x}) \wedge \varphi_0$ then yields a Boolean specification $F(X,Y)$. An interpretation f^α is now just a Boolean function vector $\Psi(X) = \langle \psi_1(X), \ldots, \psi_m(X) \rangle$, and it satisfies f exactly when $\exists Y\, F(X,Y) \equiv F(X,\Psi(X))$. This is precisely the condition defining a Skolem function vector for F, so counting UF interpretations reduces to *counting Skolem functions*, which we now define.

Algorithm 6 Stopping Rule (ε, δ)

$t \leftarrow 0, x \leftarrow 0, s \leftarrow 4\ln(2/\delta)(1+\varepsilon)/\varepsilon^2$
while $x < s$ **do**
    $t \leftarrow t + 1$
    Generate Random Variable Z_t
    $x \leftarrow x + Z_t$
Est $\leftarrow s/t$
return Est

Problem Statement. Given a Boolean specification $F(X, Y)$, tolerance parameter ε, confidence parameter δ, let $\ell = \log(|\mathsf{Skolem}(F, Y)|)$, the task of approximate Skolem function counting is to give an estimate Est, such that $\Pr\left[(1-\varepsilon)\ell \leq \mathsf{Est} \leq (1+\varepsilon)\ell\right] \geq 1 - \delta$.

In practical scenarios, the input *specification* is often given as a quantified Boolean formula (QBF). The output of the synthesis problem is a function, which is expressed as a Boolean circuit. In our setting, even if two functions have different circuits, if they have identical input-output behavior, we consider them to be the same function. For example, let $f_1(x) = x$ and $f_2 = \neg(\neg x)$. We'll consider f_1 and f_2 to be the same function.

Illustrative Example. Let's examine a formula defined on three sets of variables X, Y_0, Y_1, where each set contains five Boolean variables, interpreted as five-bit integers: $\exists Y_0 Y_1 F(X, Y_0 Y_1)$, where F represents the constraint for factorization, $X = Y_0 \times Y_1, Y_0 \leq Y_1, Y_0 \neq 1$. The number of Skolem functions of F gives the number of distinct ways to implement a factorization function for 5-bit input numbers. There exist multiple X's for which there are multiple factorizations: A Skolem function S_1 may factorize 12 as 4×3 and a function S_2 may factorize 12 as 2×6.

Stopping Rule Algorithm. Let $Z_1, Z_2, \ldots$ denote independently and identically distributed (i.i.d.) random variables taking values in the interval $[0, 1]$ and with mean μ. Intuitively, Z_t is the outcome of experiment t. Then the Stopping Rule algorithm (Algorithm 6) approximates μ as stated by Theorem 6.2.1 [DKLR95; DL97].

Theorem 6.2.1 (Stopping Rule Theorem)**.** For all $0 < \varepsilon \leq 2, \delta > 0$, if the Stopping Rule algorithm returns Est, then $\Pr[\mu(1-\varepsilon) \leq \mathsf{Est} \leq \mu(1+\varepsilon)] > (1-\delta)$.

6.3 Algorithm SkolemFC: Counting without Synthesis

In this section, we introduce the primary contribution of this work: the SkolemFC algorithm. The algorithm takes in a formula $F(X, Y)$ and returns an estimate for $\log(|\mathsf{Skolem}(F, Y)|)$. We first outline the key technical ideas that inform the design of SkolemFC and then present the pseudocode for implementing this algorithm.

6.3.1 Core Ideas

Since finding even a single Skolem function is computationally expensive, our approach is to estimate the count of Skolem functions without enumerating even a small number of Skolem functions. The key idea is to observe that the number of Skolem functions can be expressed as a product of the model counts of formulas. A Skolem function $\Psi \in \mathsf{Skolem}(F, Y)$ is a function from 2^X to 2^Y. A useful quantity in the context of counting Skolem functions is to define, for every assignment $\sigma \in 2^X$, the set of elements in 2^Y that $\Psi(X)$ can belong to. We refer to this quantity as $\mathsf{range}(\sigma)$ and formally define it as follows:

Definition 6.3.1.

$$\mathsf{range}(\sigma) = \begin{cases} \mathsf{Sol}(F \wedge (X = \sigma))_{\downarrow Y} & |\mathsf{Sol}(F \wedge (X = \sigma))| > 0 \\ 1 & \text{otherwise} \end{cases}$$

Lemma 6.3.2. $|\mathsf{Skolem}(F, Y)| = \prod_{\sigma \in 2^X} |\mathsf{range}(\sigma)|$

Proof. First of all, we observe that

$$\forall \sigma \in 2^X, \forall \pi \in \mathsf{range}(\sigma), \exists \Psi \text{ s.t. } \Psi(\sigma) = \pi$$

which is easy to see for all $\sigma \in 2^X$ for which there exists $\pi \in 2^Y$ such that $F(\sigma, \pi) = 1$. As for $\sigma \in 2^X$ for which there is no π such that $F(\sigma, \pi) = 1$, Skolem functions that differ solely on inputs $\sigma \notin \mathsf{Sol}(F)_{\downarrow X}$ are regarded as identical. Consequently, such inputs have no impact on the count of distinct Skolem functions, resulting in $\mathsf{range}(\sigma) = 1$ for these cases. Recall, $\mathsf{Skolem}(F, Y)$ is the set of all functions from 2^X to 2^Y. It follows that $|\mathsf{Skolem}(F, Y)| = \prod_{\sigma \in 2^X} |\mathsf{range}(\sigma)|$. □

Lemma 6.3.2 allows us to develop a connection between Skolem function counting and propositional model counting. As stated in the problem statement, we focus on estimating $\log |\mathsf{Skolem}(F, Y)|$. To formalize our approach, we need to introduce the following notation:

Algorithm 7 SkolemFC$(F(X,Y),\varepsilon,\delta)$

1: $\varepsilon_f \leftarrow 0.6\varepsilon, \delta_f \leftarrow 0.4\delta, s \leftarrow 4\ln(2/\delta_f)(1+\varepsilon_f)/{\varepsilon_f}^2$
2: $\varepsilon_s \leftarrow 0.2\varepsilon, \delta_c \leftarrow 0.4\delta/ms, \varepsilon_c \leftarrow 4\sqrt{2}-1$
3: $\varepsilon_g \leftarrow 0.1\varepsilon, \delta_g \leftarrow 0.1\delta$
4: $G(X,Y,Y') := F(X,Y) \wedge F(X,Y') \wedge (Y \neq Y')$
5: **while** $x < s$ **do**
6: $\quad \sigma \leftarrow \mathsf{AlmostUniformSample}(G, X, \varepsilon_s)$
7: $\quad c \leftarrow \log(\mathsf{ApproxCount}(F \wedge (X=\sigma), \varepsilon_c, \delta_c))/m$
8: $\quad x \leftarrow x + c$
9: $\quad t \leftarrow t + 1$
10: $g \leftarrow \mathsf{ApproxCount}(G, X, \varepsilon_g, \delta_g)$
11: $\mathsf{Est} \leftarrow s/\mathsf{iter} \times m \times g$
12: **if** $(g\log(1+\varepsilon_c) > 0.1\mathsf{Est})$ **then return** $\perp$
13: **return** Est

Proposition 6.3.3.

$$\log|\mathsf{Skolem}(F,Y)| = \sum_{\sigma\in S_2} \log|R_{F\wedge(X=\sigma)}|$$
$$\text{where, } S_2 := \{\sigma \in 2^X \mid |\mathsf{Sol}(F\wedge(X=\sigma))| \geq 2\}$$

Proof. From Lemma 6.3.2, we have $|\mathsf{Skolem}(F,Y)| = \prod_{\sigma\in 2^X} |\mathsf{range}(\sigma)|$. Taking logs on both sides, partitioning 2^X into S_2 and $2^X \backslash S_2$, and observing that $\log|\mathsf{range}(\sigma)| = 0$ for $\sigma \notin S_2$, we get the desired result. □

6.3.2 Algorithm Description

The pseudocode for SkolemFC is delineated in Algorithm 7. It accepts a formula $\exists Y F(X,Y)$, a tolerance level ε, and a confidence parameter δ. The algorithm SkolemFC then provides an approximation of $\log|\mathsf{Skolem}(F,Y)|$ following Proposition 6.3.3. To begin, SkolemFC almost-uniformly samples σ from S_2 at random in line 6, utilizing an almost-uniform sampler. Subsequently, SkolemFC approximates $|R_{F\wedge(X=\sigma)}|$ through an approximate model counter at line 7. The estimate Est is computed by taking the product of the mean of c's and $|S_2|$. In order to sample $\sigma \in S_2$, SkolemFC constructs the formula G whose solutions, when projected to X represent all the assignments $\sigma \in S_2$ (line 4). Finally, SkolemFC returns the estimate Est as logarithm of the Skolem function count.

The main loop of SkolemFC (from lines 5 to 9) is based on the Stopping Rule algorithm presented in **??**. The Stopping Rule algorithm is utilized to approximate the mean of a collection of i.i.d. random variables that fall within the range $[0,1]$. The method repeatedly adds the outcomes of these variables until the cumulative value reaches a set threshold s. This threshold value is influenced by the input parameters ε and δ. The result yielded by the algorithm is represented as s/t, where t denotes the number of random

variables aggregated to achieve the threshold s. In the context of SkolemFC, this random variable is defined as $\log(\mathsf{ApproxCount}(F \wedge (X = \sigma), \varepsilon_c, \delta_c))/m$. Line 12 asserts that the error introduced by the approximate model counting oracle is within some specific bound.

Oracle Access. We assume access to approximate model counters and almost-uniform samplers as oracles. The notation $\mathsf{ApproxCount}(F, P, \varepsilon, \delta)$ represents an invocation of the approximate model counting oracle on Boolean formula F with a projection set P, tolerance parameter ε, and confidence parameter δ. $\mathsf{AlmostUniformSample}(F, S, \varepsilon)$ denotes an invocation of the almost uniform sampler on a formula F, with projection set S and tolerance parameter ε. The particular choice of values of ε_s, ε_c, δ_c, ε_g, δ_g used in the counting and sampling oracle aids the theoretical guarantees.

6.3.3 Illustrative Example

We will now examine the specification of factorization as outlined in Section 6.2, and investigate how SkolemFC estimates the count of Skolem functions meeting that specification.

1. In line 4, SkolemFC constructs G such that $|\mathsf{Sol}(G)_{\downarrow X}| = 7$, $\mathsf{Sol}(G)_{\downarrow X} = \{12, 16, 18, 20, 24, 28, 30\}$.
2. In line 6, it samples σ from $\mathsf{Sol}(G)_{\downarrow X}$. Let's consider $\sigma = 30$. Then $\mathsf{Sol}(F \wedge (X = \sigma))_{\downarrow Y_0, Y_1} = \{(2, 15), (3, 10), (5, 6)\}$. Therefore, $c = \log(3)$ in line 7.
3. Suppose in the next iteration it samples $\sigma = 16$. Then $\mathsf{Sol}(F \wedge (X = \sigma)) = \{(2, 8), (4, 4)\}$. Therefore, $c = \log(2)$ in line 7.
4. Now suppose that the termination condition of line 5 is reached. At this point, the estimate Est returned from line 11 will be $\approx \frac{(\log(3)+\log(2))}{2} \times 7 \approx 6$.
5. Finally, SkolemFC will return the value Est= 6.

Note that the approach is in stark contrast to the state-of-the-art counting techniques in the context of propositional models, which either construct a compact representation of the entire solution space or rely on the enumeration of a small number of models.

6.4 Complexity and Guarantees

Let $F(X, Y)$ be a propositional CNF formula over variables X and Y. In the section we'll show that SkolemFC works as an approximate counter for the number of Skolem functions. We create a formula $G(X, Y, Y') = F(X, Y) \wedge F(X, Y') \wedge (Y \neq Y')$ from $F(X, Y)$, where Y' is a fresh set of variables and $m = |Y'|$. We show that, if we pick a solution from $G(X, Y, Y') = F(X, Y) \wedge F(X, Y') \wedge (Y \neq Y')$, then the assignment to X in that solution to G will have at least two solutions in $F(X, Y)$.

Lemma 6.4.1.

$$\mathsf{Sol}(G)_{\downarrow X} = \left\{\sigma \mid \sigma \in 2^X \wedge |\mathsf{Sol}(F \wedge (X = \sigma))| \geq 2\right\}$$

Proof. We can write the statement alternatively as,

$$\sigma \in \mathsf{Sol}(G, X) \iff |\mathsf{Sol}(F \wedge (X = \sigma))| \geq 2$$

($\implies$) For every element $\sigma \in \mathsf{Sol}(G)$, we write σ as $\langle \sigma_{\downarrow X}, \sigma_{\downarrow Y}, \sigma_{\downarrow Y'} \rangle$. Now according to the definition of G, both $\langle \sigma_{\downarrow X}, \sigma_{\downarrow Y} \rangle$ and $\langle \sigma_{\downarrow X}, \sigma_{\downarrow Y'} \rangle$ satisfy F. Moreover, $\sigma_{\downarrow Y}$ and $\sigma_{\downarrow Y'}$ are not equal. Therefore, $|\mathsf{Sol}(F \wedge (X = \sigma))| \geq 2$.

($\impliedby$) If $|\mathsf{Sol}(F \wedge (X = \sigma))| \geq 2$, then $F(X, Y)$ has solutions of the form $\langle \sigma, \gamma_1 \rangle$ and $\langle \sigma, \gamma_2 \rangle$, where $\gamma_1 \neq \gamma_2$. Now $\langle \sigma \gamma_1 \gamma_2 \rangle$ satisfies G. □

Theorem 6.4.2. SkolemFC takes in input $F(X, Y)$, $\varepsilon > 0$, and $\delta \in (0, 1]$, and returns Est such that

$$\Pr\left[(1 - \varepsilon)\ell \leq \mathsf{Est} \leq (1 + \varepsilon)\ell\right] \geq 1 - \delta$$

where, $\ell = \log(|\mathsf{Skolem}(F, Y)|)$. Furthermore, it makes $\tilde{O}\left(\frac{m}{\varepsilon^2} \ln \frac{2}{\delta}\right)$ many calls to a SAT oracle, where $\tilde{O}$ hides poly-log factors in parameters $m, n, \varepsilon, \delta$.

Proof. We prove the correctness and complexity of the algorithm in two different parts.

Proof for Correctness

There are four sources of error in the approximation of the count. In the following table we list those. We'll use the following shorthand notation $\mathsf{C_L}$ for $\log(|\mathsf{Sol}(()F \wedge X = \sigma)|)$.

Reason of Error	Estimated Amount
Approximation by SkolemFC (E_F)	$\varepsilon_f \sum_{\sigma \in S_2} \mathsf{C_L}$
Non-uniformity in Sampling (E_S)	$\varepsilon_s \sum_{\sigma \in S_2} \mathsf{C_L}$
Approximation by ApproxCount in line 7(E_M)	$\lvert S_2 \rvert \log(1 + \varepsilon_c)$
Approximation by ApproxCount in line 10 (E_G)	$\varepsilon_g \sum_{\sigma \in S_2} \mathsf{C_L} + \varepsilon_g E_M$

In the following part, we will establish the significance of the bounds on individual errors, culminating in a demonstration of how these errors combine to yield the error bound for SkolemFC.

(1) Bounding the error from SkolemFC

For a given formula $\exists Y F(X, Y)$, we partition 2^X, the assignment space of X, into three subsets, S_0, S_1, S_2 as discussed in Section 6.3.1. In line 5 of Algorithm 7, we pick an assignment to X almost uniformly at random from $\mathsf{Sol}(G)_{\downarrow X}$. Let σ_i be the random assignment picked at i^{th} iteration of the for loop. For each of the random event of picking an assignment σ_i, we define a random variable W_i such that, W_i becomes a function of σ_i in the following way:

$$W_i = \frac{\log(|\mathsf{Sol}(F \wedge (X = \sigma_i))|)}{m}$$

Therefore, we have iter random variables $W_1, W_2, \ldots W_{\text{iter}}$ on the sample space of S_2. Now as σ_i is picked from $\mathsf{Sol}(G)_{\downarrow X}$ at line 4 of SkolemFC, from Lemma 6.4.1 we know, $2 \leq |\mathsf{Sol}(F \wedge (X = \sigma_i))|$. Therefore the following holds,

$$\forall i, 2 \leq |\mathsf{Sol}(F \wedge (X = \sigma_i))| \leq 2^m$$

Hence, from the definition of W_i:

$$\forall i, \frac{1}{m} \leq W_i \leq 1$$

Now assume that the model counting oracle of line 7 returns an exact model count instead of an approximate model count; and the value retured at line 11 is EEst instead of Est. Now, due to approximation by SkolemFC, according to Theorem 6.2.1 (stopping rule theorem),

$$\Pr\left[\left|\frac{\mathsf{EEst}}{m|S_2|} - \mu\right| \leq \varepsilon_f \mu\right] \geq 1 - \delta_f \tag{6.1}$$

(2) Bounding the error from sampling oracle

Let p_j be the probability an assignment $\sigma_j \in S_2$ appears. Then,

$$\mu = E[W_i] = \sum_{\sigma_j \in S_2} p_j \frac{\mathsf{LC}(\sigma_j)}{m}$$

Now from the guarantee provided by the almost oracle, we know

$$\frac{1}{(1+\varepsilon)|S_2|} \leq p_j \leq \frac{(1+\varepsilon)}{|S_2|}$$

Therefore,

$$\frac{\sum_{\sigma \in S_2} \mathsf{C_L}}{m|S_2|(1+\varepsilon)} \leq \mu \leq \frac{(1+\varepsilon)\sum_{\sigma \in S_2} \mathsf{C_L}}{m|S_2|}$$

$$\left| m|S_2|\mu - \sum_{\sigma \in S_2} \mathsf{C_L} \right| \leq E_S \tag{6.2}$$

(3) Bounding the error from counting oracle in line 7

We'll use the following shorthand notation $\mathsf{LA}(\sigma)$ for $\log(\mathsf{ApproxCount}(F \wedge X = \sigma, \varepsilon_c, \delta_c))$. From properties of ApproxCount oracle, for arbitrary σ,

$$\Pr[|\mathsf{C_L} - \mathsf{LA}(\sigma)| \leq \log(1+\varepsilon_c)] \geq 1 - \delta_c$$

Let $S_s \subseteq S_2$ be the set of t samples picked up at line 6. (t is the number of iterations taken by the algorithm, as in line 9):

$$\Pr\left[\left|\sum_{\sigma \in S_s} \mathsf{LA}(\sigma) - \sum_{\sigma \in S_s} \mathsf{C_L}\right| \leq t \log(1+\varepsilon_c)\right] \geq 1 - t\delta_c$$

Now $\mathsf{EEst} = \frac{|S_2|}{t} \sum_{\sigma \in S_s} \mathsf{C_L}$ and let $\mathsf{Est}' = \frac{|S_2|}{t} \sum_{\sigma \in S_s} \mathsf{LA}(\sigma)$. Therefore,

$$\Pr\left[|\mathsf{EEst} - \mathsf{Est}'| \leq E_M\right] \geq 1 - t\delta_c \tag{6.3}$$

(4) Bounding the error from counting oracle in line 10

From the property of ApproxCount oracle running with parameters $(\varepsilon_g, \delta_g)$, we get

$$\Pr\left[\frac{|S_2|}{1+\varepsilon_g} \leq g \leq (1+\varepsilon_g)|S_2|\right] \geq 1 - \delta_g$$

Now, $\mathsf{Est} = \frac{g}{t} \sum_{\sigma \in S_s} LA(\sigma)$, which gives us:

$$\Pr\left[\frac{\mathsf{Est}'}{1+\varepsilon_g} \leq \mathsf{Est} \leq (1+\varepsilon_g)\mathsf{Est}'\right] \geq 1 - \delta_g$$

$$\Pr\left[|\mathsf{Est} - \mathsf{Est}'| \leq (1+\varepsilon_g)\mathsf{Est}'\right] \geq 1 - \delta_g \tag{6.4}$$

Now, combining Equation (6.4) with Equation (6.3), we get

$$\Pr\left[|\mathsf{Est} - \mathsf{EEst}| \leq E_M + E_G\right] \geq 1 - \delta_g - t\delta_c \tag{6.5}$$

(5) Summing-up all the errors

Combining Equation (6.1) and (6.2)

$$\Pr\left[\left|\mathsf{EEst} - \sum_{\sigma \in S_2} \mathsf{C}_\mathsf{L}\right| \leq E_S + E_F\right] \geq 1 - \delta_f \tag{6.6}$$

Combining Equation (6.5) and (6.6), putting $\delta = \delta_f + \delta_g + t\delta_c$

$$\Pr\left[\left|\mathsf{Est} - \sum_{\sigma \in S_2} \mathsf{C}_\mathsf{L}\right| \leq E_S + E_M + E_F + E_G\right] \geq 1 - \delta \tag{6.7}$$

According to Algorithm 7, $\varepsilon_s = 0.2\varepsilon$ and $\varepsilon_f = 0.6\varepsilon$. Now, line 12 asserts that $E_M \leq 0.1\varepsilon \sum_{\sigma \in 2^X} \mathsf{C}_\mathsf{L}$. That makes $E_S + E_M + E_F + E_G \leq \varepsilon \sum_{\sigma \in S_2} \mathsf{C}_\mathsf{L}$. Therefore,

$$\Pr\left[\left|\mathsf{Est} - \sum_{\sigma \in 2^X} \mathsf{C}_\mathsf{L}\right| \leq \varepsilon \sum_{\sigma \in 2^X} \mathsf{C}_\mathsf{L}\right] \geq 1 - \delta$$

Or,

$$\Pr\left[(1-\varepsilon)\ell \leq \mathsf{Est} \leq (1+\varepsilon)\ell\right] \geq 1 - \delta$$

where $\ell = \log(|\mathsf{Skolem}(F, Y)|)$.

Proof for Complexity

The formula G in line 4 of SkolemFC can be constructed with a time complexity that is a polynomial in $|F|$. As the minimum value for c in line 7 is 1, the for loop, located between lines 5 and 9, is executed a maximum of $\mathsf{iter} = \left(4m\ln(2/\delta_f)(1+\varepsilon_f)/\varepsilon_f^2\right)$ times. During each iteration, the algorithm makes one call to an approximate counting oracle and an almost uniform sampling oracle, resulting in a total of iter calls to these oracles. In line 10, we invoke approximate model counting oracle once more. Following [Sto83] and [DM22], $\log(m+n)$ many calls to a SAT oracle suffices to give both approximate counting and almost uniform sampling. Therefore, SkolemFC returns within $\tilde{O}\left(\frac{m}{\varepsilon^2}\ln\frac{2}{\delta}\right)$ many calls to a SAT oracle, where $\tilde{O}$ hides poly-log factors in parameters $m, n, \varepsilon, \delta$. □

6.5 Experimental Evaluation

We conducted a thorough evaluation of the performance and accuracy of results of the SkolemFC algorithm by implementing a functional prototype in C++. The following experimental setup was used to evaluate the performance and quality of results of the SkolemFC algorithm[1].

Baseline. To assess the effectiveness of SkolemFC, we conducted comparisons with the tool QCounter [PMS23], which is designed for level-2 solution counting. However, given that QCounter is restricted to true QBFs, we aim to develop an additional baseline counter to broaden our evaluation scope.

A possible approach to count Skolem functions, following Lemma 6.3.2, is given in Algorithm 8. The Count(F) oracle denotes an invocation of exact model counter. We implemented that to compare with SkolemFC. In the implementation, we relied on the latest version of Ganak [SRSM19] to get the necessary exact model counts. We use a modified version of the SAT solver CryptoMiniSat [SNC09] as AllSAT solver to find all solutions of a given formula, projected on X variables. We call this implementation Baseline in the following part.

Algorithm 8 Baseline($F(X, Y)$)

1: Est $\leftarrow 0$
2: $G(X, Y, Y') := F(X, Y) \wedge F(X, Y') \wedge (Y \neq Y')$
3: SolG $\leftarrow$ AllSAT(G, X)
4: **for each** $\sigma \in$ SolG **do**
5: $\quad c \leftarrow \log(\text{Count}(F \wedge (X = \sigma)))$
6: $\quad$ Est $\leftarrow$ Est $+ c$
7: **return** Est

Environment. All experiments were carried out on a cluster of nodes consisting of AMD EPYC 7713 CPUs running with 2x64 real cores. All tools were run in a single-threaded mode on a single core with a timeout of 10 hrs, i.e., 36000 seconds. A memory limit was set to 32 GB per core.

Parameters for Oracles and Implementation. In the implementation, we utilized different state-of-the-art tools as counting and sampling oracles, including UniSamp [DM22] as an almost uniform sampling oracle, and the latest version of ApproxMC [YM23] as an approximate counting oracle. SkolemFC was tested with $\varepsilon = 0.8$ and $\delta = 0.4$. That gave the following values to error and tolerance parameters for model counting and sampling oracles. The almost uniform sampling oracle UniSamp is run with $\varepsilon_s = 0.16$. The approximate model counting oracle ApproxMC in line 7 was run with $\varepsilon_c = 4\sqrt{2} - 1$ and $\delta_c = \frac{0.4}{m \cdot s}$, where s comes from the algorithm, based on input (ε, δ) and m is number of output variables in the specification. We carefully select error and tolerance values $\varepsilon_s, \varepsilon_c, \delta_c$ for counting and sampling oracles to ensure the

[1]All benchmarks and experimental data are available at https://doi.org/10.5281/zenodo.10404174

Algorithm	# Instances solved
Baseline	8
SkolemFC	375

Table 6.1: Instances solved (out of 609).

validity of final bounds for SkolemFC while also aiming for optimal performance of the counter based on these choices. The relationship between these values and the validity of bound of SkolemFC is illustrated in the proof of Theorem 6.4.2.

In our experiments, we sought to evaluate the run-time performance and approximation accuracy of SkolemFC. Specifically, the following questions guided our investigation:

RQ1. How does SkolemFC scale in terms of solving instances and the time taken in our benchmark set?

RQ2. What is the precision of the SkolemFC approximation, and does it outperform its theoretical accuracy guarantees in practical scenarios?

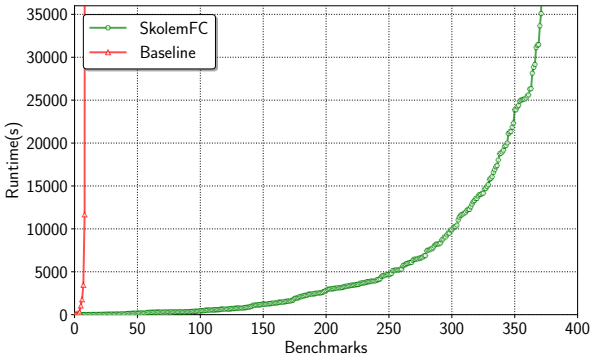
Benchmarks. To evaluate the performance of SkolemFC, we chose two sets of benchmarks.

1. *Efficiency benchmarks.* 609 instances from recent works on Boolean function synthesis [GRM20; ACJS17], which includes different sources: the Prenex-2QBF track of QBF Evaluation 2017 and 2018, disjunctive [ACJS17], arithmetic [TV17] and factorization [ACJS17].
2. *Correctness Benchmarks.* The benchmarks described in the paragraph above are too hard for the baseline algorithm and QCounter to solve. As Section 6.5.1 reveals, the number of instances solved by the baseline is just eight out of the 609 instances. Therefore, to check the correctness of SkolemFC (RQ2), we used a set of 158 benchmarks from SyGuS instances [GRM21]. These benchmarks have very few input variables ($m \leq 8$), and takes seconds for SkolemFC to solve.

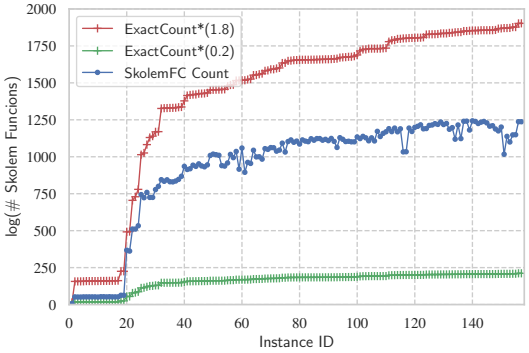
Summary of Results. SkolemFC achieves a huge improvement over QCounter or Baseline by resolving 375 instances in a benchmark set consisting of 609, while Baseline only solved 8, and QCounter solved none. The accuracy of the approximate count is also noteworthy, with an average error of a count by SkolemFC of only 21%.

6.5.1 Performance of SkolemFC

We evaluate the performance of SkolemFC based on two metrics: the number of instances solved and the time taken to solve the instances.



(a) Runtime performance of SkolemFC and Baseline.



(b) SkolemFC's estimate, vs. the theoretical bounds.

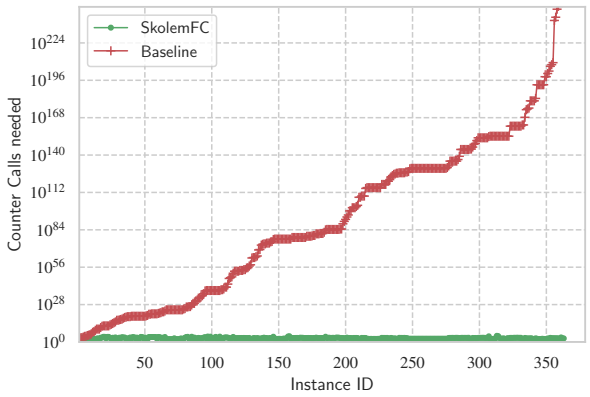
Figure 6.1: Performance and Correctness of SkolemFC

Instances Solved. In Table 6.1, we compare the number of benchmarks that can be solved by Baseline and SkolemFC. First, it is evident that the Baseline only solved 8 out of the 609 benchmarks in the test suite, indicating its lack of scalability for practical use cases. Conversely, SkolemFC solved 375 instances, demonstrating a substantial improvement compared to Baseline. QCounter solves none of the instances from these benchmark sets. It is worth noting that on these benchmarks, there are many false QBFs, which are out of the scope of QCounter.

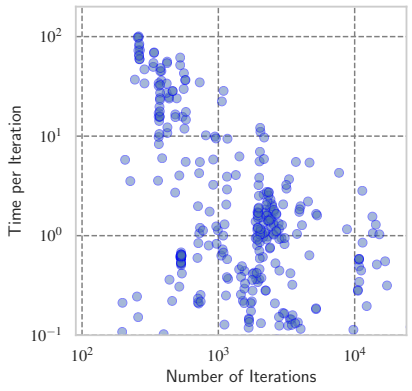
Solving Time Comparison. A performance evaluation of Baseline and SkolemFC is depicted in Figure 6.1a, which is a cactus plot comparing the solving time. The x-axis represents the number of instances, while the y-axis shows the time taken. A point (i, j) in the plot represents that a solver solved j benchmarks out of the 609 benchmarks in the test suite in less than or equal to j seconds. The curves for Baseline and SkolemFC indicate that for a few instances, Baseline was able to give a quick answer, while in the long run, SkolemFC could solve many more instances given any fixed timeout.

Counter Call Comparison. We analyze the algorithms' complexity in terms of counter calls, comparing Baseline and SkolemFC across benchmarks in Figure 6.2a. The x axis represents benchmarks, and the y axis shows required counter calls, sorted by the increasing order of calls needed by Baseline. A red or green point (i, j) signifies that Baseline or SkolemFC, respectively, requires j counting oracle calls for the i^{th} instance. Baseline requires up to a staggering 10^{230} counter calls for some instances, emphasizing the need for a scalable algorithm like SkolemFC, which incurs significantly fewer counter calls.

We analyze the scalability of SkolemFC by examining the correlation between average time per counter call and total counter calls, depicted in Figure 6.2b. The point (i, j) means that if SkolemFC needs i counter calls, the average time per call is j seconds. The figure showcases diverse scenarios: some with fewer iterations and longer durations per call, others with high counts and minimal time per call.



(a) Counter calls needed by SkolemFC and Baseline to solve the benchmarks.



(b) Relation between number of iterations needed by SkolemFC and average time taken in each counter call.

Figure 6.2: Couter calls (iterations) and runtime.

6.5.2 Quality of Approximation

In the experiments, 158 accuracy benchmarks were measured using QCounter, enabling comparison between QCounter and SkolemFC results, shown in Figure 6.1b. The counts' close alignment and error reduction below theoretical guarantees were observed. We quantify the SkolemFC performance with error $e = \frac{|b-s|}{b}$, where b is the count from QCounter and s from SkolemFC. Analysis of all 158 cases found the average e to be 0.005, geometric mean 0.003, and maximum 0.035, contrasting sharply with a theoretical guarantee of 0.8. This signifies SkolemFC substantially outperforms its theoretical bounds. Our findings underline SkolemFC's accuracy and potential as a dependable tool for various applications.

6.6 Summary

This chapter presents the first scalable approximate Skolem function counter, SkolemFC, which has been successfully tested on practical benchmarks and showed impressive performance. Our proposed method employs propositional model counting and sampling techniques to provide theoretical guarantees for its results. The implementation leverages the progress made in the last two decades in the fields of constrained counting and sampling, and the practical results exceeded the theoretical guarantees. Beyond its central question, there are couple of interesting takeaways for the general quantitative SMT problem from this work.

First, quantitative SMT problems need not be limited to counting satisfying assignments. In richer theories, the natural object to count may be more semantic. In the case of uninterpreted functions, counting interpretations provides a way to measure how much freedom remains in the unknown function.

Second, even highly restricted SMT fragments can lead to nontrivial counting problems. The fragment studied in this chapter removes many sources of difficulty, such as multiple UF applications, congruence between repeated calls, and function composition. Nevertheless, the resulting problem still requires reasoning over a space of functions rather than a space of assignments.

Third, reductions to propositional counting remain a powerful tool. Although the original problem is motivated by SMT with uninterpreted functions, bit-vector finiteness allows the problem to be transformed into a Boolean specification. Once this connection is made, techniques from model counting, projected counting, and sampling can be used to reason about the size of the function space.

Finally, this chapter shows that approximate counting can be useful when exact counting is infeasible. Exact counting provides strong guarantees but often does not scale to function-counting problems. Approximate methods, by contrast, can give useful quantitative information without constructing the objects being counted. This lesson is central to the thesis: for many SMT counting problems, the goal is not only to decide satisfiability, but to obtain scalable quantitative estimates of the underlying solution space.

These findings open several directions for further investigation. One such area of potential extension is the application of the algorithm to other types of functions, such as counting uninterpreted functions in SMT with a more general syntax. This extension would enable the algorithm to handle a broader range of applications and provide even more accurate results. In summary, this research contributes significantly to the field of Skolem function counting and provides a foundation for further studies.

Part III

In the wild and Concluding

Chapter 7

Case Study: Probabilistic Model Checking via Model Counting

The problem of finite-horizon reachability in discrete-time Markov chains (DTMCs) is to determine the probability of reaching a designated target state within h steps. Finite-horizon reachability is a fundamental problem in probabilistic model checking with a wide range of applications. Despite sustained progress, the scalability of probabilistic and statistical model checkers is severely impacted by state-space explosion, particularly for systems with parallel processes, thereby preventing their adoption in application domains requiring verification of large-scale concurrent systems. In this chapter, we reduce that problem to a bitvector model counting problem and observe a huge speedup.

The thesis opens with the motivation that quantifying uncertainty is a serious problem in the modern world, where systems operate under uncertainty, and that #SMT can be a useful tool for solving it. Probabilistic model checking is a domain in which the systems under consideration work stochastically and we need to bound the probability of errors. In this chapter, we sought to determine whether #SMT can solve some of the scalability issues in probabilistic model checking.

Discrete-time Markov chains (DTMCs) are the standard model to describe system behavior subject to probabilistic uncertainty. A key verification question for Markov chains is finite-horizon reachability, e.g., *what is the probability that my battery depletes within ten time units?* Broadly, the state-of-the-art verification approaches for Markov chains are either based on (1) constructing and solving an underlying system of linear equations precisely, as implemented by mature *probabilistic model checking* [BK08; BAFK18] tools, including Storm [Hen+22] and Prism [KNP11] or (2) based on sampling paths through the Markov

Figure 7.1: The factory example, taken from [Hol+21].

chain and statistically inferring the probability from those samples, as implemented in *statistical model checking* [HLMP04; YS02; Bud+25] approaches, including those implemented in Modest [HH14; BDHS18] and Uppaal [Dav+15]. In this work, we contribute an alternative approach to statistical model checking that intuitively relies on iteratively solving probabilistic model checking queries for the question *what is the probability that my battery depletes at exactly i steps?*, for different values of i.

To ground the discussion, we consider a simple factory benchmark from [Hol+21] (Figure 7.1) and that exposes the core scalability barriers. Suppose there are n factories. On each day, the workforce at factory i is in one of two states: not striking (s_i) or striking (t_i). The daily evolution of factory i is as follows: when in s_i, the factory initiates a strike with probability p_i, and when in t_i, it ends the strike with probability q_i. All factories update synchronously and independently in parallel. The bounded horizon reachability query, $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$ asks: what is the probability that the joint system reaches the *allstrike* configuration $(t_1, \ldots, t_n)$; i.e., all factories simultaneously striking: at least once within h days? Although the model is structurally simple, existing tools typically do not scale beyond a few dozen factories for this query.

In probabilistic model checking, the key technique consists of iteratively updating the transition probability to reach a target within i steps. Concretely, let $p_i(s)$ denote the probability of reaching the target from state s within i steps; the update p_{i+1} is obtained by applying the transition operator, typically realized as a sparse matrix–vector multiplication with the system's transition matrix. However, when the state space grows exponentially, the induced transition matrix becomes too large even to construct: In the factory model, the global state space grows as 2^n, and the transition matrix has 4^n entries in the worst case, so explicit-state model checkers typically cease to scale beyond roughly 15 factories. Standard symbolic approaches based on decision diagrams fail as the number of different probabilities also explodes [DDM16].

A complementary response to state-space explosion is to turn to *statistical* model checking [HLMP04; YS02]. The key idea is to replace exhaustive numerical computation with randomized simulation, returning estimates equipped with rigorous statistical guarantees, often expressed as *additive* error bounds on reachability probabilities. Additive guarantees, however, become problematic in the rare-event regime. Budde et al. [BDHS18] designed modes as part of Modest toolchain [HH14], which can return probabilities with *multiplicative* guarantees. In our experiments, we observed that it fails to return a probability below 10^{-7}. For the factory benchmark, the *allstrike* event probability decays rapidly with n, pushing the reachability query into the rare-event regime; as a result, statistical model checkers struggle to scale.

Taken together, these observations indicate a gap: we seek a method that avoids explicit state-space construction *and* remains effective for rare events under multiplicative accuracy requirements. This work therefore asks: *Can we compute bounded-horizon reachability probabilities with rigorous multiplicative guarantees in regimes where state-of-the-art approaches fail due to state-space explosion or rare-event probabilities?*

In this work, we answer this question affirmatively. In our tool, MC3[1], we reduce probabilistic model checking to a specialized model counting problem and develop an efficient approximate model counter for this problem. Empirically, MC3 shows substantial scalability gains, solving instances with up to 32× more parallel processes than existing methods can handle.

The primary contribution of this work is to address this scalability bottleneck: we present a novel reduction from probabilistic model checking to weighted model counting, together with an approximate weighted model counter tailored to handle these instances efficiently. Our approach provides (ε, δ)-style multiplicative guarantees: i.e., the computed probability is within a $(1+\varepsilon)$-factor of the true reachability probability with confidence at least $1-\delta$. The core algorithmic innovation is a covering-based model counting technique that leverages a disjunctive decomposition naturally exposed by our encoding. In an extensive experimental evaluation across standard probabilistic model checking benchmarks, the resulting tool solves instances with up to 40× more parallel processes than state-of-the-art probabilistic model checkers and up to 30× more than statistical model checkers. Furthermore, our approach successfully handles rare-event instances with probabilities as low as 10^{-303}, whereas existing statistical model checkers are limited to 10^{-7}.

Holtzen et al. [Hol+21] reduce probabilistic model checking to *weighted model counting* (WMC), as realized in their tool Rubicon. This perspective can substantially outperform explicit state-space methods on models dominated by state explosion; however, pushing to larger systems or longer horizons remains challenging because the resulting WMC instances become prohibitively hard.

In this work, we revisit and refine the model-counting view by distinguishing reachability problems. For a Markov chain $\mathcal{M}$, the standard *bounded-horizon* query asks for the probability of reaching a target *within* horizon h, denoted $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$. A closely related query asks for the probability of reaching the target *exactly at* step h, denoted $\Pr_{\mathcal{M}}[\Diamond^{=h}]$. Using the encodings from Section 7.2.2, we find the performance of model counters vary dramatically between these two queries for some of benchmark families. For instance, on factory instances, a direct WMC encoding of $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$ becomes infeasible beyond 12 factories, whereas the corresponding $\Pr_{\mathcal{M}}[\Diamond^{=h}]$ encoding scales to 990 factories. Interestingly, the aforementioned behavior is not present in Rubicon, which does not solve a direct WMC reduction but leverages a probabilistic-programming-based pipeline.

Unfortunately, the *within* query does not reduce immediately to a set of *exactly at* query, despite the relation $\Pr_{\mathcal{M}}[\Diamond^{\leq h}] = \Pr[\bigcup_{i\in[h]} \Diamond^{=i}]$. This relation, however, places the problem in direct correspondence with

[1] MC3 abbreviates Markov Chain Model Checking to Model Counting

the classical *union-of-sets* estimation task: given potentially overlapping sets, estimate the size of their union. Karp and Luby [KL83] showed that this task admits an FPRAS under the ability to sample from each set and to compute individual set sizes. More recently, significantly more efficient estimators have been developed [CVM22] and applied successfully in domains including approximate counting for DNF formulas [Soo+24] and volume computation in SMT settings [SSM25]. The key obstacle in our setting is that classical union estimators assume *unweighted* elements, whereas WMC assigns an *individual weight* to each model.

Motivated by this gap, we define a *disjunctive cover form* (DCF) formula, in which a formula F is represented as a collection $\widehat{F} = \{G_1, \ldots, G_m\}$ over a shared variable set whose disjunction covers F. We then design *covering-based counting* – a new paradigm for approximate weighted model counting applicable to the *DCF formulas*. This algorithm provides an approximation with (ε, δ) multiplicative guarantees via calls to $\mathrm{WMC}(G_i)$. Crucially, this interface allows us to exploit the empirical hardness separation between the *within-horizon* query and the *exactly at* queries: we use the encodings of $\Pr_{\mathcal{M}}[\Diamond^{=i}]$ as the cover components G_i. Integrated into our reduction-based model checker MC3, this yields a system that solves instances up to 32× larger than those tractable by current state-of-the-art probabilistic model checkers. For instance, on factory instances, the state of the art probaiblistic model checker, STORM, could handle up to 14 factories and the state of the art approximate checker could handle up to 30 factories, while MC3 can handle up to 770 factories.

The magnitude of the observed speedups also motivates re-examining a standard modeling assumption in probabilistic model checking, which adopts a *absorbing* reachability convention, where $\Diamond^{=h} \implies \Diamond^{=h+1}$ Under our encoding, this absorbing convention limits the extent to which our covering-based counting machinery can be exploited. In contrast, a *non-absorbing* semantics, in which the system may leave the target after reaching it, interacts more naturally with our approach and provides substantially more structure. This raises a broader question of whether similar benefits can be obtained in other verification and analysis tasks by revisiting absorbing assumptions and favoring non-absorbing formulations when appropriate.

7.1 Preliminaries and Problem Statement

Discrete-Time Markov Chains. Let $\Delta(X)$ be the set of distributions over X. A Markov chain (DTMC) is a tuple $\mathcal{M} = \langle S, \iota, P, T \rangle$ with S a finite set of *states*, $\iota \in S$ the *initial state*, $P \colon S \to \Delta(S)$ the *transition (probability) function*, and $T \subseteq S$ a set of *target states*. A bounded path $s_0 s_1 s_n \in S^{n+1}$ is a sequence of states such that $P(s_i, s_{i+1}) > 0$ for each $i < n$. We call n the length of the path. With $\Diamond^{\leq n}$ we denote the set of paths of lengths n such that (1) s_0 is the initial state and (2) there is an i such that $s_i \in T$. With $\Diamond^{=n}$ we denote the set of paths of lengths n such that (1) s_0 is the initial state and (2) there is an i such that $s_n \in T$.

The probability $\Pr(\tau)$ of a bounded path τ is the product of the individual transition probabilities. The probability $\Pr_{\mathcal{M}}(Z)$ of a set of paths Z is the sum of the individual probabilities (as paths are exclusive events). For a formal treatment, we refer to [BK08, Ch. 10].

Problem Statement. This work focuses on the following problem.

> Given an Markov chain $\mathcal{M}$, a horizon h, and ε, δ, compute an (ε, δ)-approximation of $\Pr_{\mathcal{M}}(\Diamond^{\leq h})$.

By an (ε, δ)-approximation we mean an estimator $\widehat{p}$ of $p = \Pr_{\mathcal{M}}[\Diamond^{\leq h}]$ such that:

$$\Pr\left[(1-\varepsilon)p \leq \widehat{p} \leq (1+\varepsilon)p\right] \geq 1-\delta$$

To compute this number, we will solve the problem to compute $\Pr(\Diamond^{=h})$ exactly as a crucial subroutine. As we wrote in the introduction, we will focus on solving these queries using weighted model counting.

Weighted Counting and Sampling. We recapitulate the specific notion of weighted model counting and weighted sampling which will be used heavily in this chapter.

Let F be a quantifier-free SMT bit-vector formula over a finite set of variables $\mathsf{Vars}(F) = B \uplus V$, where $B = \{b_1, \ldots, b_n\}$ are Boolean variables and $V = \{v_1, \ldots, v_m\}$ are bit-vector variables (of fixed widths). A literal is a Boolean variable or its negation, $\mathsf{Lits}(B) = \{b_1, \neg b_1, \ldots, b_n, \neg b_n\}$. An assignment is a function σ that maps each $b \in B$ to $\{0,1\}$ and each $v \in V$ to a bit-vector value of the appropriate width. ; we write $\sigma \models F$ if σ satisfies F. We denote $\mathsf{models}(F)$ the set of satisfying assignments of F. A *weight function* is a map $w : \mathsf{Lits}(B) \to [0,1]$ that assigns weights only to Boolean literals. The weight of an assignment σ is

$$w(\sigma) \triangleq \prod_{b \in B} \begin{cases} w(b) & \text{if } \sigma(b) = 1, \\ w(\neg b) & \text{if } \sigma(b) = 0. \end{cases}$$

Bit-vector variables contribute no factors.

We want to define a weighted model counts, projected on Boolean variables only.

The *weighted model count* of F (w.r.t. w) is $\mathsf{WMC}(F, w) \triangleq \sum_{\sigma \models F} w(\sigma)$. A *weighted sampler* for (F, w) is a randomized procedure that returns a satisfying assignment $\sigma \models F$ such that $\Pr[\sigma] = w(\sigma)/\mathsf{WMC}(F, w)$: assignments are drawn proportional to the weights.

7.2 From Markov Chains to Model Counting

The simple idea of weighted model counting for DTMCs is that we ensure that the set of models of a SAT formula corresponds to the set of paths reaching a target.This idea basically mimics bounded model checking. This encoding must be concise and maintain the structure of the problem. Early experiments confirmed that building an encoding from an explicit representation of a Markov chain does not lead

```
x1, x2, x3 : [0..1] init 1;
module {
 [step] x1 =x3 -> 1/3 : (x1'=0) + 2/3 : (x1'=1);
 [step] x1!=x3 -> (x1'=x3);
}
module {
 [step] x2 =x1 -> 1/3 : (x2'=0) + 2/3 : (x2'=1);
 [step] x2!=x1 -> (x2'=x1);
}
module {
 [step] x3 =x2 -> 1/3 : (x3'=0) + 2/3 : (x3'=1);
 [step] x3!=x2 -> (x3'=x2);
}
```

Figure 7.2: SPF example (here: Herman's protocol).

to a tractable approach. We therefore formalize a (high-level) symbolic description of Markov chains that supports synchronous processes with handshakes. We provide an encoding of Markov chains into a weighted model counting problem that leverages the parallel (factored) nature of the input.

7.2.1 Symbolic Markov Chains with Parallel Composition

While our experiments use (a large and common fragment of) PRISM [KNP11] or JANI [Bud+17], which are rooted in reactive modules [AH99], we introduce *SPF*, a simple PRISM fragment, which is a slightly simplified modelling language for exposition, which still features multiple synchronized modules that each probabilistically update variables. In fact, the simplifications are five assumptions regarding well-formedness, that we syntactically restrict ourselves to constant probabilities in the language, and that we omit syntactic sugar.

Example 1. Consider Fig. 7.2 for an example. SPF contains a set of bounded variables (`x1`, `x2`, and `x3`) a set of implicitly defined synchronization primitives (only `step`), and a set of modules. Each module is a list of commands:

```
[synclabel] guard -> prob:  (updates) + prob:  (updates)
```

In the example, all modules run in lockstep fashion, as there is a single synchronization primitive. In a particular state, based on guards, in each module there is one matching command. This set of commands is executed, yielding a distribution over successor states.

More generally, SPF describes symbolic Markov chains. We use $\mathsf{BoolExpr}(V)$ and $\mathsf{IntExpr}(V)$ for expressions of Boolean and (bounded) integer type, respectively, using some set of variables V. A symbolic Markov chain is a tuple $\langle V, A, \mathsf{init}, M_1, \dots M_n, \mathsf{T}\rangle$ where V is a finite set of variables with integer values $0, \dots K$, A is a finite set of synchronisation primitives, $\mathsf{init}\colon V \to \{0, \dots, K\}$ assigns to every variable an inititial value,

and T is a Boolean expression over the V that encodes the target states, and the modules M_i are sets of commands: $M_i \subseteq A \times \mathsf{BoolExpr}(V) \times \Delta(V \nrightarrow \mathsf{IntExpr}(V))$. Every command c is a triple: a *synchronization primitive* a_c, a *guard* g_c, and a distribution Δ_c over updates, where updates are a function describing how variables are updated, using the current value of the variables. The synchronization label ensures that commands in different modules can only be executed together (when all guards are satisfied).

Specifically, the symbolic Markov chain describes an underlying DTMC: States are assignments $S = V \to \{0, \ldots, K\}$ with initial state $\iota = \mathsf{init}$ and target states $T = \{s \in S \mid s \models \mathsf{T}\}$. Intuitively, the transitions are defined as follows. For each state, the successor distribution is governed by a unique primitive a and a unique sequence of commands C_s, one per module, and each labelled with primitive a. The next state is then obtained by executing all commands concurrently: This distribution over the successor states is obtained by the combinatorial combination of all individual updates. We provide details and the assumptions (A2-A5) in **??** and exemplify the transitions below.

Example 2. A state of the SPF in Fig. 7.2 is $(1, 1, 1)$ for variables $(\mathtt{x1}, \mathtt{x2}, \mathtt{x3})$. In this case, the first, third, and fifth commands are enabled. Jointly executing them describes a distribution over eight(!) successor states. One of them is $(0, 0, 1)$, which is reached with probability $2/27$.

7.2.2 Encoding

We present a concise encoding of SPF into an SMT formula that describes the set of paths that reach a target within or at h steps. We annotate the variables in the encoding with weights to make the problem of computing the probability amenable to model counting. In the following, let $\langle V, A, \mathsf{init}, M_1, \ldots M_n, \mathsf{T}\rangle$ denote a symbolic Markov chain. We use X_i as the *coin-flip variables* for process M_i, which contains flip variables for every command in the process[2]:

$$X_i = \{x^i_{c,1}, \ldots, x^i_{c,|\Delta_c|} \mid \langle a_c, g_c, \Delta_c\rangle \in M_i\}.$$

The set of global coin flip variables is the disjoint union $X = \bigcup X_i$. We also use auxiliary indicator variables for actions $Y = \{y_a, y^i_a \mid a \in A, 1 \le i \le n\}$.

Step encoding. We use variables V along with their primed copies $V' = \{v' \mid v \in V\}$. We introduce an SMT formula $\mathsf{step}(V, V', X)$ that encodes the possibility to move from a state s — described by the assignment to V — to a state s' — described by V' and depending on the assignment (outcome) of the coin flips X:

$$\mathsf{step}(V, V', X, Y) = \mathsf{valid}(X) \wedge \mathsf{enabled}(V, Y) \wedge \mathsf{execute}(V, V', X, Y)$$

[2]Note that the set of coin flip variables can be reduced by sharing these variables along multiple mutually exclusive commands. We observed that such optimizations are easily spotted by a preprocessor for weighted model counting.

The assignment to the coin flip variables must assign one outcome for every command, which is encoded in $\mathsf{valid}(X)$ as the conjunction of the following pseudoboolean equalities:

$$\sum\nolimits_j x^i_{c,j} = 1 \qquad \text{for each } 1 \le i \le n, c \in M_i.$$

Furthermore, a synchronization primitive is enabled by a module if a guard is enabled. A synchronization primitive is globally enabled if it is enabled by every process, i.e., $\mathsf{enabled}(V, Y)$ is the conjunction of

$$\bigwedge_i \left(\Big(\bigvee_{c \in M_i, a_c = a} g_c \Big) \leftrightarrow y^i_a \right) \wedge \left(\Big(\bigwedge_i y^i_a \Big) \leftrightarrow y_a \right) \qquad \text{for each } a \in A.$$

Exactly one primitive is enabled by the Assumptions A2 and A3. The disjunction is not empty by A1. We define the main part of the formula, which encodes whether s', defined over V' is a successor of s, defined over V. In particular, $\mathsf{execute}(V, V', X, Y)$ is the conjunction of

$$y_{a_c} \wedge g_c \wedge x^i_{c,j} \rightarrow \bigwedge_{v \in V} v' = (\Delta_c)_j(v) \quad \text{for each } 1 \le i \le n, c \in M_i, 0 \le j < |\Delta_c|,$$

where $(\Delta_c)_j$ denotes the j'th update, a function from variables to expressions, and v' is the primed variable matching v. Assumption A4 prevents contradictions (data races), and A5 ensures that every variable is assigned in the updates.

Full encoding. The full encoding is obtained as standard in bounded model checking: We create h copies of the state variables V and $h-1$ copies of the auxiliary X and Y variables. We then unroll the one-step encoding, fix the initial state, and define which paths are targets:

$$\Phi(\mathcal{M}, h) = \bigwedge_{0 \le i < h} \mathsf{step}(V[i], V[i+1], X[i], Y[i]) \wedge \mathsf{init}(V[0]) \wedge \bigvee_{*} \mathsf{target}(V[i]) \tag{7.1}$$

where * denotes either $1 \le i \le h$ or $i = h$ depending on whether we want to compute $\Pr(\Diamond^{\le h})$ or $\Pr(\Diamond^{=h})$.

Weights. We encode the coin-flip probabilities of the X_i variables as literal weights: the positive literal receives $w(X_i) = \Pr(X_i)$ and the negative literal receives $w(\neg X_i) = 1 - \Pr(X_i)$. In the encoding, all variables other than the coin-flip variables are deterministically determined once the X_i are fixed; accordingly, we assign weight 1 to both their positive and negative literals.

Size of the encoding. The size of the encoding is linear in the size of the input format and logarithmic in the maximal value of the largest value K. In particular, the outdegree of the underlying Markov chain can be exponential in the number of processes, but this encoding does not suffer from this blowup.

Correctness. The reachability probability in a Markov chain is the sum over the probabilities of every path that starts in the initial state and ends in the target. Thus, we argue that for our encoding $\Phi(\mathcal{M}, h)$, the weighted model count of assignments that agree with a path coincides with the probability of that path. More precisely, let $\psi_s(V)$ encode state s: $\psi_s(V) = \bigwedge_{v \in V} v = s(v)$. Then, for any path τ, $\psi_\tau(V[0], \ldots, V[h])$ encodes following this path using $\psi_\tau = \bigwedge_i \psi_{\tau_i}(V[i])$.

Lemma 7.2.1. For any τ in $\Diamond^{\leq h}$:

$$\mathrm{WMC}(\Phi(\mathcal{M}, h) \wedge \psi_\tau) = \Pr(\tau).$$

Note that in $\Phi(\mathcal{M}, h) \wedge \psi_\tau$, all assignments to satisfying assignments in V and Y are predetermined. Furthermore, the assignment to $X[i]$ must ensure that we transition from τ_i to τ_{i+1}. This assignment has a weight equal to the product of the parallely executed updates, as intended. Furthermore, the coin flips must agree on every step: the weighted model count is obtained by multiplying the weight for the individual steps, just like the definition of path probabilities. From the lemma above and the definition of reachability probabilities, we then obtain:

Lemma 7.2.2. For any Markov chain $\mathcal{M}$ and horizon h:

$$\mathrm{WMC}(\Phi(\mathcal{M}, h)) = \sum_{\tau \in \Diamond^{\leq h}} \mathrm{WMC}(\Phi(\mathcal{M}, h) \wedge \psi_\tau) = \sum_{\tau \in \Diamond^{\leq h}} \Pr(\tau) = \Pr_{\mathcal{M}}(\Diamond^{\leq h}).$$

7.3 Covering-based Weighted Model Count

In MC3's workflow, once the weighted model counting (WMC) instance is constructed, we hand it to a specialized weighted model counter. This section presents the model counter and its underlying algorithm.

7.3.1 Formula Representation

Before presenting our counting algorithm, we introduce a formula representation that makes the algorithm applicable and exposes the structure we will exploit.

Disjunctive Cover Form (DCF). A bit-vector formula F is said to be in *Disjunctive Cover Form (DCF)* if it is represented by a family of subformulas $\widehat{F} = \{G_1, \ldots, G_m\}$ such that $\mathsf{models}(F) = \bigcup_{i=1}^{m} \mathsf{models}(G_i)$. Equivalently, $F \equiv \bigvee_{i=1}^{m} G_i$.

Core–Selector DCF is a representation of DCF where the covering subformulas are given implicitly. Here, F admits a syntactic decomposition $F = B \wedge U$, where B is a shared *core* and U is a syntactically distinguished clause $U := \bigvee_{i=1}^{m} u_i$ over *selector* literals. The respective *cover* is given by $G_i := B \wedge u_i$ for each $u_i \in U$.

Returning to Equation (7.1), the encoding is naturally in core–selector DCF:

$$\underbrace{\Big(\bigwedge_{0 \le i < h} \mathsf{step}(V[i], V[i+1], X[i], Y[i]) \wedge \mathsf{init}(V[0]) \Big)}_{\text{core}} \wedge \underbrace{\Big(\bigvee_{1 \le i \le h} \mathsf{target}(V[i]) \Big)}_{\text{selector}}$$

The clause $\bigvee \mathsf{target}(V[i])$ serves as the selector disjunction, while the remainder of the encoding constitutes the core B. In the following part, we develop an approximate weighted model counting algorithm specialized to formulas in DCF.

7.3.2 Algorithm

Given a formula F in disjunctive cover form, and a set of weights w for the literals, our algorithm, CoverMC, returns an approximation of the weighted model count of F with (ε, δ) guarantees. For each subformula G_i, it invokes a weighted model counter to compute $\mathrm{WMC}(G_i)$, and then uses a weighted sampler to draw assignment s with probability proportional to their weights, $w(s)$. Using these samples, CoverMC estimates the weighted model count of the formula F.

The algorithm's key idea is to maintain a set $\mathcal{X}$ of sampled satisfying assignments of the input formula F, where each assignment s is included with probability $(p \cdot w(s)/g)$. Therefore, the expected size of the set $\mathcal{X}$ is $(p/g) \cdot \sum_{s \models F} w(s)$. The set $\mathcal{X}$ is populated incrementally by iterating over the subformulas G_i, while maintaining the invariant that $\mathrm{WMC}(F_i, w) \approx g \cdot |\mathcal{X}|/p$, where $F_i := G_1 \vee \cdots \vee G_i$. A constant Thresh denotes the target size of $\mathcal{X}$; throughout the execution of the algorithm, $\mathcal{X}$ is halved, and p is updated accordingly to ensure the Thresh bound remains valid. Below we provide a detailed description of the CoverMC algorithm.

Given the formula F in the *core-selector* form, the first step of the algorithm is to get the covering subformulas $\widehat{F} = \{G_1, \ldots, G_m\}$. This step is done at line 1 of Algorithm 9. After that, CoverMC sets the key constants and initializes the core components of the algorithm at lines 2 and 3. It then enters the main loop, which iterates over the subformulas. In the main loop at lines 4-16, the algorithm processes each subformula G_i. It begins

Algorithm 9 CoverMC $(F, w, \varepsilon, \delta)$

1: $\widehat{F} \leftarrow \mathsf{GetCover}(F), m \leftarrow |\widehat{F}|$
2: $\mathsf{Thresh} \leftarrow \max\left(24 \cdot \frac{\ln(24/\delta)}{(1-\varepsilon')\varepsilon'^2}, 6(\ln\frac{6}{\delta} + \ln m)\right)$
3: $g \leftarrow \frac{\mathsf{Thresh}}{\mathsf{WMC}(G_1)}$; $\mathcal{X} \leftarrow \emptyset; p \leftarrow g$
4: **for** $i = 1$ to m **do**
5: $\quad t \leftarrow \mathsf{WMC}(G_i, w)$
6: $\quad$ **for** $s \in \mathcal{X}$ **do**
7: $\quad\quad$ **if** $s \in G_i$ **then** remove s from $\mathcal{X}$
8: $\quad$ **while** $p \geq \frac{\mathsf{Thresh}}{t \cdot g}$ **do**
9: $\quad\quad$ Remove every element of $\mathcal{X}$ with prob. $1/2$
10: $\quad\quad p \leftarrow p/2$
11: $\quad N_i \leftarrow \mathsf{Poisson}(t \cdot p)$
12: $\quad$ **while** $N_i + |\mathcal{X}| > \mathsf{Thresh}$ **do**
13: $\quad\quad$ Remove every element of $\mathcal{X}$ with prob. $1/2$
14: $\quad\quad N_i \leftarrow \mathsf{Poisson}(t \cdot p/2)$ and $p \leftarrow p/2$
15: $\quad S \leftarrow \mathsf{GenerateSamples}(G_i, w, N_i)$
16: $\quad \mathcal{X} \leftarrow \mathcal{X} \cup S$
17: Output $|\mathcal{X}|/p$

by computing the weighted model count t of G_i (line 5). Next, in lines 6-7, CoverMC removes from $\mathcal{X}$ all assignments that satisfy G_i. After this filtering step, $\mathcal{X}$ contains samples from $\mathsf{models}(F_{i-1}) \setminus \mathsf{models}(G_i)$, so that when samples from $\mathsf{models}(G_i)$ are subsequently added, $\mathcal{X}$ becomes uniformly distributed over $\mathsf{models}(F_i)$.

In lines 8-10, CoverMC reduces p and randomly discards half of the elements of $\mathcal{X}$ to free capacity for the new samples contributed by G_i. At line 11, it determines how many new samples to draw from $\mathsf{models}(G_i)$ by sampling from a Poisson distribution with mean $t \cdot p$. Scaling by t ensures that each assignment s is included with probability $w(s)/w(G_i)$.

The loop at lines 12-14 applies the same halving-and-update procedure again, guaranteeing that $\mathcal{X}$ remains within the target size bound Thresh while reserving space for the imminent additions. Finally, at lines 15 and 16, CoverMC samples assignments from G_i and appends them to $\mathcal{X}$. After all subformulas have been processed, the algorithm outputs $|\mathcal{X}|/p$ at line 17, which serves as the estimate of $\mathsf{WMC}(F, w)$.

7.3.3 Analysis

Theorem 7.3.1. Given a formula F, and parameters $\varepsilon, \delta \in (0, 1]$, MC3 returns an estimate Est of $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$ such that,

$$\Pr\left[(1-\varepsilon) \cdot \Pr_{\mathcal{M}}[\Diamond^{\leq h}] \leq \mathsf{Est} \leq (1+\varepsilon) \cdot \Pr_{\mathcal{M}}[\Diamond^{\leq h}]\right] \geq (1-\delta).$$

Theorem 7.3.1. MC3 proceeds in two steps. First, given the input model and horizon, it constructs a weighted model-counting instance (F, w). By the encoding correctness established in Section 7.2, this construction preserves the target quantity, i.e., $\Pr_{\mathcal{M}}[\Diamond^{\leq h}] = \mathsf{WMC}(F, w)$.

Second, MC3 invokes CoverMC on (F, w). By Lemma 7.3.2 below, CoverMC returns an estimate Est satisfying the standard (ε, δ) multiplicative approximation guarantee for $\mathsf{WMC}(F, w)$. Combining the identity above with the guarantee from Lemma 7.3.2 yields $\Pr\big[(1-\varepsilon)\Pr_{\mathcal{M}}[\Diamond^{\leq h}] \leq \mathsf{Est} \leq (1+\varepsilon)\Pr_{\mathcal{M}}[\Diamond^{\leq h}]\big] \geq 1-\delta$, as claimed. Since MC3 outputs exactly this estimate as its probability estimate, the same (ε, δ) guarantee applies to MC3's output. □

Lemma 7.3.2. Given a formula F , weight function w and parameters $\varepsilon, \delta \in (0, 1]$, CoverMC returns an estimate Est of $\mathsf{WMC}(F, w)$ such that,

$$\Pr\left[(1-\varepsilon)\cdot \mathsf{WMC}(F,w) \leq \mathsf{Est} \leq (1+\varepsilon)\cdot \mathsf{WMC}(F,w)\right] \geq (1-\delta).$$

Remark 5 (Relation to [Soo+24])**.** The following proof for weighted CNF counting closely parallels the argument of Soos et al. [Soo+24] for unweighted DNF counting. We reproduce their proof structure with minimal modification and introduce only the changes required for the weighted CNF setting. This presentation is intended to make the correspondence transparent so that an informed reader can readily compare the two proofs and follow the adaptations.

Lemma 7.3.2. Let Z_i be the set of all satisfying assignments of $F_i := G_1 \vee \cdots \vee G_i$. The weight function $w(z)$ denotes the weight of assignment z, $w(Z) = \sum_{z \in Z} w(z)$, for a set Z. To begin with, we will inductively define the array $R_i^{(j)}$ of distributions that may arise over the sets Z_i during the execution of the algorithm for $i \in \{1, \ldots, m\}$, and j being a non-negative integer. The set $R_0^{(0)}$ is defined to be $\emptyset$. Then we consider two sampling moves:

Decrease p move: To obtain $R_i^{(j+1)}$ from $R_i^{(j)}$, we sample each element of $R_i^{(j)}$ independently with probability $\frac{1}{2}$. This corresponds to line 9 and line 13 in Algorithm 9.

Next set move: To obtain $R_{i+1}^{(j)}$ from $R_i^{(j)}$, the following steps are performed:

(1) **Remove Satisfying Assignments:** Remove all elements from $X_i^{(j)}$ that are satisfying assignments of the formula G_{i+1}. This corresponds to line 7 of Algorithm 9.

(2) **Sample New Assignments:** Let N_{i+1} is sampled from the distribution $\mathrm{Poisson}\left(\frac{g \cdot w(G_{i+1})}{2^j}\right)$. Draw N_{i+1} satisfying assignments $\{s_1, s_2, \ldots s_{N_{i+1}}\}$ of G_{i+1} with replacement, according to the weights of the assignments, i.e., $\Pr[s_k = s] = \frac{w(s)}{w(G_{i+1})}$. This corresponds to line 15 of Algorithm 9.

(3) **Update the Set:** Add the newly sampled assignments to $R_i^{(j)}$, resulting in the updated set $R_{i+1}^{(j)}$. This corresponds to line 16 of Algorithm 9.

Let $k_i^{(j)}$ be a function such that it maps each assignment s of G_i to the number of times it appears in the set $R_i^{(j)}$. By Claim 2 of [Soo+24], for every satisfying assignment $s \in G_i$, $k_i^{(j)}(s)$ is independently drawn from the distribution $\text{Poisson}\left(\frac{g \cdot w(s)}{2^j}\right)$.

We make the following lemma.

Lemma 7.3.3. For all i, j, the random multiset $R_i^{(j)}$ contains samples from set Z_i and, for each satisfying assignment $s \in Z_i$, $R_i^{(j)}$ contains $k_i^{(j)}(s)$ many copies of s where $k_i^{(j)}(s)$ is drawn independently from $\text{Poisson}\left(\frac{g \cdot w(s)}{2^j}\right)$.

Now we shall construct a sequence of events to bound the probability that the algorithm outputs a number that is not within the bound $[\ w(S_m)(1-\varepsilon),\ w(S_m)(1+\varepsilon)\ ]$. For $1 \le i \le m$, and $j \ge 0$, define the event $E_i^{(j)}$ to be the event that after processing the first i subformulas, the value of p is $\frac{g}{2^j}$. Formally, $E_i^{(j)}$ is defined as:

$$E_i^{(j)} := \{\text{ The value of } p \text{ is } \frac{g}{2^j} \text{ after processing the first } i \text{ subformulas. }\}$$

These events can depend arbitrarily on the random sets $R_i^{(j)}$'s sampled during the execution of the algorithm. These $R_i^{(j)}$'s correspond to an arbitrary sequence of the aforementioned moves over the array of distributions $R_i^{(j)}$ depending on the decisions of Algorithm 9 in the loops of 8 and 12.

Assuming $j = j$ at the end, the algorithm outputs $|R_m^{(j)}| \cdot \frac{2^j}{g}$. The algorithm fails if:

$$|R_m^{(j)}| \cdot \frac{2^j}{g} \notin [\ w(S_m)(1-\varepsilon),\ w(S_m)(1+\varepsilon)\ ]$$

To analyze this, we define another sequence of events $A_i^{(j)}$, where after processing the first i subformulas, the following condition holds:

$$|R_i^{(j)}| \notin \left[\ w(Z_i)\, 2^{-j} g(1-\varepsilon),\ w(Z_i)\, 2^{-j} g(1+\varepsilon)\ \right]$$

Now, the event corresponding to the failure of the algorithm occurs when the algorithm fails to produce a value within the desired bound, that is, after consuming m subformulas, the value of p is $\frac{g}{2^j}$ and $|R_m^{(j)}|\, 2^j g \notin$

$[\ w(S_m)(1-\varepsilon),\ w(S_m)(1+\varepsilon)\ ]$ for some $j \geq 0$. This event is given by $\bigcup_{j=0}^{\infty}\left(E_m^{(j)} \wedge A_m^{(j)}\right)$. Since $E_m^{(j)} \wedge A_m^{(j)} \subseteq E_m^{(j)}$ and $E_m^{(j)} \wedge A_m^{(j)} \subseteq A_m^{(j)}$, we derive the following bound for any j^*:

$$\begin{aligned}
\Pr\left[\bigcup_{j=0}^{\infty}\left(E_m^{(j)} \wedge A_m^{(j)}\right)\right] &= \Pr\left[\left(\bigcup_{j\geq j^*}\left(E_m^{(j)} \wedge A_m^{(j)}\right)\right) \cup \left(\bigcup_{j=0}^{j^*-1}\left(E_m^{(j)} \wedge A_m^{(j)}\right)\right)\right] \\
&\leq \Pr\left[\left(\bigcup_{j\geq j^*} E_m^{(j)}\right) \cup \left(\bigcup_{j=0}^{j^*-1} A_m^{(j)}\right)\right] \\
&\leq \Pr\left[\bigcup_{j\geq j^*} E_m^{(j)}\right] + \sum_{j=0}^{j^*-1}\Pr\left[A_m^{(j)}\right] \qquad \text{(union bound).}
\end{aligned}$$

Now let j^* be the smallest j such that $\frac{g}{2^j} < \frac{\mathsf{Thresh}}{4w(S_m)}$. The value of g ensures that $j^* > 2$. Next, we bound $\sum_{j=0}^{j^*-1}\Pr\left[A_m^{(j)}\right]$ and $\Pr\left[\bigcup_{j\geq j^*} E_m^{(j)}\right]$.

From lemma 7.3.3, for any fixed j, $R_m^{(j)}$ contains k_j copies of each satisfying assignment of S_m. Therefore, from Claim 2 of [Soo+24] the size of $R_m^{(j)}$ is distributed according to Poisson $\left(\frac{g \cdot w(S_m)}{2^j}\right)$.

By Lemma 1 (Chernoff bound) of [Soo+24], we have that since $\frac{g \cdot w(S_m)}{2^{j^*-1}} > \frac{\mathsf{Thresh}}{4}$,

$$\sum_{j=0}^{j^*-1}\Pr\left[A_m^{(j)}\right] \leq 2e^{-\varepsilon^2\mathsf{Thresh}/12} + 2e^{-\varepsilon^2\mathsf{Thresh}/6} + \cdots < 4e^{-\varepsilon^2\mathsf{Thresh}/12} \leq \frac{\delta}{6},$$

where we use that $\mathsf{Thresh} > \frac{12\ln(24/\delta)}{\varepsilon^2}$ and $\varepsilon \leq 1$.

To bound $\Pr\left[\bigcup_{j\geq j^*} E_m^{(j)}\right]$, we observe that the event $\bigcup_{j\geq j^*} E_m^{(j)}$ can only occur if the *decrease p move* is triggered from $R_i^{(j^*-1)}$ to $R_i^{(j^*)}$ for some $i \in [m]$. This happens when Poisson $\left(\frac{g \cdot w(Z_i)}{2^{j^*-1}}\right)$ exceeds Thresh for a specific i. Since $\frac{g}{2^{j^*-1}} < \frac{\mathsf{Thresh}}{2w(S_m)}$ and $w(Z_i) \leq w(S_m)$, it follows that

$$\mathsf{Thresh} - \frac{g \cdot w(Z_i)}{2^{j^*-1}} > \frac{\mathsf{Thresh}}{2}.$$

By applying Lemma 1 of [Soo+24], the probability of this event is bounded by

$$e^{-\mathsf{Thresh}/6} \leq \frac{\delta}{6m},$$

where we use that $\mathsf{Thresh} \geq 6\left(\ln\left(\frac{6}{\delta}\right) + \ln m\right)$. By a union bound over i, we have that

$$\Pr\left[\bigcup_{j \geq j^*} E_m^{(j)}\right] \leq m \cdot \frac{\delta}{6m} = \frac{\delta}{6}.$$

□

7.4 Experimental Evaluation

We perform a comprehensive experimental evaluation to answer the following research questions:

RQ1. How does MC3 scale with instance size on bounded-horizon reachability queries relative to statistical and probabilistic model checkers?

RQ2. How effectively does MC3 handle rare-event reachability compared to the baselines, and what accuracy does it achieve?

RQ3. How do off-the-shelf model counters perform across different reachability queries?

Summary of Results. MC3 exhibits strong empirical scalability. Across many case studies, it outperforms all baselines and solves instances with up to hundreds of parallel processes, whereas the state-of-the-art can only solve 50. MC3 remains scalable in the rare-event regime, successfully solving instances with probabilities as low as 10^{-303}, whereas the baselines are limited to 10^{-7}. Substituting CoverMC with an off-the-shelf model counter causes a substantial performance degradation, indicating that CoverMC is a critical component of the overall pipeline[3].

Case Studies. We use all (scalable, parameterized) benchmarks from [Hol+21] and add two (standard) *Rendezvous* problems. All benchmarks describe a large number of parallel processes. See **??** for detailed description.

1. *Factory.* n independent two-state strike processes; compute the probability that all factories (or $\geq 90\%$) are striking at least once within h days, optionally with strike probabilities conditioned on a persistent two-state weather process.
2. *Rendezvous.* n participants do parallel bounded random walks on a 1D grid (or ring) of size m; compute the probability they all coincide at one position within horizon h, with density variants (e.g., $m = 2n$ vs. $3n/2$).
3. *Queue.* k capacity-n queues with stochastic arrivals; compute the probability that within k steps all three type-1 queues are full and at least one type-2 queue is full.

[3] We ran all experiments with a 32 GB memory cap, 8 CPU cores, and a 3600s timeout per (model checker, instance) run.

	Exact tools					Approximate tools	
		Prism		Storm			
class	Rubicon	(sym)	(exp)	(sym)	(exp)	Modest	mc3
factory	18	10	11	11	14	30	770
factory (weather)	17	8	10	10	13	17	550
factory (>90%)	–	11	11	11	14	50	95
factory (weather, >90%)	–	9	10	10	13	20	40
rendezvous (line)	–	5	–	5	5	8	24
rendezvous (line, dense)	5	6	–	6	5	8	22
rendezvous (ring)	–	6	–	6	5	–	8
rendezvous (ring, dense)	–	6	–	7	5	–	11
queue (10)	6	–	5	–	–	–	–
queue (15)	6	–	–	–	–	–	1000
herman	–	13	13	–	17	25	11
herman (fixed)	–	15	13	–	17	25	11

Table 7.1: Largest problem instances solved for $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$, at horizon 15.

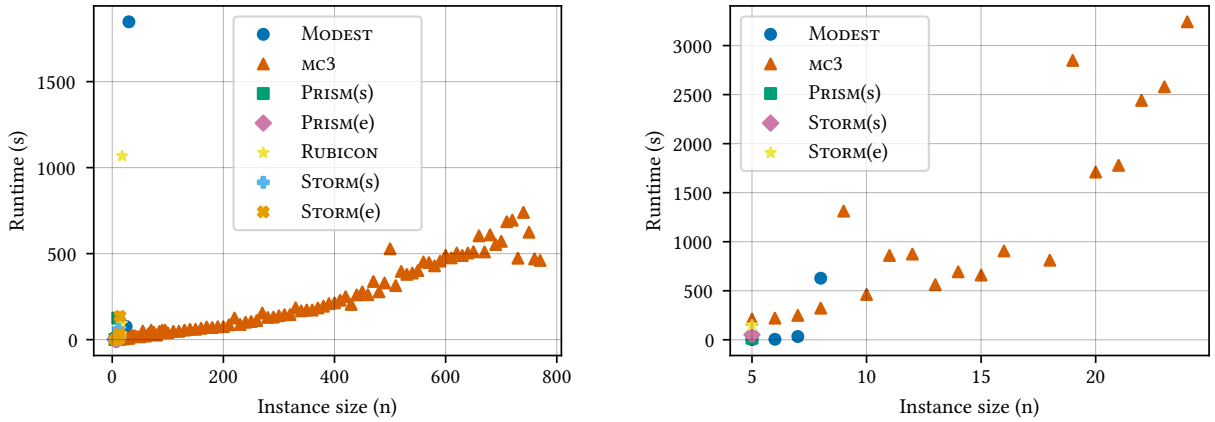
4. *Herman. n* tokens move independently on a k-ring with collision annihilation; compute the probability stabilization to a single token occurs within h steps.

Baseline. mc3 computes multiplicative approximations of probability values. Accordingly, we benchmark it against the statistical model checker Modest configured to produce multiplicative approximations as well. We use $\varepsilon = 0.8$ and $\delta = 0.2$ for both tools. We compare against Rubicon [Hol+21], which reduces probabilistic model checking to (exact) probabilistic program inference/BDD-based model counting to highlight the difference. For completeness, we also include the probabilistic model checkers Storm [Hen+22] and Prism [KNP11], using both explicit-state and symbolic modes.

RQ1. Performance w.r.t. state-of-the-art

We first evaluate scalability across instance families while keeping the horizon fixed at 15. The results are reported in Table 7.1. Each table row corresponds to a problem class, and the entries in that row indicate the largest instance from that class that each tool can solve. Instance size is measured by the parameter n for each class in the case-study description above. The graphs in Figure 7.3 show scalability in terms of time required to solve for different problem sizes by different tools. A point (x, y) denotes that an instance of size x is solved by the corresponding tool in y seconds.

On the *factory* instances, among the existing tools, Modest solves instances with up to 30 factories, whereas mc3 scales to 770 factories. Storm, Prism, and Rubicon solve instances with between 10 and 18 factories. Figure 7.3a provides further details on scalability. The runtime of mc3 grows slowly with the number of factories: the largest instance it solves (770 factories) takes 460 seconds. By comparison, Modest requires



(a) Factory instances.

(b) Rendezvous (line) instances.

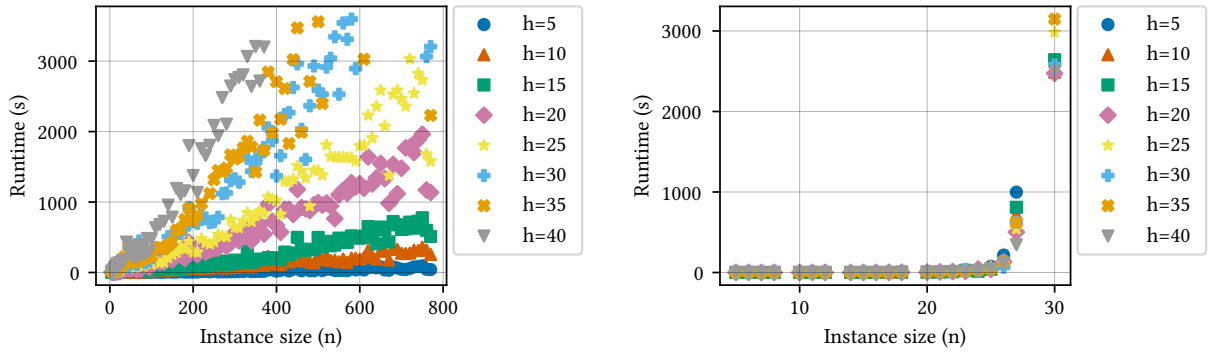
Figure 7.3: Efficiency of mc3: instance size vs. time ($h = 15$).

1847 seconds for the 30-factory instance. The weather-factory variants show comparable gains. On the weather variant, the largest instance solved by Modest has size 17, whereas mc3 solves instances of size 550. On the 90% variants, mc3 solves instances with 95 factories, almost twice as many as Modest solves. Across all cases, the exact model checkers either return an answer within a few minutes (e.g., 150 seconds for explicit-state Storm) or fail to solve the instance. In many runs, they terminate due to out-of-memory errors, and increasing the memory limit typically does not change this outcome.

On the *rendezvous (line)* benchmarks, Modest performs best among the existing tools, solving instances with up to 8 participants. In contrast, mc3 scales to 24 participants, i.e., three times as many. Figure 7.3b shows that runtimes increase in a pattern similar to the factory instances, but with substantially steeper growth. For the 8-participant instance, Modest takes 627 seconds, whereas mc3 takes 322 seconds. The largest instance solved by mc3 (24 participants) requires 3242 seconds. The exact model checkers do not scale beyond 5–6 participants.

On the *queue* and *Herman* benchmarks, we do not see improvement by mc3 over the state-of-the-art. For queues of length 10, none of the evaluated tools except Rubicon scales beyond 5 parallel queues. For queues of length 15, mc3 scales even to 1000 queues. This performance can be attributed to our choice of horizon 15: reaching a full queue within h steps is supported by a single feasible execution pattern: at each step, the queue must be extended to remain on track to saturation. Finally, for some QComp competition problems (nand, crowds, brp), our approach is not competitive as they lack a high number of processes.

Scalability w.r.t. horizon. As the horizon increases, the underlying model-counting instances become substantially more challenging, and the performance of mc3 degrades. We evaluate horizons in $\{5, 10, \ldots, 40\}$ across multiple benchmark families. On the factory benchmarks, the largest instance solved at horizon 40



(a) MC3 (b) MODEST.

Figure 7.4: Instance size vs. time at different horizons on factory instances.

contains 370 factories, which is less than half of the 770 factories solved at horizon 15. Runtime likewise increases sharply: for instance, with 370 factories, MC3 requires 3200 seconds at horizon 40, compared to 183 seconds at horizon 15. This horizon sensitivity is largely absent in existing model checkers, whose performance is typically (less than) linear in the horizon. Despite this, MC3 still solves instances with 370 factories at horizon 40, representing a substantial improvement over the state of the art: MODEST does not handle any instance with more than 30 factories. Figure 7.4a reports MC3's runtime as a function of the number of factories for different horizons, illustrating that runtime grows markedly faster as the horizon increases. For comparison, Figure 7.4b presents the analogous plot for MODEST, where the runtime varies only weakly with horizon.

RQ2. Handling rare-event scenarios and correctness

We compare the ability of different model checkers and MC3 to handle rare-event scenarios in Table 7.2. Rows correspond to benchmark families, and each entry reports the smallest probability that a tool can compute on instances from that family. For instance, on the factory benchmarks, MODEST returns probabilities as small as 1.81×10^{-7}, whereas MC3 reaches 3.17×10^{-303}. As in the table, we omit mantissas and report only exponents. These results show that, when extremely small probabilities are required, MC3 substantially outperforms state-of-the-art tools; the remaining benchmark families exhibit the same overall trend.

Correctness. We could compute exact probabilities for many instances using STORM, which allowed us to directly measure the error incurred by MC3 on those instances. We quantify the *observed error* of MC3 using $e = \frac{|b-s|}{b}$, where b is the value reported by STORM and s is the value returned by MC3. The observed error serves as an empirical analogue of the theoretical error guarantees provided by MC3. We evaluated 80 factory instances spanning 10 different sizes and 8 different horizons, and found a median error of 0.038, a geometric mean of 0.025, and a maximum of 0.163, which is substantially smaller than the theoretical

	Exact tools					Approximate tools	
		Prism		Storm			
class	Rubicon	(sym)	(exp)	(sym)	(exp)	Modest	mc3
factory	10^{-4}	10^{-2}	10^{-2}	10^{-2}	10^{-3}	10^{-7}	10^{-303}
factory (weather)	10^{-7}	10^{-4}	10^{-4}	10^{-4}	10^{-6}	10^{-7}	10^{-302}
factory (>90%)	–	10^{-1}	10^{-1}	10^{-1}	10^{-1}	10^{-7}	10^{-15}
factory (weather, >90%)	–	10^{-2}	10^{-3}	10^{-3}	10^{-4}	10^{-7}	10^{-14}
rendezvous (line)	–	10^{-3}	–	10^{-4}	10^{-4}	10^{-7}	10^{-40}
rendezvous (line, dense)	10^{-3}	10^{-4}	–	10^{-5}	10^{-3}	10^{-7}	10^{-32}
rendezvous (ring)	–	10^{-5}	–	10^{-5}	10^{-3}	–	10^{-7}
rendezvous (ring, dense)	–	10^{-4}	–	10^{-5}	10^{-3}	–	10^{-11}
queue (10)	10^{-96}	–	10^{-93}	–	–	–	–
queue (15)	10^{-160}	–	–	–	–	–	10^{-172}
herman	–	10^{-1}	10^{-1}	–	10^{-2}	10^{-2}	10^{-2}
herman (fixed)	–	10^{-1}	10^{-1}	–	10^{-2}	10^{-5}	10^{-5}

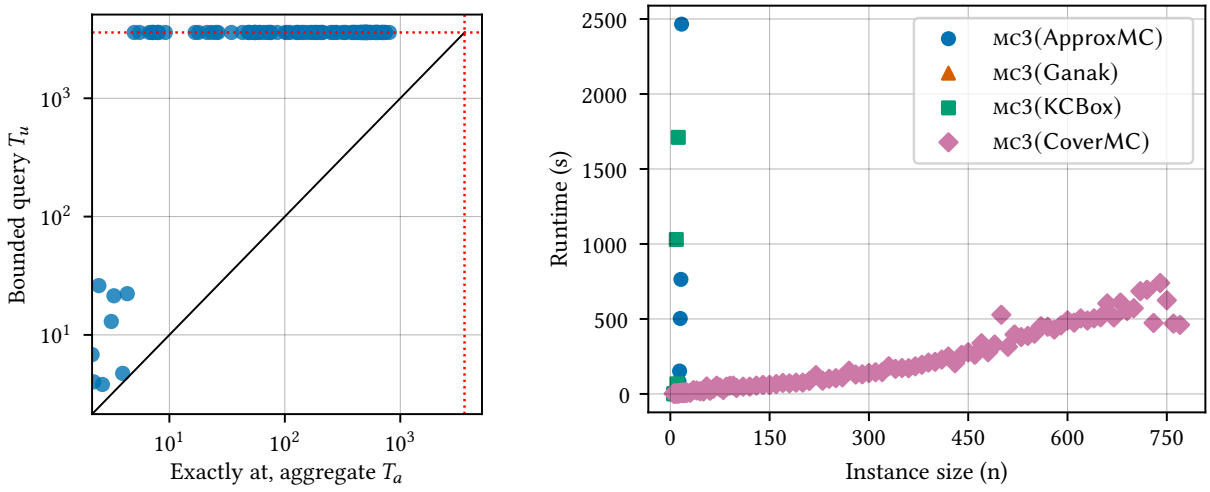
Table 7.2: Capacity to handle rare-event scenario: smallest probabilities computed.

guarantee of 0.8. These results indicate that mc3 performs significantly better than what the worst-case bounds predict.

RQ3. Performance of off-the-shelf model counters

Performance on different queries. A key empirical observation motivating this work is that the aggregate cost of answering all h step-exact queries, i.e., $\Pr_{\mathcal{M}}[\Diamond^{=i}]$ for $i \in [h]$, is often substantially smaller than answering the corresponding step-bounded query $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$. In this subsection, we substantiate this observation via experiments across our bounded-horizon reachability benchmark suite. For each bounded-horizon reachability instance, we evaluate two query families: (1) *Bounded query.* We construct the formula F encoding $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$, and measure T_u, the runtime of Ganak on F. (2) *Exactly-at queries.* For each $i \in [h]$, we construct the formula G_i encoding $\Pr_{\mathcal{M}}[\Diamond^{=i}]$. We then invoke Ganak on all G_i and record T_a, the total runtime to answer the entire family G_i, for $i \in [h]$. In both cases, we enforce a single aggregate timeout of 3600s. Figure 7.5a compares T_a and T_u via a scatter plot, where each point (x, y) corresponds to an instance with $T_a = x$ and $T_u = y$. Across the entire suite, we observe $T_a < T_u$. Moreover, for a substantial fraction of instances, T_u exceeds T_a by a wide margin, and in many cases the bounded query $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$ hits the timeout while the collection of exact queries completes. Notably, state-of-the-art model checkers are largely insensitive to this distinction between query types (see **??** in the appendix).

Performance of different counters. To isolate the contribution of covering-based model counting in mc3, we conduct an ablation study in which CoverMC is replaced by state-of-the-art exact model counters, Ganak [SM25b] and KCBox [**lmy21**], as well as the approximate model counter ApproxMC [**ym23**]. We



(a) (b)

Figure 7.5: Performance of off-the-shelf model counters: (a) Ganak on $\Pr_{\mathcal{M}}[\Diamond^{=h}]$ and $\Pr_{\mathcal{M}}[\Diamond^{\leq h}]$ queries. (b) Impact of replacing CoverMC from MC3 at $h = 15$.

denote the resulting variants by MC3(Ganak), MC3(KCBox), and MC3(ApproxMC), respectively. For clarity, we refer to the default configuration as MC3(CoverMC). Figure 7.5b reports the resulting scalability using n vs time plot. The variant MC3(Ganak) solves instances up to 13 factories, the other variants have similar performance. In contrast, MC3(CoverMC) scales to 770 factories. This gap highlights the central role of CoverMC in enabling the performance of the overall toolchain.

7.4.1 Related Work

Exact probabilistic model checking computes the probability that a stochastic model satisfies a temporal-logic specification, typically by reducing queries to numerical problems over the underlying transition system [BK08; BAFK18]. We focus on DTMCs. To tackle large state spaces, the prevalent approach is based on decision diagrams [Bai+97]. A key problem is that these decision diagrams grow with the number of different probabilities in the underlying, flat, representation of the Markov chain [DDM16]. For finite horizions, Holtzen et al. [Hol+21] used exact BDD based model counting via a probabilistic programming, which has the potential to improve upon these approaches. Alternative approaches are based on the synthesis of inductive invariants, e.g. [**DCJ+23**], or martingales, e.g., [CGMZ22], but require a small 'characteristic' (transition), whereas the distributed models that we focus on have exponentially growing characteristic functions.

Statistical probabilistic model checking aims to provide statistical guarantees. The prevalent approach (for Markov chains) replaces exhaustive state-space exploration with simulation-based inference [AP18;

Gro+25]: the system is sampled to generate execution traces, and the satisfaction probability of a property is estimated with statistical guarantees [Bud+25] : In our experiments, we compare with a statistical model checker within MODEST. Recent approaches *for MDPs*, like [Brá+25], solve a harder approach, and *do* learn an explicit state space. Closer to our work are approaches that use stochastic SAT [FHT08] or [Rab+14] , but predate advances in model counting and do not exploit the parallel structure in the DTMCs.

7.5 Open Problems and Outlook

In this work, we establish a new route to scalable probabilistic reachability analysis by reducing the problem to weighted model counting and solving it using a covering-based approximate counter. Our results open several promising directions for future work. From the perspective of probabilistic model checking: (i) While the current approach is practical for certain problem families, a key next step is to extend its applicability to a substantially wider class of model-checking tasks; (ii) The present implementation exhibits limited scalability with respect to the time horizon, and improving performance for long-horizon properties is an important direction; and (iii) It would be valuable to investigate whether the non-absorbing viewpoint, which is instrumental to the efficiency gains observed here, also provides deeper conceptual leverage for understanding probabilistic model checking more generally. From the model counting perspective, an important next step is to evaluate whether the proposed model-counting algorithm generalizes beyond the instances studied here, and to characterize the classes of model-counting problems for which it delivers robust performance.

Chapter 8

Conclusion

This thesis explored four classes of quantitative questions across SMT theories, and applied a range of algorithmic techniques drawn from the community. Some of these techniques, described in the preceding chapters, proved effective; others did not, and those failed attempts have so far gone unreported. This chapter concludes the dissertation by summarizing along two axes. (i) *Lessons learnt.* What has this thesis taught us about quantitative techniques: what worked, and, equally important, what did not. We also offer some intuition for why certain approaches failed, which we hope will guide further exploration in the community. (ii) *Future directions.* We view this work as a first modest step toward a broader program of quantitative analysis for SMT, yet it opens several avenues on both the application and solver sides.

8.1 Lessons Learnt

In Section 2.4.2, we discussed different techniques for counting that has been widely used in literature. In different chapters of this thesis, we used many of those techniques. Here, we want to reflect upon different approaches we took that did not work for different theories, in comparison to what worked in that theories.

Theory of Bit-vectors. As we studied in Chapter 3, the most scalable way to count over the theory of bitvectors is to use bitblast and count over propositional formula. This is very unsatisfying as it shows that we can't really make use of any of the structural information availabile in the bitvector formula. We tried two ways to count which uses the structural information.

1. *Decision Diagrams.* One route to decision diagram based model counting for We intended to design decision diagrams directly to the bit-vectors for counting. Jonáš and Strejček [JS19] designed decision diagrams (BDD) for solving *quantified* bit-vector formulas. We were needed to design algebraic decision diagrams for counting [DPV20]. The generated decision diagrams indeed becomes smaller.

However, we observed limited speed-ups in comparison to bit-blasting and then converting to ADD. One possible reason is that our decision diagrams are required to follow a very strict variable ordering while CNFs converted to DD are free to choose a variable order. As DDs are sensitive to ordering, our method did not scale.

2. *Hashing with integers.* In our hybrid counting based work (Chapter 4), we observed that integer-based hashing works worse than bit-blasted hashing with XORs. One of the reasons of this failure is the hashing constraints like remainder are inherently integer type constraint. Zohar et al. [Zoh+22] proposed int-blasting to convert bit-vector constraints to integers as an efficient method of solving. We tried int-blasting based underlying solver while using pact in integer hashing modes. However, that did not show significant speedup over the normal bitvector solver.

Theory of Linear Integer Arithmetic. Parallel to real arithmetic, a theory of much interest is the theory of linear integer arithmetic (QF_LIA). We wanted to port the improvement seen in terms of real arithmetic (Chapter 5) for QF_LIA as well. However, counting and sampling from the lattice space turned out to be a harder problem. The volume computation work in Chapter 5 gave a nice framework for approximating measure of SMT solution space, which can be thought of as a generalization to CDCL(T) framework. Here, the CDCL framework is replaced by the DNF formula and then a theory volume tool is used. However, this was made possible by availability of volume computation and sampling algorithms made over decades of research. The theory measure and sampling problems are still very hard for integer and bit-vectors, stopping us from using the ttc framework for other theories. Our framework allows

1. *Generating Functions* Counting number of lattice points with Barvinok's algorithm [Bar94] is the best known algorithm. Efficient implementations exist like LattE [DHTY04] and Barvinok [Ver+07]. However, the algorithm has formidable computational complexity of $O((d2^d)^{O(d)})$, and struggle to scale. Ge and Biere [GB21] suggested a decomposition based method, but that did not help much.
2. *Approximation Techniques* Despite approaches for approximating with JVV techniques by Ge [Ge24a], there has been limited improvement. Barvinok and Hartigan [BH10] used a entropy-based approximation technique, however, that had even poorer practical performance in comparison to the exact Barvinok algorithm.

Hybrid formulas. While our approach for hybrid formulas worked well for approximate counting, major practical domains need support for weights over the Boolean variables. We implemented the idea of chain formulas from Chakraborty et al. [CFMV15]. However, that added substantial overhead as the technique introduces lots of auxiliary variables. A real way forward would be to mimic the decision diagram based techniques as used in propositional model counters.

1. *Decision Diagrams.* Parallel to our hashing based approach, another possible way to do counting is to use decision diagrams. Michelutti et al. [MMSS24] made the first attempt, and extended by Masina et al. [Mas+26] towards d-DNNFs. However, both approaches require enumeration over theory lemmas and has scalability issues. We made an approach towards doing a top-down compilation. Which also has similar scalability issues. One possible way forward here is to use model constructing SAT [JBD13], which does in-solver theory propagation.

Theory of Uninterpreted functions. The Skolem function work in Chapter 6 introduced as a first step towards counting uninterpreted functions.

1. *Restrictions in Applications.* However, uninterpreted functions allow a huge class of programs including compositional applications and dependencies of variables. That should reduce to Dependency-QBF counting. Handling these would require Akkermanization and more involved studies.
2. *Support for Broader Theories.* Currently, we only support for UF over bitvectors. However, the theory of UF is applicable in any SMT theory. Extending the current structure requires efficient sampling and counting tools for the underlying theory. We have developed some of them in this thesis, but making the whole toolchain for counting with theoretical guarantees for each theory would require more theoretical studies as well as in depth implementations.

8.2 Future Directions

This thesis opens several directions, both for extending the framework to settings it does not yet cover and for applying the resulting counters to problems where quantitative analysis has been historically out of reach. We sketch the most concrete of these below.

8.2.1 Extensions of the framework

Weights in hybrid counting. The pact framework of Chapter 4 counts hybrid SMT formulas under the counting measure, but several of the driving applications below — network reliability, cyber-physical simulation validation, probabilistic program analysis — are natively weighted: each Boolean variable carries a probability, and each satisfying assignment a product weight. Our current implementation handles weights through the chain-formula reduction of Chakraborty et al. [CFMV15], which is correct but introduces auxiliary variables that erode performance. A native weighted framework for hybrid counting — analogous to how weighted CNF counters extend their unweighted counterparts — would remove this overhead and is, in our view, the most pressing extension of the present work.

Weighted Model Integration. WMI generalizes volume computation by integrating a weight density over the satisfying region and is one of the more active problems at the intersection of SMT and machine learning [BPV15; MPS19]. The volume framework of Chapter 5 computes the unweighted Lebesgue measure of an LRA formula; extending it to handle piecewise-polynomial or piecewise-linear weight densities would yield an approximate WMI counter that inherits the union-of-polytopes scalability of ttc. Identifying the largest class of weight functions for which our union-based decomposition still admits an (ε, δ) guarantee is, we believe, a promising open direction.

Non-linear arithmetic. Non-linear arithmetic, over both the integers and the reals, underlies many applications in cryptography, control, and machine-learning verification, yet remains out of reach for the techniques developed in this thesis. Even qualitative solvers for non-linear constraints frequently fall back on linear abstractions, and the polytope volume primitives that drive our LRA work do not generalize to semialgebraic sets. Two routes are worth pursuing: adapting hashing-based counting to non-linear theories, and extending the union-of-sets decomposition of Chapter 5 from polytopes to cylindrical-algebraic-decomposition cells.

8.2.2 Applications

Network Reliability. Duenas-Osorio et al. [DMPV17] cast network reliability as a propositional model counting problem. Their formalization, however, captures only connectivity constraints and ignores the powerflow constraints that govern real systems. Incorporating powerflow turns reliability estimation into a hybrid SMT counting problem, which fits naturally within the pact framework of Chapter 4. That framework relies on a weighted-to-unweighted reduction [CFMV15]; handling edge-failure probabilities more faithfully therefore motivates a weighted counting framework for hybrid constraints.

Machine Learning Models Verification. SMT solvers are among the most competitive methods for neural network verification [DND25]. Quantitative verification queries, such as robustness and fairness [Bal+19], can be naturally encoded as SMT problems. Notably, Albarghouthi et al. [ADDN17] formulated fairness quantification of programs as a weighted model integration query. This line of work, however, lost momentum as ML models grew rapidly in size, causing the community to retreat from rigorous verification toward empirical fairness testing. With the scalable SMT verification techniques we develop in this thesis, we aim to revive formal fairness verification for models of practical scale.

Cyber-physical systems verification. Autonomous vehicles require verification before deployment, and because they operate under real-world uncertainty, the question is inherently quantitative. A central problem here is sim-to-real validation: checking whether failures found in simulation also occur on real sensor data. Kim et al. [Kim+22] cast one form of this check as an SMT satisfiability query over scene labels,

asking whether a labelled scene can be generated by a Scenic scenario program. Their formulation returns a Boolean verdict and cannot attach a confidence. The SMT model counters developed in this thesis lift the query from satisfiability to counting, and so supply the missing quantitative answer.

Multiobjective Optimization. Performance indicators are central to the empirical evaluation of multiobjective optimization algorithms [Aud+21]. Among them, the hypervolume indicator is particularly prominent because it captures both convergence toward the Pareto front and diversity along the front, and is widely used to compare multiobjective evolutionary algorithms [ZDT00; GFP21]. Computing hypervolume, however, is itself a quantitative reasoning problem: each Pareto point induces an axis-aligned box bounded by a reference point, and the hypervolume is the volume of the union of these boxes [GFP21]. Equivalently, the dominated region can be encoded as a disjunction of linear real arithmetic constraints, where each disjunct describes one box. Exact hypervolume algorithms are highly optimized in low dimensions [FPL06], but the high-dimensional union-volume view suggests a complementary route through randomized approximation algorithms for geometric union volume [BF10]. This connection opens a natural application of the counting techniques developed in this thesis: scalable SMT model counters can serve as approximate hypervolume engines for many-objective optimization, especially in regimes where exact hypervolume algorithms become computationally prohibitive.

Quantum Systems Verification SMT solvers have been used extensively for verifying quantum programs [BLS23; LZS24] and quantum compilers [Tao+22], but all of these efforts target exact, noise-free semantics. Real quantum hardware operates under noise, so the operationally relevant question is not whether a circuit exactly implements its specification, but whether it agrees with the specification up to a bounded error with quantifiable confidence. I plan to extend these frameworks so that the SMT encoding captures bounded error and fidelity constraints rather than exact unitary equivalence, and then use the volume computation tool developed in Chapter 5 to compute the measure of the parameter regime under which two circuits agree within a target tolerance, yielding the first quantitative equivalence checker for noisy parameterized quantum circuits.

8.3 A Final Reflection

When this thesis began, the working hypothesis was that the qualitative apparatus of SMT — decades of solver engineering, theory reasoning, abstraction architectures — could be lifted, with care, to answer questions that are quantitative at heart. The four tools and the unifying framework reported in the preceding chapters are an attempt to make good on that hypothesis across discrete, continuous, hybrid, and functional domains, and the experimental gains suggest the hypothesis was at least worth betting on. None of these tools is the last word on its problem; each is a working system that pushes its corner of the quantitative

landscape further than it had been pushed before, and each makes a little clearer what the next push will require. As more of the systems we build behave probabilistically, learn from data, and operate under genuine uncertainty, the questions they pose to automated reasoning will increasingly be quantitative ones, and the apparatus that answers them will have to grow with them. The work in this thesis is one small step in that direction; we hope it is useful to those who take the next.

Bibliography

[ABL13] Carlos Ansótegui, Maria Luisa Bonet, and Jordi Levy. “SAT-based MaxSAT algorithms”. In: *Artificial Intelligence* (2013).

[ACD20] Ralph Abboud, İsmail İlkan Ceylan, and Radoslav Dimitrov. “On the Approximability of Weighted Model Integration on DNF Structures”. In: *Proc. of KR*. 2020.

[ACD22] Ralph Abboud, İsmail İlkan Ceylan, and Radoslav Dimitrov. “Approximate weighted model integration on DNF structures”. In: *Artif. Intell.* (2022).

[ACJS17] S Akshay, Supratik Chakraborty, Ajith K John, and Shetal Shah. “Towards parallel Boolean functional synthesis”. In: *Proc. of TACAS*. Springer. 2017.

[ADDN17] Aws Albarghouthi, Loris D’Antoni, Samuel Drews, and Aditya V Nori. “Fairsquare: probabilistic verification of program fairness”. In: *Proceedings of the ACM on Programming Languages* OOPSLA (2017).

[ADG16] Aws Albarghouthi, Isil Dillig, and Arie Gurfinkel. “Maximal specification synthesis”. In: *ACM SIGPLAN Notices* 1 (2016).

[AFT26] Samantha Archer, Mohammad Rahmani Fadiheh, and Caroline Trippel. “Helium: Quantifying Microarchitectural Side-Channel Leakage with Probabilistic Guarantees”. In: (2026).

[AH99] Rajeev Alur and Thomas A. Henzinger. “Reactive Modules”. In: *Formal Methods Syst. Des.* 1 (1999).

[AK91] David Applegate and Ravi Kannan. “Sampling and integration of near log-concave functions”. In: *Proceedings of the twenty-third annual ACM symposium on Theory of computing*. 1991.

[Aks+18] S Akshay, Supratik Chakraborty, Shubham Goel, Sumith Kulal, and Shetal Shah. “What’s hard about Boolean Functional Synthesis?” In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2018.

[Aks+19] S Akshay, Jatin Arora, Supratik Chakraborty, S Krishna, Divya Raghunathan, and Shetal Shah. “Knowledge Compilation for Boolean Functional Synthesis”. In: *Proceedings of Formal Methods in Computer-Aided Design (FMCAD)*. 2019.

[AP18] Gul Agha and Karl Palmskog. “A Survey of Statistical Model Checking”. In: *ACM Trans. Model. Comput. Simul.* 1 (2018).

[Aud+21] Charles Audet, Jean Bigeon, Dominique Cartier, Sébastien Le Digabel, and Ludovic Salomon. "Performance indicators in multiobjective optimization". In: *European journal of operational research* (2021).

[BAFK18] Christel Baier, Luca de Alfaro, Vojtěch Forejt, and Marta Kwiatkowska. "Model Checking Probabilistic Systems". In: *Handbook of Model Checking*. 2018.

[Bai+97] Christel Baier, Edmund M. Clarke, Vasiliki Hartonas-Garmhausen, Marta Z. Kwiatkowska, and Mark Ryan. "Symbolic Model Checking for Probabilistic Processes". In: *Proceedings of International Colloquium on Automata, Languages and Programming (ICALP)*. 1997.

[Bal+19] Teodora Baluta, Shiqi Shen, Shweta Shinde, Kuldeep S Meel, and Prateek Saxena. "Quantitative verification of neural networks and its security applications". In: *Proc. of CCS*. 2019.

[Bar+05] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K Rustan M Leino. "Boogie: A modular reusable verifier for object-oriented programs". In: *Proceedings of International Symposium on Formal Methods for Components and Objects*. 2005.

[Bar+22] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. "cvc5: A versatile and industrial-strength SMT solver". In: *Proc. of TACAS*. 2022.

[Bar94] Alexander I Barvinok. "A polynomial time algorithm for counting integral points in polyhedra when the dimension is fixed". In: *Mathematics of Operations Research* 4 (1994).

[BB09] Robert Brummayer and Armin Biere. "Boolector: An efficient SMT solver for bit-vectors and arrays". In: *Proc. of TACAS*. 2009.

[BDHS18] Carlos E. Budde, Pedro R. D'Argenio, Arnd Hartmanns, and Sean Sedwards. "A Statistical Model Checker for Nondeterminism and Rare Events". In: *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 2018.

[BELM12] Bernd Becker, Rüdiger Ehlers, Matthew Lewis, and Paolo Marin. "ALLQBF solving by computational learning". In: *Proc. of ATVA*. 2012.

[BF10] Karl Bringmann and Tobias Friedrich. "Approximating the volume of unions and intersections of high-dimensional geometric objects". In: *Computational Geometry* 43 (2010).

[BFT16] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. *The Satisfiability Modulo Theories Library (SMT-LIB)*. `www.SMT-LIB.org`. 2016.

[BH10] Alexander Barvinok and John A Hartigan. "Maximum entropy Gaussian approximations for the number of integer points and volumes of polytopes". In: *Advances in Applied Mathematics* 2 (2010).

[BJ11] Valeriy Balabanov and Jie-Hong R Jiang. "Resolution proofs and Skolem functions in QBF evaluation and applications". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2011.

[BK08] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. 2008.

[BLS23] Fabian Bauer-Marquart, Stefan Leue, and Christian Schilling. "symQV: Automated Symbolic Verification". In: *Proceedings of Formal Methods*. 2023.

[BM15] Benoit Baudry and Martin Monperrus. "The multiple facets of software diversity: Recent developments in year 2000 and beyond". In: *ACM Computing Surveys (CSUR)* 1 (2015).

[BPV15] Vaishak Belle, Andrea Passerini, and Guy Van den Broeck. "Probabilistic inference in hybrid domains by weighted model integration". In: *Proc. of IJCAI*. 2015.

[Brá+25] Tomás Brázdil, Krishnendu Chatterjee, Martin Chmelik, Vojtech Forejt, Jan Kretínský, Marta Kwiatkowska, Tobias Meggendorfer, David Parker, and Mateusz Ujma. "Learning Algorithms for Verification of Markov Decision Processes". In: *TheoretiCS* (2025).

[Bry86] Randal E Bryant. "Graph-based algorithms for boolean function manipulation". In: *Computers, IEEE Transactions on* 100.8 (1986), pp. 677–691.

[BSST21] Clark Barrett, Roberto Sebastiani, Sanjit A Seshia, and Cesare Tinelli. "Satisfiability modulo theories". In: *Handbook of satisfiability*. 2021.

[BST+10] Clark Barrett, Aaron Stump, Cesare Tinelli, et al. "The smt-lib standard: Version 2.0". In: *Proc. of SMT Workshop*. 2010.

[Bud+17] Carlos E. Budde, Christian Dehnert, Ernst Moritz Hahn, Arnd Hartmanns, Sebastian Junges, and Andrea Turrini. "JANI: Quantitative Model and Tool Interaction". In: *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 2017.

[Bud+25] Carlos E. Budde, Arnd Hartmanns, Tobias Meggendorfer, Maximilian Weininger, and Patrick Wienhöft. "Sound Statistical Model Checking for Probabilities and Expected Rewards". In: *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 2025.

[BZG20] Gabrielle Beck, Maximilian Zinkus, and Matthew Green. "Automating the development of chosen ciphertext attacks". In: *29th USENIX Security Symposium (USENIX Security 20)*. 2020.

[CDM15] Dmitry Chistikov, Rayna Dimitrova, and Rupak Majumdar. "Approximate counting in SMT and value estimation for probabilistic programs". In: *Proc. of TACAS*. 2015.

[CDM17] Dmitry Chistikov, Rayna Dimitrova, and Rupak Majumdar. "Approximate counting in SMT and value estimation for probabilistic programs". In: *Acta Informatica* 8 (2017).

[CF21] Apostolos Chalkis and Vissarion Fisikopoulos. "volesti: Volume Approximation and Sampling for Convex Polytopes in R." In: *R Journal* 2 (2021).

[CFMV15] Supratik Chakraborty, Dror Fried, Kuldeep S. Meel, and Moshe Y. Vardi. "From Weighted to Unweighted Model Counting". In: *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*. 2015.

[CFTV18] Supratik Chakraborty, Dror Fried, Lucas M Tabajara, and Moshe Y Vardi. "Functional synthesis via input-output separation". In: *Proc. of FMCAD*. 2018.

[CGMZ22] Krishnendu Chatterjee, Amir Kafshdar Goharshady, Tobias Meggendorfer, and Dorde Zikelic. "Sound and Complete Certificates for Quantitative Termination Analysis of Probabilistic Programs". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. Lecture Notes in Computer Science. 2022.

[CGSS13] Alessandro Cimatti, Alberto Griggio, Bastiaan Joost Schaafsma, and Roberto Sebastiani. "The mathsat5 SMT solver". In: *Proc. of TACAS*. 2013.

[Cha+15] Supratik Chakraborty, Daniel J Fremont, Kuldeep S Meel, Sanjit A Seshia, and Moshe Y Vardi. "On parallel scalable uniform SAT witness generation". In: *Proc. of TACAS*. Springer. 2015.

[CM19] Sourav Chakraborty and Kuldeep S. Meel. "On testing of Uniform Samplers". In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2019.

[CMMV16] Supratik Chakraborty, Kuldeep Meel, Rakesh Mistry, and Moshe Vardi. "Approximate probabilistic inference via word-level counting". In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2016.

[CMT12] Alessandro Cimatti, Sergio Mover, and Stefano Tonetta. "SMT-based verification of hybrid systems". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 1. 2012.

[CMV13] Supratik Chakraborty, Kuldeep S Meel, and Moshe Y Vardi. "A scalable approximate model counter". In: *Proceedings of International Conference on Constraint Programming (CP)*. 2013.

[CMV14] Supratik Chakraborty, Kuldeep S. Meel, and Moshe Y. Vardi. "Balancing Scalability and Uniformity in SAT-Witness Generator". In: *Proc. of DAC*. 2014.

[CMV16] Supratik Chakraborty, Kuldeep S Meel, and Moshe Y Vardi. "Algorithmic Improvements in Approximate Counting for Probabilistic Inference: From Linear to Logarithmic SAT Calls." In: *Proc. of IJCAI*. 2016.

[CMV21] Supratik Chakraborty, Kuldeep S Meel, and Moshe Y Vardi. "Approximate model counting". In: *Handbook of Satisfiability*. 2021.

[Coo22] Byron Cook. *Automated reasoning's scientific frontiers*. [link]. Amazon Science blog post. 2022.

[CV16] Ben Cousins and Santosh Vempala. "A Practical Volume Algorithm". In: *Mathematical Programming Computation* 2 (2016).

[CV18] Ben Cousins and Santosh Vempala. "Gaussian Cooling and Oˆ*(nˆ3) Algorithms for Volume and Gaussian Volume". In: *SIAM Journal on Computing* 3 (2018).

[CVM22] Sourav Chakraborty, N. V. Vinodchandran, and Kuldeep S. Meel. "Distinct Elements in Streams: An Algorithm for the (Text) Book". In: *Proceedings of European Symposium of Algorithms (ESA)*. 2022.

[CW77] J Lawrence Carter and Mark N Wegman. "Universal classes of hash functions". In: *Proc. of STOC*. 1977.

[Dar01] Adnan Darwiche. "Decomposable negation normal form". In: *Journal of the ACM (JACM)* 4 (2001).

[Dav+15] Alexandre David, Kim G. Larsen, Axel Legay, Marius Mikucionis, and Danny Bøgsted Poulsen. "Uppaal SMC tutorial". In: *Int. J. Softw. Tools Technol. Transf.* 4 (2015).

[DB08] Leonardo De Moura and Nikolaj Bjørner. "Z3: An efficient SMT solver". In: *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer. 2008.

[DDM16] David Deininger, Rayna Dimitrova, and Rupak Majumdar. "Symbolic Model Checking for Factored Probabilistic Models". In: *Proceedings of International Symposium on Automated Technology for Verification and Analysis (ATVA)*. Lecture Notes in Computer Science. 2016.

[DF88] M. E. Dyer and A. M. Frieze. "On the Complexity of Computing the Volume of a Polyhedron". In: *SIAM Journal on Computing* 5 (1988).

[DFK91] Martin Dyer, Alan Frieze, and Ravi Kannan. "A random polynomial-time algorithm for approximating the volume of convex bodies". In: *Journal of the ACM (JACM)* 1 (1991).

[DHKV21] Arnaud Durand, Anselm Haak, Juha Kontinen, and Heribert Vollmer. "Descriptive complexity of #P functions: A new perspective". In: *Journal of Computer and System Sciences* (2021).

[DHTY04] Jesús A De Loera, Raymond Hemmecke, Jeremiah Tauzer, and Ruriko Yoshida. "Effective lattice point counting in rational convex polytopes". In: *Journal of symbolic computation* 4 (2004).

[Die96] Martin Dietzfelbinger. "Universal hashing and k-wise independent random variables via integer arithmetic without primes". In: *Proc. of STACS*. 1996.

[DKLR95] P Dagum, R Karp, M Luby, and S Ross. "An optimal algorithm for Monte Carlo estimation". In: *Proceedings of IEEE 36th Annual Foundations of Computer Science*. 1995.

[DL97] Paul Dagum and Michael Luby. "An optimal approximation algorithm for Bayesian inference". In: *Artificial Intelligence* (1997).

[DM02] Adnan Darwiche and Pierre Marquis. "A knowledge compilation map". In: *Journal of Artificial Intelligence Research* 17 (2002), pp. 229–264.

[DM22] Remi Delannoy and Kuldeep S Meel. "On Almost-Uniform Generation of SAT Solutions: The power of 3-wise independent hashing". In: *Proc. of LICS*. 2022.

[DMPV17] Leonardo Duenas-Osorio, Kuldeep Meel, Roger Paredes, and Moshe Vardi. "Counting-based reliability estimation for power-transmission grids". In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2017.

[DND25] Hai Duong, ThanhVu Nguyen, and Matthew B Dwyer. "Neuralsat: A high-performance verification tool for deep neural networks". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2025.

[DPV20] Jeffrey M. Dudek, Vu H.N. Phan, and Moshe Y. Vardi. "ADDMC: Weighted model counting with algebraic decision diagrams". In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)* (2020). ISSN: 2159-5399.

[EMS07] Niklas Eén, Alan Mishchenko, and Niklas Sörensson. "Applying logic synthesis for speeding up SAT". In: *Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT)*. 2007.

[FHS24a] Johannes Fichte, Markus Hecher, and Arijit Shaw. *Model Counting Competition 2024: Submitted Solvers*. Zenodo, 2024. DOI: 10.5281/zenodo.14249109.

[FHS24b] Johannes Fichte, Markus Hecher, and Arijit Shaw. "Model Counting Competition Data Format (version 1.1)". In: (2024).

[FHT08] Martin Fränzle, Holger Hermanns, and Tino Teige. "Stochastic Satisfiability Modulo Theory: A Novel Technique for the Analysis of Probabilistic Hybrid Systems". In: *HSCC*. Lecture Notes in Computer Science. 2008.

[FM85] Philippe Flajolet and G Nigel Martin. "Probabilistic counting algorithms for data base applications". In: *Journal of computer and system sciences* 2 (1985).

[FNS23] Dror Fried, Alexander Nadel, and Yogev Shalmon. "AllSAT for Combinational Circuits". In: *Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT)*. 2023.

[FNSS24] Dror Fried, Alexander Nadel, Roberto Sebastiani, and Yogev Shalmon. "Entailing generalization boosts enumeration". In: *Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT)*. 2024.

[FPL06] Carlos M Fonseca, Luís Paquete, and Manuel López-Ibánez. "An improved dimension-sweep algorithm for the hypervolume indicator". In: *IEEE international conference on evolutionary computation*. 2006.

[Fro+21] Nils Froleyks, Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda. "SAT competition 2020". In: *Artificial Intelligence* (2021).

[GB21] Cunjing Ge and Armin Biere. "Decomposition Strategies to Count Integer Solutions over Linear Constraints." In: *Proc. of IJCAI*. 2021.

[GD07] Vijay Ganesh and David L Dill. "A decision procedure for bit-vectors and arrays". In: *International conference on computer aided verification*. Springer. 2007.

[Ge+18] Cunjing Ge, Feifei Ma, Tian Liu, Jian Zhang, and Xutong Ma. "A New Probabilistic Algorithm for Approximate Model Counting". In: *Proc. of IJCAR*. 2018.

[Ge+19] Cunjing Ge, Feifei Ma, Xutong Ma, Fan Zhang, Pei Huang, and Jian Zhang. "Approximating integer solution counting via space quantification for linear constraints". In: *Proc. of IJCAI*. 2019.

[Ge24a] Cunjing Ge. "Approximate Integer Solution Counts over Linear Arithmetic Constraints". In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2024.

[Ge24b] Cunjing Ge. "sharpSMT: A Scalable Toolkit for Measuring Solution Spaces of SMT(LA) Formulas". In: *Frontiers of Computer Science (FCS)* (2024).

[GFB21] Guillaume Girol, Benjamin Farinier, and Sébastien Bardin. "Not All Bugs Are Created Equal, But Robust Reachability Can Tell the Difference". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. Springer. 2021.

[GFP21] Andreia P Guerreiro, Carlos M Fonseca, and Luís Paquete. "The hypervolume indicator: Computational problems and algorithms". In: *ACM Computing Surveys (CSUR)* (2021).

[GMZ18] Cunjing Ge, Feifei Ma, and Jian Zhang. "VolCE: An Efficient Tool for Solving #SMT(LA) Problems". In: *Proc. of PRUV Workshop at IJCAR*. 2018.

[GMZZ18] Cunjing Ge, Feifei Ma, Peng Zhang, and Jian Zhang. "Computing and estimating the volume of the solution space of SMT (LA) constraints". In: *Theoretical Computer Science* (2018).

[GRM20] Priyanka Golia, Subhajit Roy, and Kuldeep S Meel. "Manthan: A data-driven approach for Boolean function synthesis". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. Springer. 2020.

[GRM21] Priyanka Golia, Subhajit Roy, and Kuldeep S. Meel. "Program Synthesis as Dependency Quantified Formula Modulo Theory". In: *Proc. of IJCAI*. 2021.

[Gro+25] Timo P. Gros, Arnd Hartmanns, Ivo Hoese, Joshua Meyer, Nicola J. Müller, and Verena Wolf. "PyDSMC: Statistical Model Checking for Neural Agents Using the Gymnasium Interface". In: *QEST+FORMATS*. Lecture Notes in Computer Science. 2025.

[GSRM21] Priyanka Golia, Friedrich Slivovsky, Subhajit Roy, and Kuldeep S Meel. "Engineering an efficient boolean functional synthesis engine". In: *Proc. of ICCAD*. IEEE. 2021.

[GSS21] Carla P Gomes, Ashish Sabharwal, and Bart Selman. "Model counting". In: *Handbook of satisfiability*. 2021.

[GT01] Phillip B Gibbons and Srikanta Tirthapura. "Estimating simple functions on the union of data streams". In: *Proc. of SPAA*. 2001.

[Had+14] Liana Hadarean, Kshitij Bansal, Dejan Jovanović, Clark Barrett, and Cesare Tinelli. "A Tale of Two Solvers: Eager and Lazy Approaches to Bit-Vectors". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2014.

[Hen+22] Christian Hensel, Sebastian Junges, Joost-Pieter Katoen, Tim Quatmann, and Matthias Volk. "The probabilistic model checker Storm". In: *International Journal on Software Tools for Technology Transfer* 4 (2022).

[HFS25] Markus Hecher, Johannes K. Fichte, and Arijit Shaw. *Model Counting Competition 2025: Description*. Model Counting Competition. 2025. URL: https://mccompetition.org/2025/mc_description.html (visited on 12/15/2025).

[HH14] Arnd Hartmanns and Holger Hermanns. "The Modest Toolset: An integrated environment for quantitative modelling and verification". In: *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 2014.

[HJ19] Ákos Hajdu and Dejan Jovanović. "solc-verify: A modular verifier for solidity smart contracts". In: *Proceedings of Working conference on verified software: theories, tools, and experiments*. Springer. 2019, pp. 161–179.

[HLMP04] Thomas Hérault, Richard Lassaigne, Frédéric Magniette, and Sylvain Peyronnet. "Approximate Probabilistic Model Checking". In: *Proceedings of International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*. 2004.

[HLZ21] Thomas A Henzinger, Mathias Lechner, and Djordje Zikelic. "Scalable verification of quantized neural networks". In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2021.

[Hol+21] Steven Holtzen, Sebastian Junges, Marcell Vazquez-Chanlatte, Todd Millstein, Sanjit A Seshia, and Guy Van den Broeck. "Model checking finite-horizon Markov chains with probabilistic inference". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2021.

[HV19] Anselm Haak and Heribert Vollmer. "A model-theoretic characterization of constant-depth arithmetic circuits". In: *Annals of Pure and Applied Logic* 9 (2019).

[JBD13] Dejan Jovanovic, Clark Barrett, and Leonardo De Moura. "The design and implementation of the model constructing satisfiability calculus". In: *Proceedings of Formal Methods in Computer-Aided Design (FMCAD)*. 2013.

[Jia09] Jie-Hong R Jiang. "Quantifier elimination via functional composition". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2009.

[JLH09] Jie-Hong Roland Jiang, Hsuan-Po Lin, and Wei-Lun Hung. "Interpolating functions from large Boolean relations". In: *Proc. of ICCAD*. 2009.

[JLS09] Susmit Jha, Rhishikesh Limaye, and Sanjit A Seshia. "Beaver: Engineering an efficient smt solver for bit-vector arithmetic". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. Springer. 2009.

[Joh+15] Ajith K John, Shetal Shah, Supratik Chakraborty, Ashutosh Trivedi, and S Akshay. "Skolem functions for factored formulas". In: *Proc. of FMCAD*. 2015.

[JS19] Martin Jonáš and Jan Strejček. “Q3B: an efficient BDD-based SMT solver for quantified bit-vectors”. In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2019.

[JVV86] Mark R Jerrum, Leslie G Valiant, and Vijay V Vazirani. “Random generation of combinatorial structures from a uniform distribution”. In: *Theoretical computer science* (1986).

[Kat+17] Guy Katz, Clark Barrett, David L Dill, Kyle Julian, and Mykel J Kochenderfer. “Reluplex: An efficient SMT solver for verifying deep neural networks”. In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2017.

[KFB16] Gergely Kovásznai, Andreas Fröhlich, and Armin Biere. “Complexity of Fixed-Size Bit-Vector Logics”. In: *Theory of Computing Systems* 2 (2016).

[Kim+22] Edward Kim, Jay Shenoy, Sebastian Junges, Daniel J Fremont, Alberto Sangiovanni-Vincentelli, and Sanjit A Seshia. “Querying labelled data with scenario programs for sim-to-real validation”. In: *International Conference on Cyber-Physical Systems (ICCPS)*. 2022.

[KJ21] Tuukka Korhonen and Matti Järvisalo. “Integrating Tree Decompositions into Decision Heuristics of Propositional Model Counters”. In: *Proc. of CP*. 2021.

[KL83] Richard M Karp and Michael Luby. “Monte-Carlo algorithms for enumeration and reliability problems”. In: *Proc. of Annual Symposium on Foundations of Computer Science*. 1983.

[KLM89] Richard M Karp, Michael Luby, and Neal Madras. “Monte-Carlo approximation algorithms for enumeration problems”. In: *Journal of algorithms* 3 (1989).

[KLS97] Ravi Kannan, László Lovász, and Miklós Simonovits. “Random walks and an o*(n5) volume algorithm for convex bodies”. In: *Random Structures & Algorithms* 1 (1997).

[KM18] Seonmo Kim and Stephen McCamant. “Bit-vector model counting using statistical estimation”. In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer. 2018.

[KNP11] Marta Kwiatkowska, Gethin Norman, and David Parker. “PRISM 4.0: Verification of probabilistic real-time systems”. In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2011.

[KNW10] Daniel M Kane, Jelani Nelson, and David P Woodruff. “An optimal algorithm for the distinct elements problem”. In: *Proc. of PODS*. 2010.

[Kol+18] Samuel Kolb, Martin Mladenov, Scott Sanner, Vaishak Belle, and Kristian Kersting. “Efficient symbolic integration for probabilistic inference”. In: *Proc. of IJCAI*. 2018.

[Kol+23] Ipsita Koley, Soumyajit Dey, Debdeep Mukhopadhyay, Sachin Singh, Lavanya Lokesh, and Shantaram Vishwanath Ghotgalkar. “CAD Support for Security and Robustness Analysis of Safety-critical Automotive Software”. In: *ACM Transactions on Cyber-Physical Systems* (2023).

[KS16] Daniel Kroening and Ofer Strichman. *Decision procedures*. 2016.

[KV97] Ravi Kannan and Santosh Vempala. "Sampling lattice points". In: *Proc. of STOC*. 1997.

[LD12] László Lovász and István Deák. "Computational results of an O(n4) volume algorithm". In: *European journal of operational research* 1 (2012).

[Lei10] K Rustan M Leino. "Dafny: An automatic program verifier for functional correctness". In: *Proceedings of International conference on logic for programming artificial intelligence and reasoning*. 2010.

[LLM16] Jean-Marie Lagniez, Emmanuel Lonca, and Pierre Marquis. "Improving Model Counting by Leveraging Definability." In: *Proc. of IJCAI*. 2016.

[LM17] Jean-Marie Lagniez and Pierre Marquis. "An Improved Decision-DNNF Compiler." In: *Proc. of IJCAI*. 2017.

[LM21] Chu Min Li and Felip Manya. "MaxSAT, hard and soft constraints". In: *Handbook of satisfiability*. 2021.

[Lov91] László Lovász. *How to compute the volume?* 1991.

[LS90] László Lovász and Miklós Simonovits. "The mixing rate of Markov chains, an isoperimetric inequality, and computing the volume". In: *Proceedings [1990] 31st annual symposium on foundations of computer science*. IEEE. 1990.

[LS92] László Lovász and Miklós Simonovits. "On the randomized complexity of volume and diameter". In: *Proc. of FOCS*. IEEE Computer Society. 1992.

[LS93] László Lovász and Miklós Simonovits. "Random walks in a convex body and an improved volume algorithm". In: *Random structures & algorithms* 4 (1993).

[LV04] László Lovász and Santosh Vempala. "Hit-and-run from a corner". In: *Proc. of STOC*. 2004.

[LV06] László Lovász and Santosh Vempala. "Simulated annealing in convex bodies and an O*(n4) volume algorithm". In: *Journal of Computer and System Sciences* 2 (2006).

[LZS24] Marco Lewis, Paolo Zuliani, and Sadegh Soudjani. "Automated Verification of Silq Quantum Programs Using SMT Solvers". In: *2024 IEEE International Conference on Quantum Software (QSW)*. IEEE. 2024, pp. 125–134.

[Mas+26] Gabriele Masina, Emanuale Civini, Massimo Michelutti, Giuseppe Spallitta, and Roberto Sebastiani. "d-DNNF Modulo Theories: A General Framework for Polytime SMT Queries". In: *arXiv e-prints* (2026), arXiv–2603.

[Mat+18] Cristian Mattarei, Makai Mann, Clark Barrett, Ross G Daly, Dillon Huff, and Pat Hanrahan. "Cosa: Integrated verification for agile hardware design". In: *Proc. of FMCAD*. 2018.

[MLZ09] Feifei Ma, Sheng Liu, and Jian Zhang. "Volume computation for boolean combination of linear arithmetic constraints". In: *Proc. of CADE*. 2009.

[MMSS24] Massimo Michelutti, Gabriele Masina, Giuseppe Spallitta, and Roberto Sebastiani. "Canonical Decision Diagrams Modulo Theories". In: *ECAI 2024*. IOS Press, 2024, pp. 4319–4327.

[MPS17] Paolo Morettin, Andrea Passerini, and Roberto Sebastiani. “Efficient weighted model integration via SMT-based predicate abstraction”. In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2017.

[MPS19] Paolo Morettin, Andrea Passerini, and Roberto Sebastiani. “Advanced SMT techniques for weighted model integration”. In: *Artificial Intelligence* (2019).

[MVC21] Kuldeep S. Meel, N.V. Vinodchandran, and Sourav Chakraborty. “Estimating the Size of Union of Sets in Streaming Models”. In: *Proc. of PODS*. 2021.

[NP23] Aina Niemetz and Mathias Preiner. “Bitwuzla”. In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2023.

[NPZ12] Đurica Nikolić, Corrado Priami, and Roberto Zunino. “A rule-based and imperative language for biochemical modeling and simulation”. In: *Proceedings of International Conference on Software Engineering and Formal Methods*. Springer. 2012, pp. 16–32.

[PFMD21] Sumanth Prabhu, Grigory Fedyukovich, Kumar Madhukar, and Deepak D’Souza. “Specification synthesis with constrained Horn clauses”. In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 2021.

[PMS23] Andreas Plank, Sibylle Möhle, and Martina Seidl. “Enumerative Level-2 Solution Counting for Quantified Boolean Formulas (Short Paper)”. In: *Proc. of CP*. 2023.

[Rab+14] Markus N. Rabe, Christoph M. Wintersteiger, Hillel Kugler, Boyan Yordanov, and Youssef Hamadi. “Symbolic Approximation of the Bounded Reachability Probability in Large Markov Chains”. In: *Proceedings of the International Conference on Quantitative Evaluation of SysTems (QEST)*. Lecture Notes in Computer Science. 2014.

[Rab19] Markus N Rabe. “Incremental Determinization for Quantifier Elimination and Functional Synthesis”. In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2019.

[RS16] Markus N. Rabe and Sanjit A. Seshia. “Incremental Determinization”. In: *Proc. of SAT*. 2016.

[RTRS18] Markus N Rabe, Leander Tentrup, Cameron Rasmussen, and Sanjit A Seshia. “Understanding and extending incremental determinization for 2QBF”. In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2018.

[SJM24] Arijit Shaw, Brendan Juba, and Kuldeep S. Meel. “An Approximate Skolem Function Counter”. In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2024.

[SJM26] Arijit Shaw, Sebastian Junges, and Kuldeep S. Meel. “Covering-based Approximate Model Counting for Statistical Model Checking”. In: *Under Submission*. 2026.

[SM19] Mate Soos and Kuldeep S Meel. “BIRD: engineering an efficient CNF-XOR SAT solver and its applications to approximate model counting”. In: *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*. 2019.

[SM24] Arijit Shaw and Kuldeep S. Meel. "Model Counting in the Wild". In: *Proceedings of International Conference on Knowledge Representation and Reasoning (KR)*. 2024.

[SM25a] Arijit Shaw and Kuldeep S Meel. "Approximate SMT Counting Beyond Discrete Domains". In: *Proceedings of Design Automation Conference (DAC)*. 2025.

[SM25b] Mate Soos and Kuldeep S Meel. "Engineering an Efficient Probabilistic Exact Model Counter". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2025.

[SM26] Arijit Shaw and Kuldeep S Meel. "CSB: A Counting and Sampling Tool for Bitvectors". In: *Acta Informatica*. 2026.

[Smi84] Robert L Smith. "Efficient Monte Carlo procedures for generating points uniformly distributed over bounded regions". In: *Operations Research* 6 (1984).

[SMKS22] Ankit Shukla, Sibylle Möhle, Manuel Kauers, and Martina Seidl. "Outercount: A first-level solution-counter for quantified boolean formulas". In: *Proc. of CICM*. 2022.

[SNC09] Mate Soos, Karsten Nohl, and Claude Castelluccia. "Extending SAT Solvers to Cryptographic Problems". In: *Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT)*. Springer. 2009.

[Soo+24] Mate Soos, Uddalok Sarkar, Divesh Aggarwal, Sourav Chakraborty, Kuldeep S Meel, and Maciej Obremski. "Engineering an efficient approximate DNF-counter". In: *arXiv preprint arXiv:2407.19946* (2024).

[Spa+22] Giuseppe Spallitta, Gabriele Masina, Paolo Morettin, Andrea Passerini, and Roberto Sebastiani. "SMT-based weighted model integration with structure awareness". In: *Uncertainty in Artificial Intelligence*. PMLR. 2022.

[Spa+24] Giuseppe Spallitta, Gabriele Masina, Paolo Morettin, Andrea Passerini, and Roberto Sebastiani. "Enhancing SMT-based Weighted Model Integration by structure awareness". In: *Artificial Intelligence* (2024).

[SRSM19] Shubham Sharma, Subhajit Roy, Mate Soos, and Kuldeep S Meel. "GANAK: A Scalable Probabilistic Exact Model Counter." In: *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*. 2019.

[SSM25] Arijit Shaw, Uddalok Sarkar, and Kuldeep S Meel. "Efficient Volume Computation for SMT Formulas". In: *Proceedings of Knowledge Representation and Reasoning (KR)*. 2025.

[Sto83] Larry Stockmeyer. "The complexity of approximate counting". In: *Proc. of STOC*. 1983.

[Tao+22] Runzhou Tao, Yunong Shi, Jianan Yao, Xupeng Li, Ali Javadi-Abhari, Andrew W Cross, Frederic T Chong, and Ronghui Gu. "Giallar: Push-button verification for the Qiskit quantum compiler". In: *Proceedings of Programming Language Design and Implementation*. 2022.

[Ter26] Wesley W. Terpstra. *GitHub Issue #12, meelgroup/csb*. 2026. URL: https://github.com/meelgroup/csb/issues/12.

[Tho15] Mikkel Thorup. "High speed hashing for integers and strings". In: *arXiv preprint arXiv:1504.06804* (2015).

[Thu06] Marc Thurley. "sharpSAT–counting models with advanced component caching and implicit BCP". In: *Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT)*. 2006.

[Tin02] Cesare Tinelli. "A DPLL-based calculus for ground satisfiability modulo theories". In: *European Workshop on Logics in Artificial Intelligence*. 2002.

[Tse83] Grigori S Tseitin. "On the complexity of derivation in propositional calculus". In: *Automation of reasoning*. 1983.

[TV17] Lucas M Tabajara and Moshe Y Vardi. "Factored Boolean functional synthesis". In: *Proc. of FMCAD*. IEEE. 2017.

[TW21] Samuel Teuber and Alexander Weigl. "Quantifying Software Reliability via Model-Counting". In: *Proc. of QEST*. 2021.

[Ver+07] Sven Verdoolaege, Rachid Seghir, Kristof Beyls, Vincent Loechner, and Maurice Bruynooghe. "Counting integer points in parametric polytopes using Barvinok's rational functions". In: *Algorithmica* 1 (2007).

[Wie11] Siert Wieringa. "On Incremental Satisfiability and Bounded Model Checking". In: *Proceedings of the First International Workshop on Design and Implementation of Formal Tools and Systems, Austin, USA, November 3, 2011*. CEUR Workshop Proceedings. 2011.

[Yan+23] Jiong Yang, Arijit Shaw, Teodora Baluta, Mate Soos, and Kuldeep S Meel. "Explaining SAT Solving Using Causal Reasoning". In: *Proceedings of the Theory and Applications of Satisfiability Testing (SAT)*. 2023.

[YM23] Jiong Yang and Kuldeep S Meel. "Rounding Meets Approximate Model Counting". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2023.

[YS02] Håkan L. S. Younes and Reid G. Simmons. "Probabilistic Verification of Discrete Event Systems Using Acceptance Sampling". In: *Proceedings of International Conference on Computer-Aided Verification (CAV)*. 2002.

[ZDT00] Eckart Zitzler, Kalyanmoy Deb, and Lothar Thiele. "Comparison of multiobjective evolutionary algorithms: Empirical results". In: *Evolutionary computation* (2000).

[Zoh+22] Yoni Zohar, Ahmed Irfan, Makai Mann, Aina Niemetz, Andres Nötzli, Mathias Preiner, Andrew Reynolds, Clark Barrett, and Cesare Tinelli. "Bit-precise reasoning via Int-blasting". In: *Proc. of VMCAI*. 2022.